Natural Language Processing

Azhar Sir's Lectures (L1 - L8)

CSE 4107 ? Natural Language Processing

Comprehensive Course Slide Deck with Embedded Navigation & Table of Contents

Table of Contents

1	Lecture 01: Introduction to Natural Language Processing	Page 3
2	Lecture 02: Regular Expressions, Tokenization & Text Normalization	Page 25
3	Lecture 03: Minimum Edit Distance & String Alignment	Page 74
4	Lecture 04: N-gram Language Models	Page 97
5	Lecture 05: Naive Bayes Classification & Sentiment Analysis	Page 161
6	Lecture 06: Logistic Regression for Text Classification	Page 213
7	Lecture 07: Vector Semantics & Word Embeddings	Page 268
8	Lecture 08: Context-Free Grammars & Syntactic Parsing	Page 366

Introduction to NLP

1

DR. K.M. AZHARUL HASAN

NLP


natural
learning
processing
NLP
language
text
automatic
linguistics
interaction
public
processed
understanding
download
process
computer
retrieval
tag
typo
design
discourse
job
analysis
word
communicate
simulation
keywords
telecommunications
output
operating
typography
information
human
systems
input
layout
data
evolution
science
cloud
testing
intelligence
coreference
programming
technology
automated
summarization
evaluation
networks
artificial
statistical
machine
media
connect

Natural Language Processing(NLP)

- **Natural language processing** (**NLP**) is a field of computer science, artificial intelligence and computational linguistics
  - concerned with the interactions between **computers and human natural languages**.
- Mid-1930s, by G. Artsrouni
  - an automatic bilingual dictionary using paper tape.
- The first attempt to use Bangla in computing was made in the early 1980s with Bangla font development in the Windows environment.

NLP vs Text mining

NLP automates things that humans do well, so that they can be done automatically.

But Text mining that's hard even for humans.

Text mining /Computational linguistics can discover underlying patterns in large datasets:
things we didn't know!

NLP

- **Natural Language Processing (NLP)** is a branch of Artificial Intelligence and Computer Science that
  - enables computers to understand, interpret, generate, and interact with human language (text and speech).

- NLP allows computers to
  - process language in a way that is useful for tasks such as answering questions, translating languages, summarizing documents, and recognizing named entities.

NLP Pipeline

- Text collection
- Text preprocessing
  - Tokenization / Sentence segmentation
  - Stop-word removal
  - Stemming/Lemmatization
- Feature representation
  - Bag of Words
  - TF-IDF
  - Word embedding's (Word2Vec, GloVe, FastText)
  - Contextual embedding's (BERT, RoBERTa)
- Model Training
- Evaluation
- Deployment

Why NLP is difficult

- Many hidden variables
  - Knowledge about the language (ins and outs)
  - Knowledge about the world
  - Knowledge about the context of the language.
  - Knowledge about human communication techniques with the language.

    Structure, semantic, tone, scope….
- Problem of scale
  - Many (infinite?) possible words, meanings, context
- Problem of sparsity
  - Very difficult to do statistical analysis, most things (words, concepts) are never seen before
  - Huge computational cost

    Processor speed increased
- Long range correlations between the variables

NLP Tools



- A lot of tools have been published to do natural language processing jobs. Mostly for English..
- OpenNLP
  - The Apache OpenNLP library is a machine learning based toolkit for the processing of natural language text. (https://opennlp.apache.org/)
- Stanford Parser/ Stanford NLP
  - Probabilistic natural language parsers, both highly optimized PCFG* and lexicalized dependency parsers, and a lexicalized PCFG parser; Arabic, Enlish, French, Spanish, Chinese, German ; (https://nlp.stanford.edu/software/)
- ScalaNLP: Natural Language Processing and machine learning.
  - Breeze: Linear algebra, numerics, visualization
  - Nak: Machine Learning
  - Epic: Natural Language Processing and Structured Prediction
  - Junto: Label Propagation
  - Puck: Super-fast GPU parser

NLP Tools(Cont...)

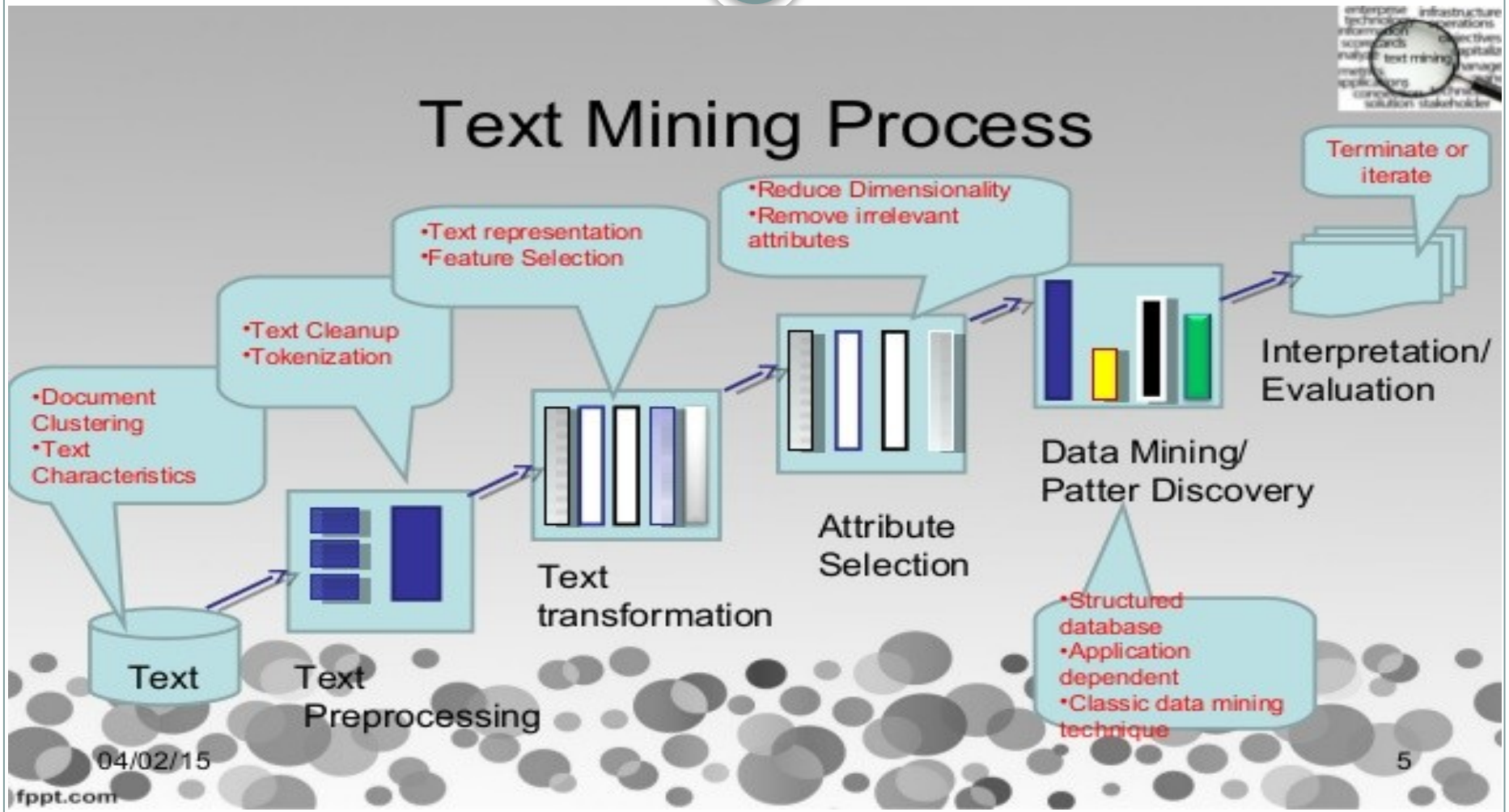


- MALLET:(MAchine Learning for LanguagE Toolkit)
  - A Java-based package for statistical natural language processing, document classification, clustering, topic modeling, information extraction, and other machine learning applications to text. (http://mallet.cs.umass.edu/)
- Apache Lucene Core
  - A library for stop-words removal and stemming (https://lucene.apache.org/)
- Stanford Topic Modelling Toolbox:
  - Modeling tools to perform analysis on datasets that have a substantial textual component. (https://nlp.stanford.edu/software/tmt/tmt-0.4/)
- **Python & NLTK-Natural Language Toolkit (4 or 5)**
  - **NLTK defines an infrastructure that can be used to build NLP programs in Python. It provides basic classes for representing data relevant to natural language processing. (http://www.nltk.org/download)**

Common NLP tasks

Task	Description
Tokenization	Split text into words or subwords
Part-of-Speech Tagging	Assign grammatical categories to words
Named Entity Recognition	Detect entities such as person, organization, disease, and location names
Text Classification	Classify text into predefined categories
Sentiment Analysis	Determine positive, negative, or neutral sentiment
Machine Translation	Translate between languages
Text Summarization	Produce concise summaries
Question Answering	Answer questions from text
Information Extraction	Extract structured information from unstructured text
Relation Extraction	Identify relationships between entities
Text Generation	Generate fluent human-like text

Text mining typical process



Text mining: The theme

Get the *right information*
To the *right people*
At the *right time*
On the *right medium*
In the *right language*
With the *right level of detail*

Relevant Text Mining Technologies

- “…right information”
- “…right people”
- “…right time”
- “…right medium”
- “…right language”
- “…right level of detail”

IR (search engines)
Classification, routing
Real time analysis
Info extraction, speech
Machine translation
Summarization

Language Technology

mostly solved

Spam detection

Let's go to Agra! ✓

Buy V1AGRA ... ✗

Part-of-speech (POS) tagging

Colorless green ideas sleep furiously.

ADJ ADJ NOUN VERB ADV

Named entity recognition (NER)

Einstein met with UN officials in Princeton

PERSON ORG LOC

making good progress

Sentiment analysis

Best roast chicken in San Francisco!

The waiter ignored us for 20 minutes.

Coreference resolution

Carter told Mubarak he shouldn't run again.

Word sense disambiguation (WSD)

I need new batteries for my ***mouse***.



Parsing

I can see Alcatraz from the window!



Machine translation (MT)

第 13 届上海国际电影节开幕…

The 13th Shanghai International Film Festival...

Information extraction

You're invited to our dinner party, Friday May 27 at 8:30

Party May 27 add

still really hard

Question answering (QA)

Q. How effective is ibuprofen in reducing fever in patients with acute febrile illness?

Paraphrase

XYZ acquired ABC yesterday

ABC has been taken over by XYZ

Summarization

The Dow Jones is up

The S&P500 jumped

Housing prices rose

Economy is good

Dialog

Where is Citizen Kane playing in SF?

Castro Theatre at 7:30. Do you want a ticket?

Information Extraction

Subject: **curriculum meeting**

To: az@cse.kuet.ac.bd

Event: Curriculum mtg
Date: Jan-16-2012
Start: 10:30am
End: 11:30am
Where: CSE 301

Dear all,

we've now scheduled the curriculum meeting.
It will be in CSE 301. tomorrow from 10:30-11:30.
-Head

Create new Calendar entry

Information Extraction & Sentiment Analysis



Attributes:

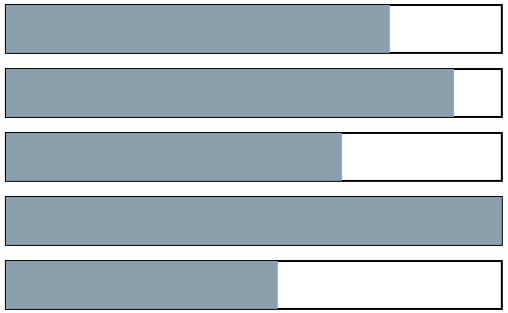
zoom

affordability

size and weight

flash

ease of use



Size and weight

- ✓ nice and compact to carry!
- ✓ since the camera is small and light, I won't need to carry around those heavy, bulky professional cameras either!
- ✗ the camera feels flimsy, is plastic and very light in weight you have to be very delicate in the handling of this camera

Machine Translation

- Fully automatic

Enter Source Text:

这 不过 是 一 个 时间 的 问题 .

Translation from Stanford's *Phrasal*:

This is only a matter of time.

- Helping human translators



Ambiguity is pervasive

New York Times headline (17 May 2000)

Fed raises interest rates

Fed raises interest rates

Fed raises interest rates 0.5%





Assistive computing

- Interfaces for disabled
- Many devices involve language issues, e.g.
  - Text simplification or summarization for users with low literacy (partially sighted, dyslexic, non-native speaker, illiterate, etc.)
  - Text completion (predictive or retrospective)

    Works on basis of probabilities or previous examples

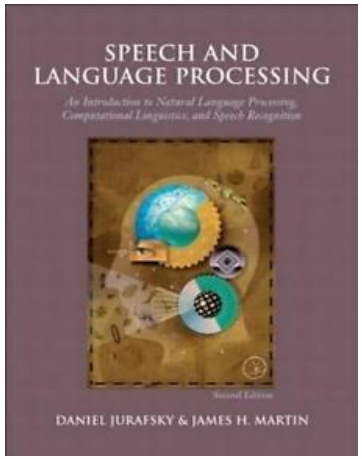
Skills you'll need

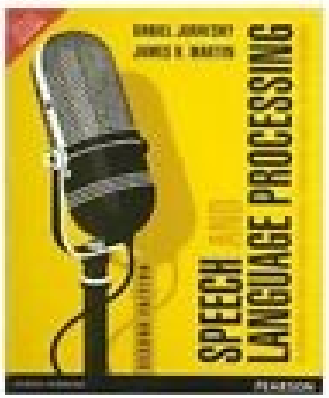


- Simple linear algebra (vectors, matrices)
- Basic probability theory
- Java or Python programming
  - (if you want to implement)

Books and References

- **Speech and Language Processing : An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition**
  - **by James H. Martin and Daniel Jurafsky, Third Edition.**
  - **2nd edition is also available in library.**
  - **Find at https://web.stanford.edu/~jurafsky/slp3/**
- **Lecture slides and materials.**





THANK YOU

Basic Text Processing: Regular Expressions and Text Normalization

1

DR. K.M. AZHARUL HASAN

Corpus

- Corpus linguistics is the compilation and analysis of collections of spoken and written texts
  - as the source of evidence for describing the nature, structure, and use of languages.
- Corpora vary greatly in size and design
  - but most are nowadays in electronic form with purpose to build computer software to support analysis.
- Corpora are often annotated to show grammatical classes, structures and functions.
- Software to analyse grammatical structures or to identify.
- A **corpus** is a large, organized dataset of naturally occurring language, often annotated with linguistic information such as part-of-speech tags, named entities, syntax, or sentiment labels.

Corpus

- Types of Corpus
  - **Raw Corpus**

    Contains only the original text.

    Ex. আজকের আবহাওয়া খুব সুন্দর।
  - **Annotated Corpus**
  - Includes additional linguistic labels.

    Example (POS tagging) :

    আমি/PRON বিশ্ববিদ্যালয়ে/NOUN যাচ্ছি/VERB ।
  - **Parallel Corpus**

    Contains the same text in two or more languages.
  - **Domain-specific Corpus**

    Focused on a particular field.

    Example: Biomedical corpus, Legal corpus

Regular expressions

- A formal language for specifying text strings
- They are particularly useful for searching in texts, when we have a pattern to search.
- A regular expression search function will search through the corpus, returning all texts that match the pattern.
- The corpus can be a single document or a collection.

Regular Expressions: Disjunctions

- The string of characters inside the braces specifies a disjunction of characters to match.

Pattern	Matches
`[wW]oodchuck`	Woodchuck, woodchuck
`[1234567890]`	Any digit

- Ranges `[A-Z]`

Pattern	Matches	
`[A-Z]`	An upper case letter	`Drenched Blossoms`
`[a-z]`	A lower case letter	`my beans were impatient`
`[0-9]`	A single digit	`Chapter 1: Down the Rabbit Hole`

Regular Expressions: Negation in Disjunction

- The pattern /[2-5]/ specifies any one of the characters

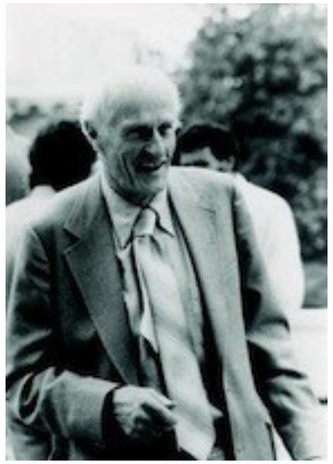
2, 3, 4, or 5. The pattern /[b-g]/ specifies one of the characters b, c, d, e, f, or g.

- Negations `[^Ss]`
  - Carat means negation only when first in [ ]

Pattern	Matches	
`[^A-Z]`	Not an upper case letter	`Oyfn pripetchik`
`[^Ss]`	Neither 'S' nor 's'	`I have no exquisite reason"`
`[^e^]`	Neither e nor ^	`Look here`
`a^b`	The pattern a carat b	`Look up a^b now`

Regular Expressions: ? * + .

Pattern	Matches	
colou?r	Optional previous char	color colour
oo*h!	0 or more of previous char	oh! ooh! oooh! ooooh!
o+h!	1 or more of previous char	oh! ooh! oooh! ooooh!
baa+		baa baaa baaaa baaaaa
beg.n		begin begun begun beg3n



Stephen C Kleene

Kleene *, Kleene +

Regular Expressions: Anchors ^ $

- Anchors are special characters that anchor regular expressions to particular places in a string. The most common anchors are the caret ^ and the dollar sign $.
  - The caret matches the start of a line. The pattern /^The/ matches the word The only at the start of a line
- The caret ^ has three uses:
  - to match the start of a line,
  - to indicate a negation inside of square brackets,
  - and just to mean a caret.

Regular Expressions: Anchors ^ $

- The dollar sign $ matches the end of a line. So the pattern $ is a useful pattern for matching a space at the end of a line
  - /^The dog\.$/ matches a line that contains only the phrase “The dog”.
- There are also two other anchors: \b matches a word boundary, and \B matches a non-boundary.
  - \bthe\b/ matches *the* word the but not the word o*the*r.

Disjunction, Grouping, and Precedence

- The disjunction operator, also called the pipe symbol |. The pattern /cat|dog/ matches **either** the string cat or the string dog.
- To apply disjunction operator only to a specific pattern, use the parenthesis operators (and).
  - the pattern /gupp(y|ies)/ would specify that we meant the disjunction only to apply to the suffixes y and ies.
  - we could write the expression /(Column [0-9]+ *)*/ to match the word Column, followed by a number and optional spaces, the whole pattern repeated any number of times

operator precedence hierarchy

- Parenthesis ()
- Counters * + ? {}
- Sequences and anchors the ^my end$
- Disjunction |

  - Counters have a higher precedence than sequences,/the*/ matches theeeee but not thethe.
  - Sequences have a higher precedence than disjunction, /the|any/ matches the or any but not theny.

Greedy pattern

- Patterns can be ambiguous in another way.
  - The expression /[a-z]*/ when matching against the text once upon a time.
  - Since /[a-z]*/ matches zero or more letters, this expression could match nothing, or just the first letter o, on, onc, or once.
- RE always match the largest string they can
  - Therefore, we say that “patterns are greedy”, expanding to cover as much of a string as they can.
- Ways to make non greedy
  - *? : Kleenee star ->as little text as possible
  - +? : Kleenee plus ->as little text as possible

Example 1

- Find all instances of the word “the” in a text.

/the/	Misses capitalized examples
[tT]he	Incorrectly returns other or theology
/\b[tT]he\b/	some context where it might also have underlines or numbers nearby (the or the25)
/[^a-zA-Z][tT]he[^a-zA-Z]/	won’t find the word the when it begins a line
/(^|[^a-zA-Z])[tT]he([^a-zA-Z]|$)/	

Example 2

Find “ any machine with at least 6 GHz and 500 GB of disk space for less than $1000” Look like the expression “6 GHz or 500 GB or mac or $999.99 "

/$[0-9]+/	regular expression for a dollar sign followed by a string of digits
/$[0-9]+\.[0-9][0-9]/	fractions of dollars; $199.99 but not $199
/(^|\W)$[0-9]+(\.[0-9][0-9])?\b/	a word boundary
/(^|\W)$[0-9]{0,3}(\.[0-9][0-9])?\b/	allows prices like $199999.99, Limit it
/\b[6-9]+ *(GHz|[Gg]igahertz)\b/	specifications for >6GHz processor speed
/\b[0-9]+(\.[0-9]+)? *(GB|[Gg]igabytes?)\b/	For disk space

Errors

- The process we just went through was based on fixing two kinds of errors
  - Matching strings that we should not have matched (there, then, other)
    - False positives (Type I)
  - Do not leave matching that we should match (The)
    - False negatives (Type II)

Errors cont.

- In NLP we are always dealing with these kinds of errors.
- Reducing the error rate for an application involves two efforts:
  - Increasing accuracy or precision (minimizing false positives)
  - Increasing coverage or recall (minimizing false negatives).

Summary

- Regular expressions play a surprisingly large role
  - Sophisticated sequences of regular expressions are the first model for any text processing text
- For many hard tasks, we use machine learning classifiers
  - But regular expressions are used as features in the classifiers
  - Can be very useful in capturing generalizations

Text Normalization

Text Normalization

- Normalizing text means converting it to a more convenient, standard form.
  - Ex. most of what we are going to do with language relies on first separating out or tokenizing words
  - Word boundary detection.
- English words are often separated from each other by whitespace, but whitespace is not always sufficient.
  - New York and rock 'n' roll are sometimes treated as large words despite the fact that they contain spaces,
  - sometimes we'll need to separate I'm into the two words I and am.
  - For processing tweets or texts we need to tokenize emoticons like :) or hashtags like #nlproc

Lemmatization

- A form of text normalization is lemmatization
  - The task of determining that two words have the same root, despite their surface differences.

    For example, the words sang, sung, and sings are forms of the verb sing
- A lemmatizer maps from all of these to sing
  - sang, sung, and sings->sing

Stemming and Segmentation

- Stemming refers to a simpler version of lemmatization in which we mainly just strip suffixes from the end of the word.
- Breaking up a text into individual sentences, using cues like periods or exclamation points.

Text Normalization

Every NLP task needs to do text normalization

1. Segmenting/tokenizing words
2. Normalizing word formats
3. Segmenting sentences in running text

Words

- What counts as a word?
  - Look one particular corpus; computer-readable collection of text or speech
  - Brown corpus is a million-word collection of samples from 500 written English texts from different genres (newspaper, fiction, non-fiction etc.)
  - How many words are in the following Brown sentence?

"He stepped out into the hall, was delighted to encounter a water brother".

- **13** words if we don't count punctuation marks as words, **15** if we count punctuation.
  - Whether we treat period (".”), comma (",”), and so on as words depends on the task.
  - Punctuation is crucial for finding boundaries.

How many words?

- **Types** are the number of distinct words in a corpus;
- **Tokens** are the total number of running words.

They picnicked by the pool, then lay back on the grass and looked at the stars.

- How many?
  - 16 tokens and 14 types

Herdan's Law/ Heaps' Law

- The larger the corpora we look at, the more word types we find
- This relationship between the number of types |V| and number of tokens N is called Herdan's Law

$$|V| = kN^{\beta}$$

where k and β are positive constants, and 0 <β <1.

  - The value of β depends on the corpus size and the genre, β ranges from 0.67 to 0.75
  - V for a text goes up significantly faster than the square root of its tokens.

How many words?

N = number of tokens

V = vocabulary = set of types

$|V|$ is the size of the vocabulary

Church and Gale (1990): $|V| > O(N^{½})$

	Tokens = N	**Types = |V|**
Switchboard phone conversations	2.4 million	20 thousand
Shakespeare	884,000	31 thousand
Google N-grams	1 trillion	13 million

Tokenization: Byte Pair Encoding

Byte Pair Encoding



Instead of

- white-space segmentation or
- single-character segmentation

- **Use the data** to tell us how to tokenize.
- **Subword tokenization**
  - because tokens can be parts of words as well as whole words

Subword tokenization

- Three common algorithms:
  - **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
  - **Unigram language modeling tokenization** (Kudo, 2018)
  - **WordPiece** (Schuster and Nakajima, 2012)
- All have 2 parts:
  - A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
  - A token **segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary

Byte Pair Encoding (BPE) token learner

Let vocabulary be the set of all individual characters

= {A, B, C, D,…, a, b, c, d….}

- Repeat:
  - Choose the two symbols that are most frequently adjacent in the training corpus (say 'A', 'B')
  - Add a new merged symbol 'AB' to the vocabulary
  - Replace every adjacent 'A' 'B' in the corpus with 'AB'.
- Until *k* merges have been done.

BPE token learner algorithm



function BYTE-PAIR ENCODING(strings C, number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters
for $i = 1$ **to** k **do** # merge tokens til k times
  $t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C
  $t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating
  $V \leftarrow V + t_{NEW}$ # update the vocabulary
  Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus
return V

Byte Pair Encoding (BPE) Addendum



Most subword algorithms are run inside space-separated tokens.

So we commonly first add a special end-of-word symbol '__' before space in training corpus

Next, separate into letters.

BPE token learner



Original (very fascinating🙄) corpus:

low low low low low lowest lowest newer newer newer newer newer newer wider wider wider new new

Add end-of-word tokens, resulting in this vocabulary:

vocabulary

_, d, e, i, l, n, o, r, s, t, w

BPE token learner



corpus

```
5   l o w _
2   l o w e s t _
6   n e w e r _
3   w i d e r _
2   n e w _
```

vocabulary

```
_, d, e, i, l, n, o, r, s, t, w
```

Merge e r to er

corpus

```
5   l o w _
2   l o w e s t _
6   n e w er _
3   w i d er _
2   n e w _
```

vocabulary

```
_, d, e, i, l, n, o, r, s, t, w, er
```

BPE



corpus	vocabulary
5 l o w _	_, d, e, i, l, n, o, r, s, t, w, er
2 l o w e s t _	
6 n e w er _	
3 w i d er _	
2 n e w _	

Merge er _ to er_

corpus	vocabulary
5 l o w _	_, d, e, i, l, n, o, r, s, t, w, er, er_
2 l o w e s t _	
6 n e w er_	
3 w i d er_	
2 n e w _	

BPE



corpus	vocabulary
5 l o w _	_, d, e, i, l, n, o, r, s, t, w, er, er_
2 l o w e s t _	
6 n e w er_	
3 w i d er_	
2 n e w _	

Merge n e to ne

corpus	vocabulary
5 l o w _	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne
2 l o w e s t _	
6 ne w er_	
3 w i d er_	
2 ne w _	

BPE



The next merges are:

Merge	Current Vocabulary
(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

Test for “lowest”
l o w e s t
lo
low

BPE token **segmenter** algorithm



On the test data, run each merge learned from the training data:

- Greedily
- In the order we learned them

So: merge every e r to er, then merge er _ to er_, etc.

- Result:
  - Test set "n e w e r _" would be tokenized as a full word
  - Test set "l o w e r _" would be two tokens: "low er_"

Lemma

- A lemma is a set of lexical forms having the same stem, the same major part-of-speech, and the same word sense.
  - Seuss's cat in the hat is different from other cats!

    cat and cats = same lemma
- Word form: the full inflected surface form

  cat and cats = different word forms

Lemmatization and Stemming

- Lemmatization is the task of determining that two words have the same root, despite their surface differences.
  - The words *am, are,* and *is* have the shared lemma *be*;
  - The words *dinner* and *dinners* both have the lemma *dinner*.
- Representing a word by its lemma is important for web search. This is especially important in morphologically complex languages.
  - Ex. *He is reading detective stories* would thus be *He be read detective story*.

How is lemmatization done?

- The most sophisticated methods for lemmatization involve complete morphological parsing of the word.
  - The way the words constructed
  - Morphology is the study of morphemes, the way words are built up from smaller meaning-bearing units called morphemes.
- Two broad classes of morphemes
  - Stems : the central morpheme of the word, supplying the main meaning
  - Affixes : adding “additional” meanings of various kinds (suffix, prefix)

Lemmatization

- Reduce inflections or variant forms to base form
  - *am, are, is → be*
  - *car, cars, car's, cars' → car*
- *The boy's cars are different colors* → *the boy car be different color*
- Lemmatization: have to find the correct dictionary headword form

The Porter Stemmer

- One of the most widely used stemming algorithms is the simple and efficient Porter (1980) algorithm.

ATIONAL -> ATE (e.g., relational->relate)
ING -> ε if stem contains vowel (e.g., motoring!->motor)
SSES -> SS (e.g., grasses -> grass)

Porter's algorithm

Step 1a

sses → ss caresses → caress
ies → i ponies → poni
ss → ss caress → caress
s → ø cats → cat

Step 1b

(*v*)ing → ø walking → walk
sing → sing
(*v*)ed → ø plastered → plaster
…

Step 2 (for long stems)

ational→ ate relational→ relate
izer→ ize digitizer → digitize
ator→ ate operator → operate
…

Step 3 (for longer stems)

al → ø revival → reviv
able → ø adjustable → adjust
ate → ø activate → activ
…

Issues in Tokenization

Some exceptions to handle

- Finland's capital → Finland Finlands Finland's ?
- what're, I'm, isn't → What are, I am, is not
- Hewlett-Packard → Hewlett Packard ?
- state-of-the-art → state of the art ?
- Lowercase → lower-case lowercase lower case ?
- San Francisco → one token or two?
- m.p.h., Ph.D. → ??
- $44.55 → prices ?
- 01/02/06 → dates ?
- abc@xyz, www.kuet.ac.bd/cse → Email and urls?

Case folding

- Case folding: Another form of normalization
  - reduce all letters to lower case
- Applications like IR from text
  - Since users tend to use lowercase
  - Possible exception: upper case in mid-sentence?

    e.g., ***General Motors***

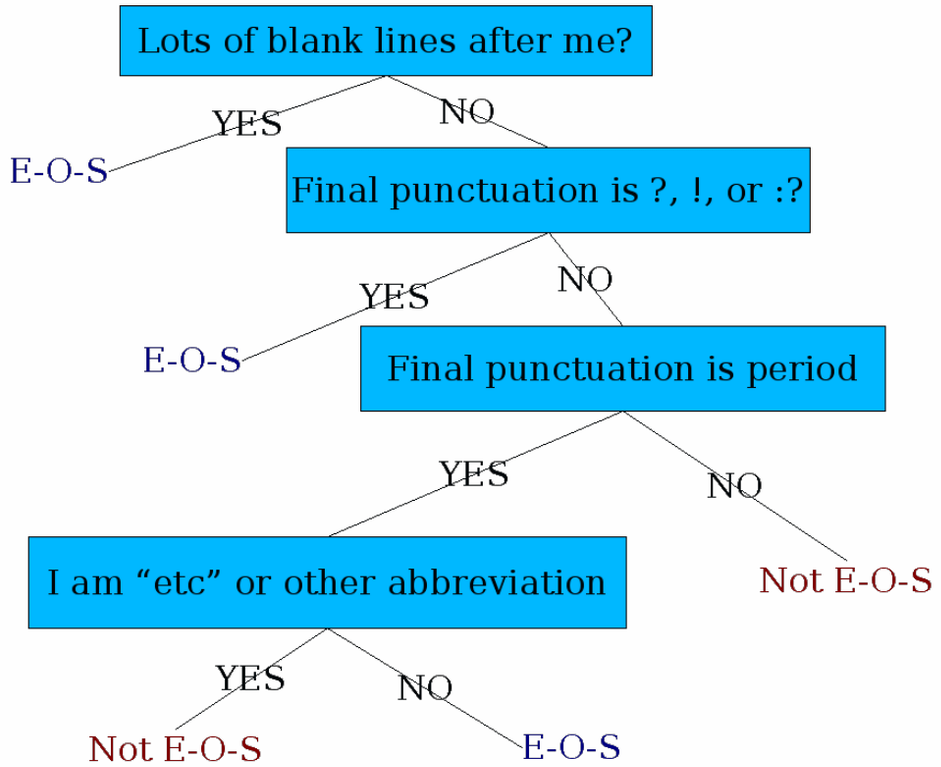
    US vs. us

    SAIL vs. ***sail***
- For sentiment analysis, MT, Information extraction
  - Case is helpful

Sentence Segmentation

- !, ? are relatively unambiguous
- Period "." is quite ambiguous
  - Sentence boundary
  - Abbreviations like Inc. or Dr.
  - Numbers like .02% or 4.3
- Build a binary classifier
  - Looks at a "."
  - Decides EndOfSentence/NotEndOfSentence
  - Classifiers: hand-written rules, regular expressions, or machine-learning

Determining if a word is end-of-sentence: a Decision Tree



THANK YOU

Minimum Edit Distance

1

DR. K.M. AZHARUL HASAN

Edit Distance

- Edit Distance
  - is a measure of similarity between two strings.
  - is defined as the minimum number of changes required *to convert string a into string b*
  - the smaller the distance, the more similar the strings are.

The Levenshtein distance algorithm

- The Levenshtein distance is a string metric for measuring the difference between two sequences. The Levenshtein distance between two words is the minimum number of single-character edits required to change one word into the other.
- In computer science, Levenshtein Distance is a method used to replace misspelled words with correct dictionary words by calculating the minimum number of single-character edits to transform one word into another.

How similar are two strings?

- Spell correction
  - The user typed “graffe”.Which is closest?
    graf
    graft
    grail
    giraffe

- Applications
  - Machine Translation,
  - Information Extraction,
  - Speech Recognition

Edit Distance

- The minimum edit distance between two strings
- Is the minimum number of editing operations
  - Insertion
  - Deletion
  - Substitution
- Needed to transform one into the other

Minimum Edit Distance

- Two strings and their **alignment**

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
```

Minimum Edit Distance

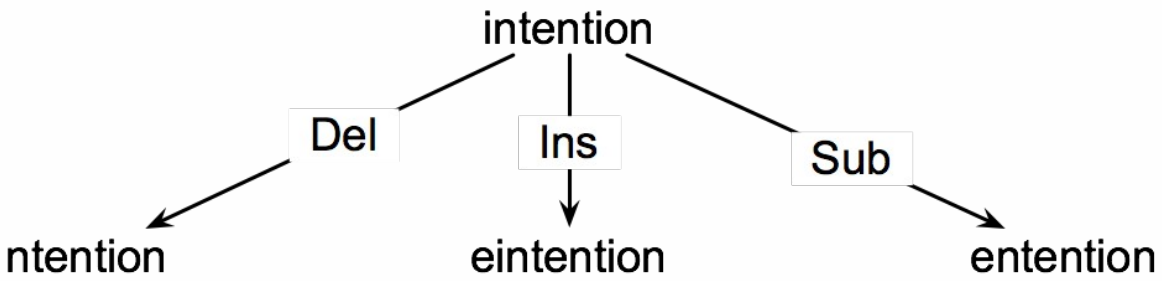

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N

d s s   i s
```

- If each operation has cost of 1
  - Distance between these is 5
- If substitutions cost 2 (Levenshtein)
  - Distance between them is 8

How to find the Min Edit Distance?

- Searching for a path (sequence of edits) from the start string to the final string:
  - **Initial state**: the word we're transforming
  - **Operators**: insert, delete, substitute
  - **Goal state**: the word we're trying to get to
  - **Path cost**: what we want to minimize: the number of edits



Minimum Edit as Search

- But the space of all edit sequences is huge!
  - We can't afford to navigate naïvely
  - Lots of distinct paths wind up at the same state.
    - We don't have to keep track of all of them
    - Just the shortest path to each of those revisited states.

Defining Min Edit Distance

- For two strings
  - X of length n
  - Y of length m
- We define D(i, j)
  - the edit distance between X[1..i] and Y[1..j]

    i.e., the first i characters of X and the first j characters of Y
  - The edit distance between X and Y is thus D(n, m)

Dynamic Programming for Minimum Edit Distance

- **Dynamic programming**: A tabular computation of D(*n*,*m*)
  - Solving problems by combining solutions to sub problems.
- Bottom-up
  - compute *D(i, j)* for small *i*, *j*
  - compute larger *D(i, j)* based on previously computed smaller values

    i.e., compute D(*i*, *j*) for all *i* (0 < *i* < n) and *j* (0 < j < m)

Defining Min Edit Distance (Levenshtein)

- Initialization

  $D(i, o) = i$

  $D(o, j) = j$

- Recurrence Relation:

```
For each  i = 1…M
      For each  j = 1…N
                          ⎧ D(i-1,j) + 1
                          ⎪
          D(i,j)=  min    ⎨ D(i,j-1) + 1
                          ⎪
                          ⎩ D(i-1,j-1) + 2;  ⎧ if X(i) ≠ Y(j)
                                             ⎩ 0;  if X(i) = Y(j)
```

- Termination:

  D(N,M) is distance

The Edit Distance Table: Initialization

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1	2								
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

The Edit Distance Table



N	9	8	9	10	11	12	11	10	9	**8**
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

Computing alignments

- Edit distance isn't sufficient
  - We often need to **align** each character of the two strings to each other
- We do this by keeping a "backtrace"
  - Every time we enter a cell, remember where we came from
- When we reach the end,
  - Trace back the path from the upper right corner to read off the alignment

Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

MinEdit with Backtrace


```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
d s s   i s
```

n	9	↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙←↓ 12	↓ 11	↓ 10	↓ 9	↙ **8**	
o	8	↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↙←↓ 11	↓ 10	↓ 9	↙ **8**	← 9	
i	7	↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9	↙ **8**	← 9	← 10	
t	6	↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙←↓ 9	↙ **8**	← 9	← 10	←↓ 11	
n	5	↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ **8**	↙←↓ 9	↙←↓ 10	↙←↓ 11	↙↓ 10	
e	4	↙ 3	← 4	↙← **5**	← **6**	← 7	←↓ 8	↙←↓ 9	↙←↓ 10	↓ 9	
t	3	↙←↓ 4	↙←↓ **5**	↙←↓ 6	↙←↓ 7	↙←↓ 8	↙ 7	←↓ 8	↙←↓ 9	↓ 8	
n	2	↙←↓ **3**	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙←↓ 8	↓ 7	↙←↓ 8	↙ 7	
i	**1**	↙←↓ 2	↙←↓ 3	↙←↓ 4	↙←↓ 5	↙←↓ 6	↙←↓ 7	↙ 6	← 7	← 8	
#	**0**	1	2	3	4	5	6	7	8	9	
	#	**e**	**x**	**e**	**c**	**u**	**t**	**i**	**o**	**n**	

Adding Backtrace to Minimum Edit Distance

- Base conditions:

  D(i,0) = i
  D(0,j) = j D(N,M) is distance

- Recurrence Relation:

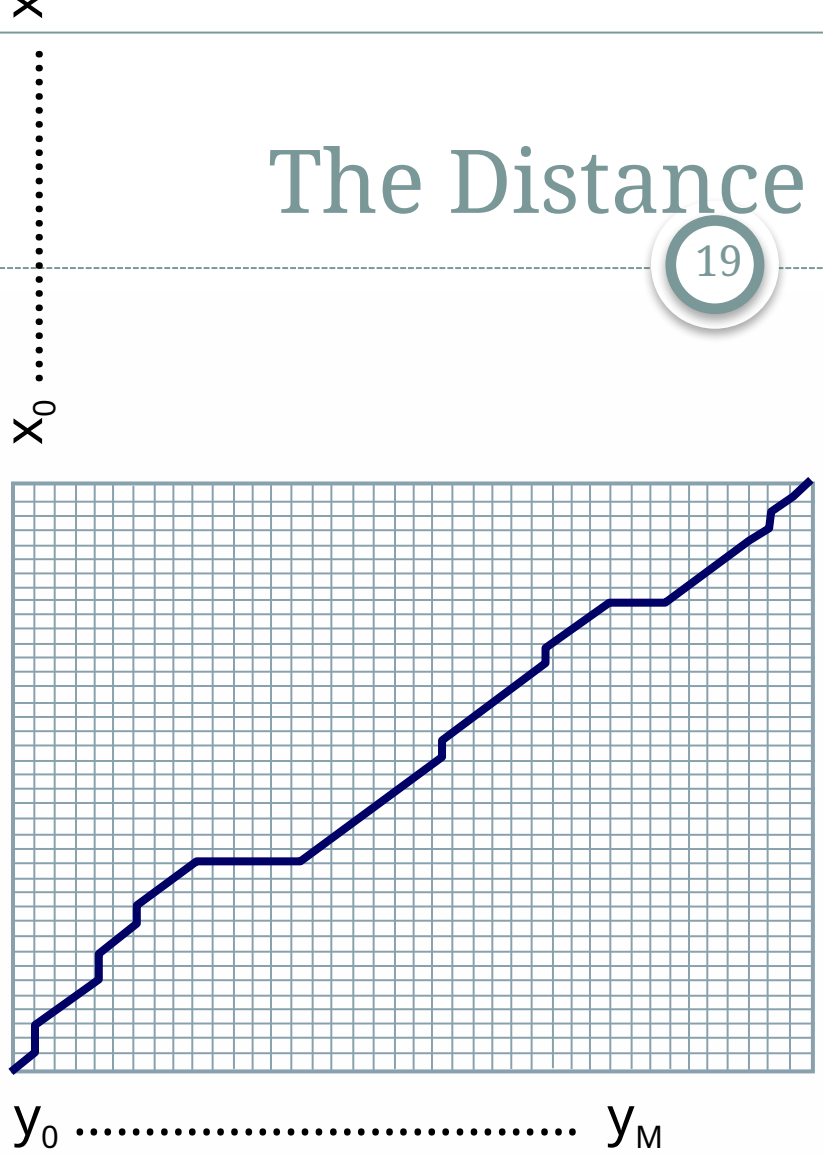
  For each i = 1…M
  For each j = 1…N

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \text{insertion} \\ D(i,j-1) + 1 & \text{deletion} \\ D(i-1,j-1) + 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases}$$

substitution

$$ptr(i,j) = \begin{cases} LEFT & \text{insertion} \\ DOWN & \text{deletion} \\ DIAG & \text{substitution} \end{cases}$$

The Distance Matrix



Every non-decreasing path

from (0,0) to (M, N)

corresponds to an alignment of the two sequences

An optimal alignment is composed of optimal sub alignments

Result of Backtrace

- Two strings and their **alignment**:

```
I N T E * N T I O N
| | | | | | | | | |
* E X E C U T I O N
```

Performance

- Time:

  O(nm)

- Space:

  O(nm)

- Backtrace

  O(n+m)

HT

- Source : abcdef
- Destination: azced

THANK YOU

N-gram Language Models



DR. K.M. AZHARUL HASAN

Basic Probability

Basic Probability

- A coin is tossed and a die is rolled. Find the probability of getting a head on the coin ***and*** a 4 on the die.
- If a coin is tossed twice, find the probability of getting 2 heads.

$$P(H \text{ and } 4) = P(H) \cdot P(4)$$

$$P(H \text{ and } 4) = \frac{1}{2} \cdot \frac{1}{6} = \frac{1}{12}$$

$$P(H \text{ and } H) = P(H) \cdot P(H)$$

$$P(H \text{ and } H) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$$

Basic Probability

- A box contains 3 red balls, 2 blue balls, and 5 white balls. A ball is selected and its color noted. Then it is replaced. A 2nd ball is selected and its color noted. Find the probability of

a. Selecting 2 blue balls

$$P(B, B) = \frac{2}{10} \cdot \frac{2}{10} = \frac{1}{25}$$

b. Selecting a blue ball and then a white ball

$$P(B, W) = \frac{2}{10} \cdot \frac{5}{10} = \frac{1}{10}$$

c. Selecting a red ball and then a blue ball

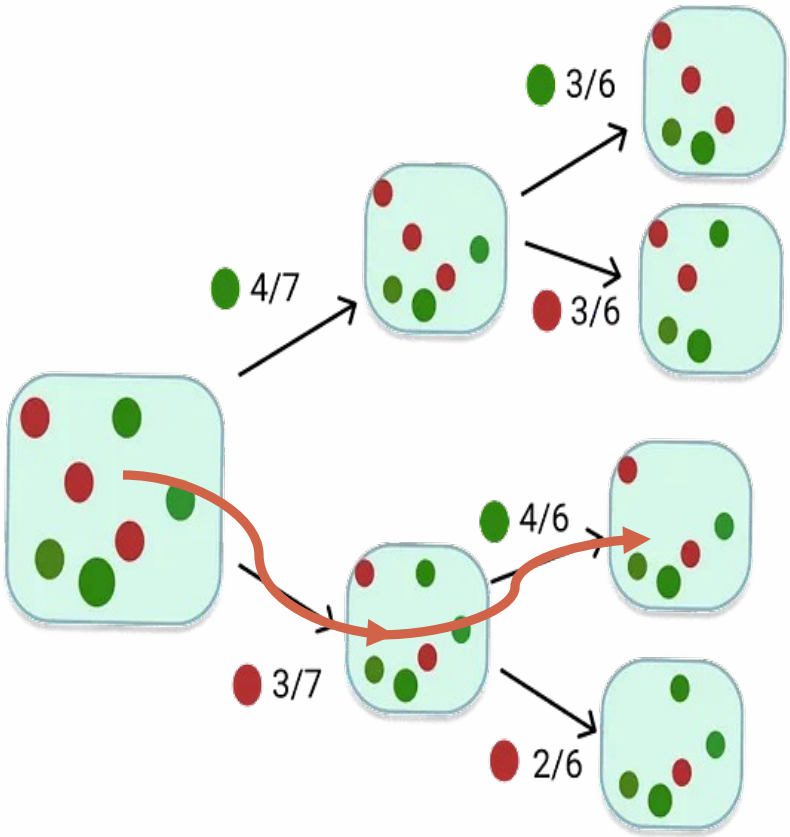
$$P(R, B) = \frac{3}{10} \cdot \frac{2}{10} = \frac{3}{50}$$

Conditional Probability

- The conditional probability of A given B (represented as P(A|B)), is the probability of occurring event A given that event B has occurred.
- The Formula for calculating is P(A|B)=P(A, B) / P(A).

- Ex.: We have 4 green and 3 red balls in a bag. What is the probability of drawing a green ball after a red ball?
  - Event A: Drawing a red ball;
  - Event B: Drawing a green ball.

Basic Conditional Probability



= 3/7 * 4/6

Event A: Drawing a red ball;
Event B: Drawing a green ball.

- The probability of drawing a red ball is P(A)= 3/7,
- The probability of drawing a green ball is P(B)= 4/6
- By the multiplicative rule of independent events

- *P(A and B) =P(A)*P(B) =P(A)*P(B|A) =3/7*4/6 =12/42* ***≈0.2.***

P(A,B) =P(A)*P(B) =P(A)*P(B|A)

=> P(B|A)= P(A,B)/ P(A)

Language Models

- Predicting is difficult
  - especially something that will occur in the future.
- Predicting something that seems much easier,
  - like the next few words someone is going to say

    Ex. Please turn your homework ...
  - Most of you concluded that a very likely word is

    in, or over,

    but not refrigerator

- We will formalize by introducing models that assign a probability to each possible next word.

"Models that assign probabilities to sequences of words are called language model"

N-gram Language Models

- An n-gram is a sequence of N words
  - a 2-gram (or bigram) is a two-word sequence of words
    Ex. “please turn”, “turn your”, or ”your homework”
  - a 3-gram (or trigram) is a three-word sequence of words
    Ex. “please turn your”, or “turn your homework”
  - And so on

N-gram Language Models

This is Big Data AI Book

Uni-Gram	This	Is	Big	Data	AI	Book
Bi-Gram	This is	Is Big	Big Data	Data AI	AI Book	
Tri-Gram	This is Big	Is Big Data	Big Data AI	Data AI Book		

Text	N-gram
Data	1-gram
Great information	2-gram
I am fine	3-gram
Nice to meet you	4-gram

N-gram Language Models

- Application
  - Spell checking
  - Grammar modeling
  - Machine translation
  - Speech recognition
  - Machine human communication
  - Summarization
  - Question-answering
  - Many more…

They are a foundational concept for tasks like text prediction, language modelling, and feature extraction in machine learning models.

N-gram Language Models

- Limitations
  - **Data Sparsity**: Higher-order n-grams (e.g., 4-grams, 5-grams) become sparse, as it's rare to find many long word sequences that repeat in a dataset.
  - **Context Limitations**: N-grams capture only local context and can miss long-distance dependencies between words.
  - **Storage Requirements**: Storing large n-gram datasets requires significant memory, especially with higher-order n-grams.
  - **Zero probability**: The **zero probability problem** occurs when an n-gram that appears in the test data was **never observed in the training corpus**. Since the model estimates probabilities based on observed frequencies, an unseen n-gram is assigned a probability of **0**.

Probabilistic Language Models

- Assign a probability to a sentence
  - Machine Translation:
    - P(**high** winds tonite) > P(**large** winds tonite)
  - Spell Correction
    - The office is about fifteen **minuets** from my house
      - P(about fifteen **minutes** from) > P(about fifteen **minuets** from)
  - Speech Recognition
    - P(I saw a van) >> P(eyes awe of an)
  - Summarization, question-answering, etc…!!

A Probabilistic Language Model (PLM) is a model that assigns a probability to a sequence of words (a sentence). It estimates how likely a sentence is to occur in a language.

Probabilistic Language Modeling

- It computes the probability of a sentence or sequence of words:

  $P(W) = P(w_1,w_2,w_3,w_4,w_5...w_n)$

- Probability of an upcoming word:

  $P(w_5|w_1,w_2,w_3,w_4)$

- A model that computes either of these:

  $P(W)$ or $P(w_n|w_1,w_2...w_{n-1})$ is called a **language model**.

How to compute P(W)

- How to compute this joint probability:
  - P(its water is so transparent that)
- Intuition: It rely on the Chain Rule of Probability

Conditional probability is the **probability** of one event occurring with some relationship to one or more other events.

Reminder: The Chain Rule

- Recall the definition of conditional probabilities

  $P(B|A) = P(A, B)/P(A)$

  Rewriting: $P(A,B) = P(A)P(B|A)$

- More variables:

  $P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$

- The Chain Rule in General

$$P(x_1,x_2,x_3,\ldots,x_n) = P(x_1)P(x_2|x_1)P(x_3|x_1,x_2)\ldots P(x_n|x_1,\ldots,x_{n-1})$$

The Chain Rule

- The Chain Rule applied to compute joint probability of words in a sentence

$$P(w_1, w_2 \ldots) \prod_i P(w_i | w_1 w_2 \ldots w_{i-1})$$

P("its water is so transparent") =

P(its) × P(water|its) × P(is|its water)

× P(so|its water is) × P(transparent|its water is so)

How to estimate these probabilities

- Could we just count and divide?

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{Count(\text{its water is so transparent that the})}{Count(\text{its water is so transparent that})}$$

- Difficult to say Yes/No! Too many possible sentences!
- We'll never see enough data for estimating these

Markov Assumption



Andrei Markov

- Simplifying assumption:

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{that})$$

- Or may be

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{transparent that})$$

The assumption that the probability of a word depends only on the previous word is called Markov assumption

Markov Assumption

$$P(w_1 w_2 \ldots w_n) \approx \prod_i P(w_i \mid w_{i-k} \ldots w_{i-1})$$

- In other words, we approximate each component in the product

$$P(w_i \mid w_1 w_2 \ldots w_{i-1}) \approx P(w_i \mid w_{i-k} \ldots w_{i-1})$$

- Condition on the previous word:

$$P(w_i \mid w_1 w_2 \ldots w_{i-1}) \approx P(w_i \mid w_{i-1})$$

N-gram models

- We can extend to trigrams, 4-grams, 5-grams
- In general this is an insufficient model of language
  - because language has **long-distance dependencies**

    "The computer(s) which I had just put into the machine room on the fifth floor is (are) crashing."

- But we can often get away with N-gram models

Estimating bigram probabilities

- The Maximum Likelihood Estimate

$$P(w_i \mid w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

Ngram: Example(1)

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(\texttt{I} \mid \texttt{<s>}) = \frac{2}{3} = .67 \qquad P(\texttt{Sam} \mid \texttt{<s>}) = \frac{1}{3} = .33 \qquad P(\texttt{am} \mid \texttt{I}) = \frac{2}{3} = .67$$

$$P(\texttt{</s>} \mid \texttt{Sam}) = \frac{1}{2} = 0.5 \qquad P(\texttt{Sam} \mid \texttt{am}) = \frac{1}{2} = .5 \qquad P(\texttt{do} \mid \texttt{I}) = \frac{1}{3} = .33$$

In general case of MLE n-gram parameter estimation

$$P(w_n | w_{n-N+1}^{n-1}) = \frac{C(w_{n-N+1}^{n-1} w_n)}{C(w_{n-N+1}^{n-1})}$$

Ngram: Example(2)

What is the most probable next word predicted by the model for the following w sequence?

Given Corpus

<S> I am Henry </S>

<S> I like college </S>

<S> Do Henry like college </S>

<S> Henry I am </S>

<S> Do I like Henry </S>

<S> Do I like college </S>

<S> I do like Henry </S>

Word	Frequency
<S>	7
</S>	7
I	6
am	2
Henry	5
like	5
college	3
do	4

Ngram: Example(2)

1) <S> Do ?

<S> I am Henry </S>
<S> I like college </S>
<S> Do Henry like college </S>
<S> Henry I am </S>
<S> Do I like Henry </S>
<S> Do I like college </S>
<S> I do like Henry </S>

Word	Frequency
<S>	7
</S>	7
I	6
am	2
Henry	5
like	5
college	3
do	4

Next word prediction probability W_{i-1}=do

$$\frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

Next word	Probability Next Word=
P(</S> | do)	0/4
P(<I> | do)	**2/4**
P(<am>| do)	0/4
P(<Henry>| do)	1/4
P(<like | do)	1/4
P(<college | do)	0/4
P(do | do)	0/4

Ngram: Example(2)

2) <S> I like Henry ?

<S> I am Henry </S>
<S> I like college </S>
<S> Do Henry like college </S>
<S> Henry I am </S>
<S> Do I like Henry </S>
<S> Do I like college </S>
<S> I do like Henry </S>

Word	Frequency
<S>	7
</S>	7
I	6
am	2
Henry	5
like	5
college	3
do	4

Next word prediction probability **W_{i-1}=Henry**

Next word	Probability Next Word= $\frac{N}{D} = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$
P(</S> | Henry)	**3/5**
P(<I> | Henry)	1/5
P(<am>| Henry)	0
P(<Henry>| Henry)	0
P(<like | Henry)	1/5
P(<college | Henry)	0
P(do | Henry)	0

</s> is most probable

Berkeley Restaurant Project sentences

- can you tell me about any good cantonese restaurants close by
- mid priced thai food is what i'm looking for
- tell me about chez panisse
- can you give me a listing of the kinds of food that are available
- i'm looking for a good place to eat breakfast
- when is caffe venezia open during the day

Raw bigram counts

- Out of 9222 sentences (T1)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Raw bigram probabilities

- Normalize by unigrams: (T2)

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Result: count(w_i, w_{i-1})/count(w_{i-1}) i.e dividing T1 by T2 ; 827/2533=.33…

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Bigram estimates of sentence probabilities

P(<s> I want english food </s>) =

P(I|<s>)

× P(want|I)

× P(english|want)

× P(food|english)

× P(</s>|food)

=0.25 × 0.33 × 0.0011 ×0.5×0.68

= .000031

What kinds of knowledge?

- P(english|want) = .0011
- P(chinese|want) = .0065
- P(to|want) = .66
- P(eat | to) = .28
- P(food | to) = 0
- P(want | spend) = 0
- P (i | <s>) = .25

Practical Issues

- We do everything in log space
- probabilities are less than (or equal to 1),
  - more probabilities we multiply together, the smaller the product becomes.
- Avoid underflow
  - also adding is faster than multiplying

$$\log(p_1 \times p_2 \times p_3 \times p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Evaluation: How good is our model?

- Does our language model prefer good sentences to bad ones?
  - Assign higher probability to “real” or “frequently observed” sentences

    than “ungrammatical” or “rarely observed” sentences?
- We train parameters of our model on a **training set**.
- We test the model’s performance on data we haven’t seen.
  - A **test set** is an unseen dataset that is different from our training set, totally unused.
  - An **evaluation metric** tells us how well our model does on the test set.

Evaluating Language Models

- The best way to evaluate the performance of a language model is to
  - embed it in an application and measure how much the application improves.

    Such end-to-end evaluation is called **extrinsic evaluation**.

    very time-consuming

    Extrinsic evaluation is the **only way to know** if a particular issue in a component is really going to help the task at hand.
- It would be nice to have a metric that can be used to quickly evaluate potential intrinsic improvements
  - An **intrinsic evaluation** metric is one that measures the quality of a model independent of any application.
  - We need a **test set**

Training on the test set

- We can't allow test sentences into the training set
  - We will assign it an artificially high probability when we set it in the test set;
  - called **"Training on the test set"**
  - Biased!! And Bad science!
  - violates the honor code

Extrinsic evaluation of N-gram models

- Best evaluation for comparing models A and B
  - Put each model in a task

    spelling corrector, speech recognizer, MT system
  - Run the task, get an accuracy for A and B

    How many misspelled words corrected properly

    How many words translated correctly
  - Compare accuracy for A and B

Difficulty of extrinsic evaluation of N-gram models

- Extrinsic evaluation
  - Time-consuming; can take days or weeks
- So
  - Sometimes use **intrinsic** evaluation: **perplexity**
  - Bad approximation

    unless the test data looks **just** like the training data

    So **generally only useful in pilot experiments**
  - But is helpful to think about.

Intuition of Perplexity

- The Shannon Game:
  - How well can we predict the next word?

    I always order pizza with cheese and ____

    The 33rd President of the US was ____

    I saw a ____

    mushrooms 0.1

    pepperoni 0.1

    anchovies 0.01

    ….

    fried rice 0.0001

    ….

    and 1e-100
  - Unigrams are terrible at this game. (Why?)
- A better model of a text
  - is one which assigns a higher probability to the word that actually occurs

Perplexity

The best language model is one that best predicts an unseen test set

- Gives the highest P(sentence)

Perplexity is the inverse probability of the test set, normalized by the number of words:

$$PP(W) = P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}}$$

$$= \sqrt[N]{\frac{1}{P(w_1 w_2 \ldots w_N)}}$$

Chain rule:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}}$$

For bigrams:

$$\mathrm{PP}(W) = \sqrt[N]{\prod_{i=1}^{N} \frac{1}{P(w_i | w_{i-1})}}$$

Minimizing perplexity is the same as maximizing probability

Perplexity as branching factor

- Let's suppose a sentence consisting of random digits
- What is the perplexity of this sentence according to a model that assign P=1/10 to each digit?

$$\begin{aligned} \mathrm{PP}(W) &= P(w_1 w_2 \ldots w_N)^{-\frac{1}{N}} \\ &= \left(\frac{1}{10}^{N}\right)^{-\frac{1}{N}} \\ &= \frac{1}{10}^{-1} \\ &= 10 \end{aligned}$$

Perplexity

- Training 38 million words, test 1.5 million words, WSJ
- Lower perplexity = better model

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

The perils of over fitting

- N-grams only work well for word prediction if the test corpus looks like the training corpus
  - In real life, it often doesn't look like
  - We need to train robust models that generalize!
  - One kind of generalization: Zeros!

    Things that don't ever occur in the training set
    - But occur in the test set

Zeros

- Training set:
  - … denied the allegations
  - … denied the reports
  - … denied the claims
  - … denied the request

- Test set
  … denied the offer
  … denied the loan

P(“offer” | denied the) = 0

Zero probability bigrams

- Bigrams with zero probability
  - mean that we will assign 0 probability to the test set!
- And hence we cannot compute perplexity (can't divide by 0)!

Unknown words

- If we know all the words in advanced
  - Vocabulary V is fixed
  - Closed vocabulary task
- Often we don't know this
  - **Out Of Vocabulary** = OOV words
  - Open vocabulary task
- Instead: create an unknown word token <UNK>
  - Training of <UNK> probabilities
    - Create a fixed lexicon L of size V
    - At text normalization phase, any training word not in L changed to <UNK>
    - Now we train its probabilities like a normal word
  - At decoding time
    - If text input: Use UNK probabilities for any word not in training

Smoothing

Smoothing

- What do we do with words that exist in our vocabulary
  - they are not unknown words
  - but appear in a test set in an unseen context

    Ex. they appear after a word they never appeared after in training
- To keep a language model from not to assign zero probability to these unseen events
  - we'll have to shave off a bit of probability mass.
- This modification is called smoothing or discounting

Smoothing

- Ways to do smoothing:
  - add-1 smoothing,
  - add-k smoothing,
  - stupid backoff, and
  - Kneser-Ney smoothing.

Smoothing: Add-1 smoothing

- The simplest way to do smoothing is to add one to all the bigram counts
  - All the counts that used to be zero will now have a count of 1, the counts of 1 will be 2, .and so on.
- The unigram probability of the word w_i is its count c_i normalized by the total number of word tokens N
  - $P(w_i) = \frac{c_i}{N}$
- Laplace smoothing merely adds one to each count
  - $P_L(w_i) = (c_i+1)/(N+V)$

Laplace smoothing

- Also called Add-one estimation
  - Probability can not be zero
  - Just add one to all the counts!
- MLE estimate:

$$P_{MLE}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- Add-1 estimate:

$$P_{Add-1}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)+1}{c(w_{i-1})+V}$$

$$P^*(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

Add-k smoothing

- Instead of adding 1 to each count, we add a fractional count k (0.5? 0.05? 0.01?).
- This algorithm is called **add-k** smoothing.
  - $P^*_{add_k}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+k}{C(w_{n-1})+kV}$
- Add-k smoothing requires a method for choosing k

Backoff and Interpolation

- If we are trying to compute $P(w_n|w_{n-2}, w_{n-1})$ but we have no examples of a particular trigram.
- We can instead estimate its probability by using the bigram probability $P(w_n|w_{n-1})$.
- We can look to the unigram $P(w_n)$ to compute bigram.
- Sometimes using less context is a good thing, helping to generalize more contexts

- There are two ways to use this n-gram "hierarchy"
  - **Backoff**

    use trigram if you have good evidence, otherwise bigram, otherwise unigram. It means we only"back off" to a lower-order n-gram if we have zero evidence for a higher-order
  - **Interpolation**

    mix all probability estimates namely unigram, bigram, trigram

  - Ingeneral, Interpolation works better

Linear Interpolation

- In linear interpolation, we combine different order n-grams by linearly interpolating all the models
- To find trigram probability (combine trigram, bigram and unigram)

$$\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1 P(w_n|w_{n-2}w_{n-1}) + \lambda_2 P(w_n|w_{n-1}) + \lambda_3 P(w_n)$$

$$\sum_i \lambda_i = 1$$

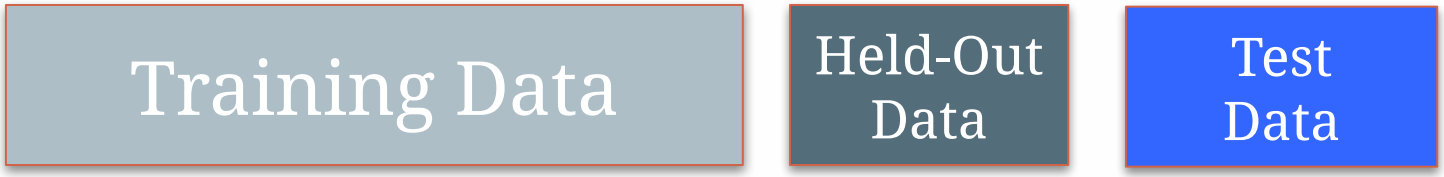
- Lambdas computed conditioning on context

$$\hat{P}(w_n|w_{n-2}w_{n-1}) = \lambda_1(w_{n-2}^{n-1})P(w_n|w_{n-2}w_{n-1}) + \lambda_2(w_{n-2}^{n-1})P(w_n|w_{n-1}) + \lambda_3(w_{n-2}^{n-1})P(w_n)$$

How to set the lambdas?

- Use a **held-out** corpus
  - A held-out corpus is an additional training corpus,

    we hold it out from the training data, that we use to set hyper parameters like λ values



- Choose λ to maximize the probability of held-out data
  - Fix the N-gram probabilities (on the training data)
  - Then search for a value of λ that give largest probability to held-out set

Kneser-Ney Smoothing

- Kneser-Ney has its roots in a method called absolute discounting
- **Kneser–Ney smoothing** is a method primarily used to calculate the probability distribution of *n*-grams in a document based on their histories.
- It is effective method of smoothing
  - discounting by subtracting a fixed value from the probability's lower order terms to omit *n*-grams with lower frequencies

Kneser-Ney Smoothing

- Suppose we wanted to subtract a little from a count of 4 to save probability mass for the zeros
  - How much to subtract ?
  - Church and Gale clever idea
  - Computed bigram for 22 million words of AP Newswire

    Training and held-out set
    for each bigram in the training set
    see the actual count in the held-out set!
  - It sure looks like c* = (c - .75)

Bigram count in training	Bigram count in heldout set
0	.0000270
1	0.448
2	1.25
3	2.24
4	3.23
5	4.21
6	5.23
7	6.21
8	7.21
9	8.26

Absolute Discounting Interpolation

- Save ourselves some time and just subtract 0.75 (or some d)!

discounted bigram

Interpolation weight

$$P_{\text{AbsoluteDiscounting}}(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i) - d}{c(w_{i-1})} + \lambda(w_{i-1})P(w)$$

unigram

- (Maybe keeping a couple of extra values of d for counts 1 and 2)

Kneser-Ney Smoothing I

- Better estimate for probabilities of lower-order unigrams!
  - Shannon game: *I can't see without my reading*_____?
  - "Kong" is more common than "glasses"
  - … but "Kong" always follows "Hong"

Kong *glasses*

- The unigram is useful exactly when we haven't seen this bigram!
- Instead of P(w): "How likely is w"
- $P_{continuation}(w)$: How likely is w to appear as a novel continuation?
  - For each word, count the number of bigram types it completes
  - Every bigram type was a novel continuation the first time it was seen

$$P_{CONTINUATION}(w) \propto \left|\{w_{i-1} : c(w_{i-1}, w) > 0\}\right|$$

Kneser-Ney Smoothing II

- How many times does w appear as a novel continuation

$$P_{CONTINUATION}(w) \propto \left|\{w_{i-1} : c(w_{i-1}, w) > 0\}\right|$$

- Normalized by the total number of word bigram types

$$\left|\{(w_{j-1}, w_j) : c(w_{j-1}, w_j) > 0\}\right|$$

$$P_{CONTINUATION}(w) = \frac{\left|\{w_{i-1} : c(w_{i-1}, w) > 0\}\right|}{\left|\{(w_{j-1}, w_j) : c(w_{j-1}, w_j) > 0\}\right|}$$

Kneser-Ney Smoothing III

- The number of word types seen to precede

(Types: number of distinct words)

$$|\{w_{i-1} : c(w_{i-1}, w) > 0\}|$$

- normalized by the # of words preceding all words:

$$P_{CONTINUATION}(w) = \frac{\left|\{w_{i-1} : c(w_{i-1}, w) > 0\}\right|}{\sum_{w'} \left|\{w'_{i-1} : c(w'_{i-1}, w') > 0\}\right|}$$

- A frequent word (Kong) occurring in only one context (Hong) will have a low continuation probability

Kneser-Ney Smoothing IV

$$P_{KN}(w_i \mid w_{i-1}) = \frac{\max(c(w_{i-1}, w_i) - d, 0)}{c(w_{i-1})} + \lambda(w_{i-1}) P_{CONTINUATION}(w_i)$$

λ is a normalizing constant; the probability mass we've discounted

$$\lambda(w_{i-1}) = \frac{d}{c(w_{i-1})} \left| \{w : c(w_{i-1}, w) > 0\} \right|$$

the normalized discount

The number of word types that can follow w_{i-1}
= # of word types we discounted
= # of times we applied normalized discount

Huge web-scale n-grams

- How to deal with large corpus?? e.g., *Google N-gram* corpus
- Pruning
  - Only store N-grams with count > threshold.
    Remove singletons of higher-order n-grams
  - prune less important n-grams

Smoothing for Web-scale N-grams

- "Stupid backoff" (Brants *et al.* 2007)
  - No discounting, just use relative frequencies
  - If higher order n-grams has zero count, move to lower order n-gram weighted by a fixed weight λ
  - λ = 0.4 works well

$$S(w_i \mid w_{i-k+1}^{i-1}) = \begin{cases} \dfrac{\text{count}(w_{i-k+1}^{i})}{\text{count}(w_{i-k+1}^{i-1})} & \text{if } \text{count}(w_{i-k+1}^{i}) > 0 \\ \lambda S(w_i \mid w_{i-k+2}^{i-1}) & \text{otherwise} \end{cases}$$

Unigram probability

$$S(w_i) = \frac{\text{count}(w_i)}{N}$$

N-gram Smoothing Summary

- Add-1 smoothing:
  - good for text categorization, not for language modeling
- The most commonly used method:
  - Extended Interpolated Kneser-Ney
- For very large N-grams like the Web
  - Stupid backoff

THANK YOU

Text Classification and Naïve Bayes

1

Dr. K. M. Azharul Hasan
Dept. of CSE, KUET

Is this spam?

2

Subject: **Important notice!**
From: Stanford University <newsforum@stanford.edu>
Date: October 28, 2011 12:34:16 PM PDT
To: undisclosed-recipients:;

Greats News!

You can now access the latest news by using the link below to login to Stanford University News Forum.

http://www.123contactform.com/contact-form-StanfordNew1-236335.html

Click on the above link to login for more information about this new exciting forum. You can also copy the above link to your browser bar and login for more information
about the new services.

© Stanford University. All Rights Reserved.

Positive or negative movie review?



- unbelievably disappointing
- Full of zany characters and richly applied satire, and some great plot twists
- this is the greatest screwball comedy ever filmed
- It was pathetic. The worst part about it was the boxing scenes.

What is the subject of this article?

4

MeSH Subject Category Hierarchy

MEDLINE Article



Brain and Cognition

Syntactic frame and verb bias in aphasia: Plausibility judgments of undergoer-subject sentences

?

- Antogonists and Inhibitors
- Blood Supply
- Chemistry
- Drug Therapy
- Embryology
- Epidemiology
- …

Google Product Search

5



HP Officejet 6500A Plus e-All-in-One Color Ink-jet - Fax / copier / printer / scanner
$89 online, **$100** nearby ★★★★☆ 377 reviews
September 2010 - Printer - HP - Inkjet - Office - Copier - Color - Scanner - Fax - 250 sh

Reviews

Summary - Based on 377 reviews

1 star	2	3	4 stars	5 stars

What people are saying

ease of use	"This was very easy to setup to four computers."
value	"Appreciate good quality at a fair price."
setup	"Overall pretty easy setup."
customer service	"I DO like honest tech support people."
size	"Pretty Paper weight."
mode	"Photos were fair on the high quality mode."
colors	"Full color prints came out with great quality."

Text Classification

- Text classification: the task of assigning a classification label or category to a text or document.
  - Assigning subject categories, topics, or genres
  - Spam detection
  - Authorship identification
  - Age/gender identification
  - Language Identification
  - Sentiment analysis
  - …

Text Classification: definition

- *Input*:
  - a document *d*
  - a fixed set of classes $C = \{c_1, c_2, \ldots, c_k\}$

- *Output*: a predicted class for *d* $c \in C$

Classification Methods: Hand-coded rules

- Rule based on combinations of words or other features
  - spam: black-list-address OR (“dollars” AND“ have been selected”)
- Accuracy can be high
  - If rules carefully refined by expert
- But building and maintaining these rules is expensive

Spam email features

- Unusual Domain Names
- Generic Greetings, Ex. Dear Customer
- Urgency or Fear Tactics: Phrases like "Act Now," "Limited Time Offer," or "Your Account Will Be Deactivated.“
- Unusual Attachments:
- Poor Grammar and Spelling: Many spam emails contain obvious language errors.
- Promotional or Unsolicited Offers: Advertising products or services, often of dubious nature.
- Request for Sensitive Information: Asking for passwords, credit card numbers, or other personal data.
- Large Use of Capital Letters or Excessive Punctuation
- High Email Volume
- Irrelevant context.
- Use of randomized text or symbols to bypass spam filters.
- Lack of Unsubscribe Options

Classification Methods: Supervised Machine Learning

- Supervised machine learning is defined by its use of labeled datasets to train algorithms that classify data or predict outcomes accurately.
- *In the context of Text classification*
- *Input:*
  - a document d
  - a fixed set of classes $C = \{c_1, c_2, \ldots, c_J\}$
  - A training set of m hand-labeled documents $(d_1,c_1),\ldots,(d_m,c_m)$
- *Output:*
  - a learned classifier $\gamma: d \rightarrow c$

Generative and Discriminative Classifiers

Suppose we're distinguishing cat from dog images





A **generative model** would have the goal of understanding what dogs look like and what cats look like

A **discriminative model**, by contrast, is only trying to learn to distinguish the classes (perhaps without learning much about them).

Generative Classifier:

- Build a model of what's in a cat image
  - Knows about whiskers, ears, eyes
  - Assigns a probability to any image:
    - how cat-y is this image?





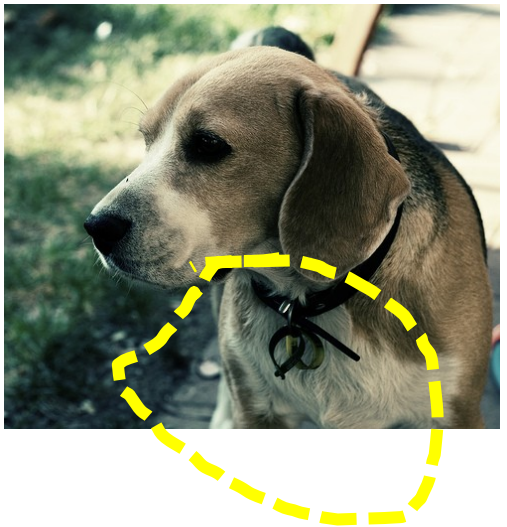
Also build a model for dog images

Now given a new image:
Run both models and see which one fits better

Discriminative Classifier

Just try to distinguish dogs from cats





Oh look, dogs have collars!
Let's ignore everything else

Generative vs Discriminative classifier

- A probabilistic classifier will tell us the probability of an observation being in the class C.
- Two ways
  - Generative classifiers (Ex. Naive Bayes)

    build a model of each class.

    given an observation, they return the class most likely to have generated the observation.
  - Discriminative classifiers (Ex. Logistic regression)

    learn what features from the input are most useful to discriminate between the different possible classes.

15

Text Classification and Naïve Bayes

Naïve Bayes

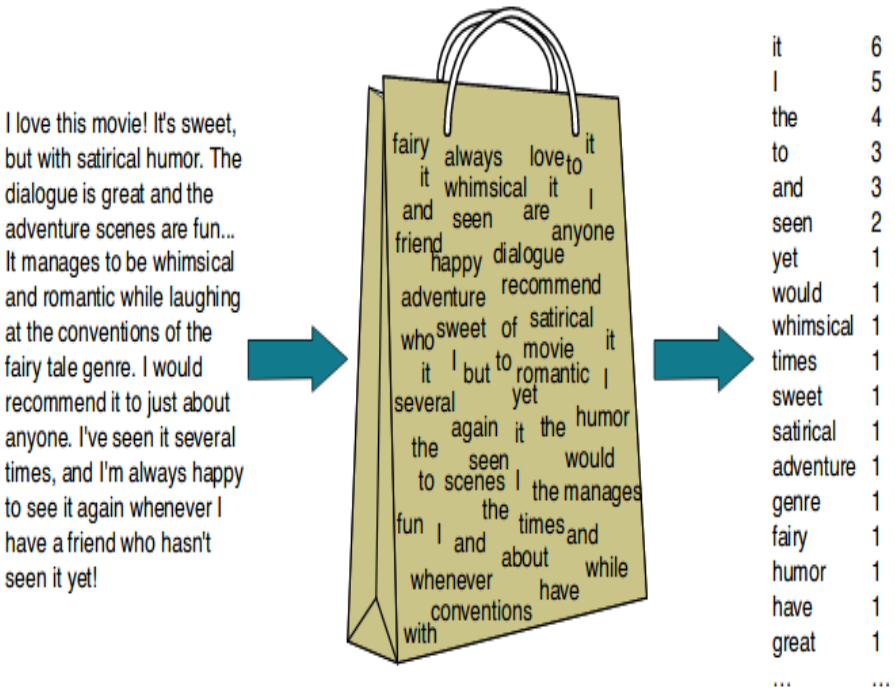
- Simple (“naïve”) classification method based on Bayes rule
- Called multinomial naive Bayes classifier
  - because it is a Bayesian classifier that makes a simple assumption about how the features interact
- Relies on a very simple representation of a document
  - Bag of words: an unordered set of words with their position ignored, keeping only their frequency in the document

Multinomial Naive Bayes

- **Multinomial Naive Bayes** is a probabilistic classifier to calculate the probability distribution of text data,
  - makes it well-suited for data with features that represent discrete frequencies or counts of events in various NLP tasks.
- The term "multinomial" refers to the type of data distribution assumed by the model.
  - The features are word counts or term frequencies.
  - The multinomial distribution is used to estimate the likelihood of a specific set of word counts in a document.

Bag of Words

- The bag-of-words : a statistical language model used to analyze text and documents based on word count.
- a text is represented as the bag of its words,
  - disregarding grammar and even word order but keeping multiplicity.
- any information about the order or structure of words in the document is discarded.



The bag of words representation



Bayes' Rule Applied to Documents and Classes

- Naive Bayes is a probabilistic classifier, meaning that
  - for a document d, out of all classes c ∈C the classifier returns the class c_{MAP} which has the maximum posterior probability for the document.
- For a document *d* and a class *c*

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

Naïve Bayes Classifier (I)

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(c \mid d)$$

MAP is "maximum a posteriori" = most likely class

$$= \underset{c \in C}{\operatorname{argmax}} \frac{P(d \mid c)P(c)}{P(d)}$$

Bayes Rule

$$= \underset{c \in C}{\operatorname{argmax}} P(d \mid c)P(c)$$

Dropping the denominator

P(d) doesn't change for each class; we are always asking about the most likely class for the same document d, which must have the same probability *P(d)*; hence *P(d)* is dropped.

This is why Naïve Bayes is a generative model because the equ. tells a kind of implicit assumption about how a document is generated

Naïve Bayes Classifier (II)

Posterior probability = highest product of two probabilities: the prior probability of the class P(c) and the likelihood of the document P(d|c):

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(d|c)P(c)$$

$$= \underset{c \in C}{\operatorname{argmax}} P(f_1, f_2, \ldots, f_n|c)P(c)$$

Document d represented as features *f1,f2,…,fn*

the probability of every possible combination of features (for example, every possible set of words and positions)

Multinomial Naïve Bayes Independence Assumptions

Naive Bayes classifier makes two simplifying assumptions. $P(f_1, f_2, \ldots, f_n|c)$

- **Bag of Words assumption**: Assume position doesn't matter.
  - Ex. the word "love" has the same effect on classification whether it occurs as the 1st, 20th, or last word in the document

- **Conditional Independence**: Assume the feature probabilities $P(x_i|c_j)$ are independent given the class c and can be multiplied.

$$P(f_1, \ldots, f_n|c) = P(f_1|c) \bullet P(f_2|c) \bullet P(f_3|c) \bullet \ldots \bullet P(f_n|c)$$

Multinomial Naïve Bayes Classifier

The final equation for the class chosen by a naive Bayes classifier is thus:

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(f_1, f_2, \ldots, f_n | c) P(c)$$

$$c_{NB} = \underset{c \in C}{\operatorname{argmax}}\, P(c) \prod_{f \in F} P(f | c)$$

Naïve Bayes: Learning

Learning the Multinomial Naïve Bayes Model

- How can we learn the probabilities *P(c)* and *P(fi/c)(or P(wi/c))*?
  - first consider the maximum likelihood estimate.
  - use the frequencies in the data.
  - For the document prior *P(c)*

    what percentage of the documents in our training set are in each class c.

    let N_c be the number of words/documents in our training data with class c and N_{doc} be the total number of documents/words.

$$\hat{P}(c) = \frac{N_c}{N_{doc}}$$

Learning the Multinomial Naïve Bayes Model

- To learn the probability P(fi/c) or *P(wi/c)*,
  - We assume a feature, the existence of a word, in the document's bag of words
  - *P(wi/c)* computes the fraction of times the word w_i appears among all words in all documents of topic c.
  - We first concatenate all documents with category c into one big "category c" text.

Learning the Multinomial Naïve Bayes Model

- Then we use the frequency of w_i in this concatenated document to give a maximum likelihood estimate of the probability:

$$\hat{P}(c_j) = \frac{doccount(C = c_j)}{N_{doc}}$$

$$\hat{P}(w_i \mid c_j) = \frac{count(w_i, c_j)}{\sum_{w \in V} count(w, c_j)}$$

  - Here the vocabulary V consists the union of all the word types in all classes, not just the words in one class c.

Parameter estimation

$$\hat{P}(w_i \mid c_j) = \frac{count(w_i, c_j)}{\sum_{w \in V} count(w, c_j)}$$

fraction of times word w_i appears among all words in documents of topic c_j

Problem with Maximum Likelihood

What if we have seen no training documents containing the word "fantastic?

$$\hat{P}(\text{"fantastic"}|\text{positive}) = \frac{count(\text{"fantastic"}, \text{positive})}{\sum_{w \in V} count(w, \text{positive})} = 0$$

Since naive Bayes naively multiplies all the feature likelihoods together.
Zero probabilities cannot be conditioned away, no matter the other evidence!

Laplace (add-1) smoothing for Naïve Bayes

- The simplest solution is the add-one (Laplace) smoothing

$$\hat{P}(w_i \mid c) = \frac{count(w_i, c) + 1}{\sum_{w \in V} (count(w, c) + 1)}$$

$$= \frac{count(w_i, c) + 1}{\left(\sum_{w \in V} count(w, c) \right) + |V|}$$

unknown words

- What do we do about words that occur in our test data but are not in our vocabulary

  because they did not occur in any training document in any class?

  - standard solution for such unknown words is to ignore such words—remove them from the test document and not include any probability for them at all.

Stop words

- Some systems choose to completely ignore another class of words: stop words, very frequent words like the and a.
  - Sort the vocabulary by frequency in the training set, and defining the top 10–100 vocabulary entries as stop words, or
  - alternatively by using one of the many pre-defined stop word list
- Every instance of stop words are removed from both training and test documents as if they had never occurred.
  - using a stop word list doesn't improve performance,
- Common to make use of the entire vocabulary and not use a stop word list.

Multinomial Naïve Bayes: A Worked Example

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

The prior P(c) for the two classes (+ and -) is computed via

$$\hat{P}(c) = \frac{N_c}{N_{doc}}$$

Worked Example

$$P(-) = \frac{3}{5} \qquad P(+) = \frac{2}{5}$$

The word **with** doesn't occur in the test set, so we drop it completely

The likelihoods from the training set for the remaining three words "predictable", "no", and "fun", are as

$$P(w/c) = \frac{count(w_i, c) + 1}{(\sum_{w \in V} count(w, c)) \; + \; |V|}$$

$$P(\text{“predictable”}|-) = \frac{1+1}{14+20} \qquad P(\text{“predictable”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“no”}|-) = \frac{1+1}{14+20} \qquad P(\text{“no”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“fun”}|-) = \frac{0+1}{14+20} \qquad P(\text{“fun”}|+) = \frac{1+1}{9+20}$$

Worked Example

For the test sentence S = “predictable with no fun”, after removing the word ‘with’, the chosen class

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$

$$P(+)P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

The model thus predicts the class negative for the test sentence.

Another Example

$$\hat{P}(c) = \frac{N_c}{N}$$

$$\hat{P}(w \mid c) = \frac{count(w,c)+1}{count(c)+|V|}$$

	Doc	Words	Class
Training	1	Chinese Beijing Chinese	c
	2	Chinese Chinese Shanghai	c
	3	Chinese Macao	c
	4	Tokyo Japan Chinese	j
Test	5	Chinese Chinese Chinese Tokyo Japan	?

Priors:

$P(c)$=3/4

$P(j)$= 1/4

Conditional Probabilities:

P(Chinese | c) = (5+1) / (8+6) = 6/14 = 3/7

P(Tokyo | c) = (0+1) / (8+6) = 1/14

P(Japan | c) = (0+1) / (8+6) = 1/14

P(Chinese | j) = (1+1) / (3+6) = 2/9

P(Tokyo | j) = (1+1) / (3+6) = 2/9

P(Japan | j) = (1+1) / (3+6) = 2/9

Choosing a class:

P(c | d5) $\propto$ 3/4 * $(3/7)^3$ * 1/14 * 1/14 $\approx$ 0.0003

P(j | d5) $\propto$ 1/4 * $(2/9)^3$ * 2/9 * 2/9 $\approx$ 0.0001

Deal with negation

- Consider the difference between "*I really like this movie (positive) and I didn't like this movie (negative)*".
  - The negation expressed by didn't completely alters the inferences we draw from the predicate like
  - Similarly, negation can modify a negative word to produce a positive review

    Ex. don't dismiss this film, doesn't let us get bored
- Add the prefix NOT to every word after a token of logical negation (n't, not, no, never) until the next punctuation mark

  "*didn't like this movie , but I*" => "*didn't NOT_like NOT_this NOT_movie , but I*"

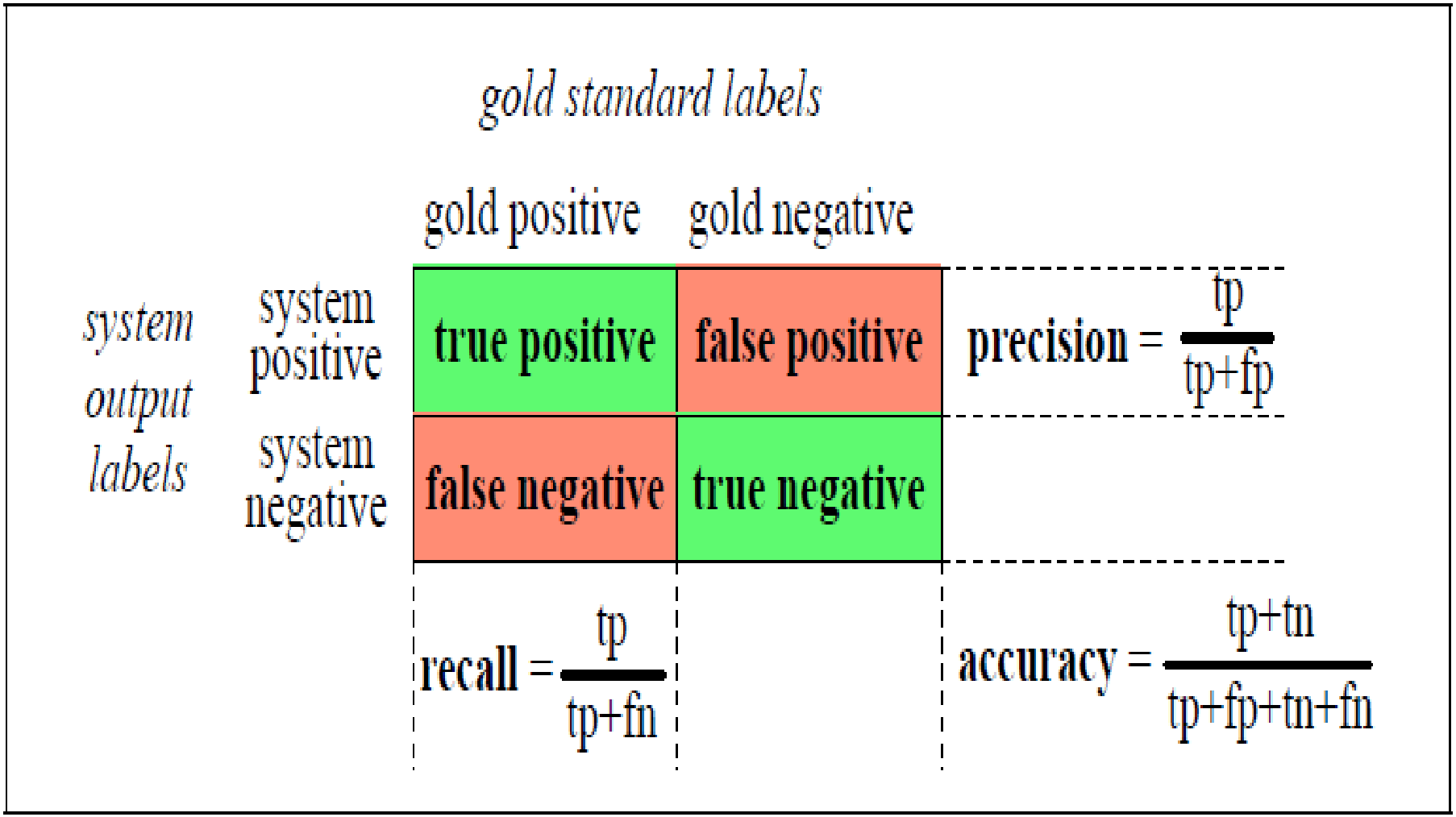
TEXT CLASSIFICATION: EVALUATION

Precision, Recall and F measure

Text Classification: Evaluation

- An email (in spam detection) is actually spam or not,
  - the human-defined labels for each document that we are trying to match.
- We will refer to these human labels as the gold labels.
- We need a metric for knowing how well our spam detector is doing.
- To evaluate any system for detecting things, we start by building a contingency table

Contingency table



The 2-by-2 contingency table

	Gold true	Gold false
System result	TP	FP
	FN	TN

True Positive(*TP*): The truly detected candidate words.
False Positive(*FP*): The falsely detected words which are not candidate.
True Negative(*TN*): The truly detected non candidate.
False Negative(*FN*): The falsely detected words which are not candidate.

Accuracy

- Accuracy, which asks what percentage of all the observations our system labeled correctly.

$$accuracy = (TP + TN) / (TP + TN + FP + FN) \times 100\%$$

  - Although accuracy might seem a natural metric, we generally don't use it always.
- Accuracy is not a good metric when
  - the goal is to discover something that is rare, or
  - at least not completely balanced in frequency, which is a very common situation in the world.

Precision and recall

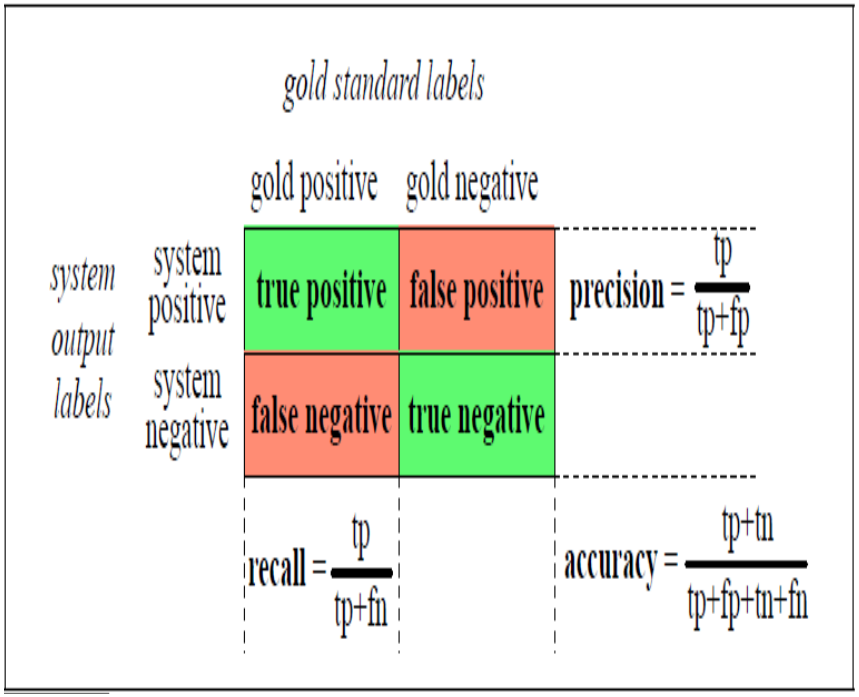
- **Precision**: % of selected items that are correct

  Precision P = tp/(tp + fp)

- **Recall**: % of correct items that are selected

  Recall R = tp/(tp + fn)

- There is typically a trade-off between precision and recall
  - To get high precision, be very reluctant to make guesses – but then you may have poor recall
  - To get high recall, be very promiscuous in making guesses – but then you may have poor precision



A combined measure: F

- A combined measure (weighted harmonic mean between precision and recall) that assesses the P/R tradeoff is F measure :

$$F = \frac{1}{\alpha\frac{1}{P} + (1-\alpha)\frac{1}{R}} = \frac{(\beta^2+1)PR}{\beta^2 P + R}$$

- The harmonic mean is a very conservative average;
- People usually use balanced F1 measure
  - i.e., with $\beta = 1$ (that is, $\alpha = ½$): $F = 2PR/(P+R)$
- If P and R are far apart, F tends to be near lower value in order to do well on F1, need to do well on BOTH P and R
- Can't beat the system by being either too reluctant or too promiscuous

More Than Two Classes: Multi class classification

- Dealing with any-of or multi-value classification
  - A document can belong to 0, 1, or more than 1 class.
    - Sentiment (+ve, -ve, Nutral)
    - Emotion
    - Disambiguation
- Two types
  - Any one of or
  - Multilevel
- One-of or multinomial classification
  - Classes are mutually exclusive: each document in exactly one class

More Than Two Classes: Sets of binary classifiers

- For any one of
  - Build a separate binary classifier for each class $c \in C$
  - Build a classifier γ_c to distinguish c from all other classes $c' \in C$
- Given test doc d,
  - Evaluate it for membership in each class using each γ_c
  - d belongs to any class for which γ_c returns true

Multi class classification

- For Multinomial
  - Build separate binary classifier
  - Train using positive and all others to negative
- For each class $c \in C$
  - Build a classifier γ_c to distinguish c from all other classes $c' \in C$
- Given test doc d,
  - Evaluate it for membership in each class using each γ_c
  - d belongs to the one class with maximum score

Confusion matrix

For each pair of classes <c_1, c_2> how many documents from c_1 were incorrectly assigned to c_2?

system output \ *gold labels*	urgent	normal	spam	
urgent	8	10	1	$\textbf{precision}_u = \frac{8}{8+10+1}$
normal	5	60	50	$\textbf{precision}_n = \frac{60}{5+60+50}$
spam	3	30	200	$\textbf{precision}_s = \frac{200}{3+30+200}$
	$\textbf{recall}_u = \frac{8}{8+5+3}$	$\textbf{recall}_n = \frac{60}{10+60+30}$	$\textbf{recall}_s = \frac{200}{1+50+200}$	

Micro- vs. Macro-Averaging

- To derive a single metric that tells us how well the system is doing, we can combine these values in two ways.
  - **Macro averaging**: Compute performance for each class, then average.
  - **Micro averaging**: Collect decisions for all classes, compute a contingency table, and evaluate.

Micro- vs. Macro-Averaging

Class 1: Urgent

	true urgent	true not
system urgent	8	11
system not	8	340

$$\text{precision} = \frac{8}{8+11} = .42$$

Class 2: Normal

	true normal	true not
system normal	60	55
system not	40	212

$$\text{precision} = \frac{60}{60+55} = .52$$

Class 3: Spam

	true spam	true not
system spam	200	33
system not	51	83

$$\text{precision} = \frac{200}{200+33} = .86$$

Pooled

	true yes	true no
system yes	268	99
system no	99	635

$$\text{microaverage precision} = \frac{268}{268+99} = \mathbf{.73}$$

$$\text{macroaverage precision} = \frac{.42+.52+.86}{3} = \mathbf{.60}$$

Micro average is dominated by the more frequent class

Macro average better reflects the statistics of the smaller classes, and so is more appropriate when performance on all the classes is equally important

THANK YOU

Text Classification and Logistic Regression

DR. K.M. AZHARUL HASAN

Detective stories are as littered with **clues** as texts are with words.

Yet for the poor reader, it can be challenging to know how to get the author's clues in order to make the crucial classification task: deciding whodunnit.

Logistic Regression

- Logistic regression is
  - an important analytic tool in the social and natural sciences.
  - Unlike linear regression, which predicts continuous values, logistic regression predicts probabilities between 0 and 1 and classifies observations

  - In NLP,

    logistic regression(LR) is the baseline supervised machine learning algorithm for classification

    also has a very close relationship with neural networks
  - can be used to classify an observation into one of two classes

    Ex. ‘positive sentiment’ and ‘negative sentiment’
  - or into one of many classes

Logistic Regression

- Advantages
  - Simple and easy to interpret
  - Produces probability estimates
  - Computationally efficient
  - Works well for binary classification
  - Coefficients indicate the effect of each feature

- Limitations
  - Assumes a linear relationship between features and the log-odds (natural log probability of event occurring)
  - Not suitable for highly complex, non-linear relationships without feature engineering
  - Sensitive to multicollinearity among features

    Multicollinearity: two or more independent variables are highly correlated with each other
  - Can be affected by outliers

Binary Classification in Logistic Regression

- *Given a series of input/output pairs:* (x^i, y^i)

- *For each observation* x^i
  - *We represent* x^i *by a* ***feature vector*** $[x_1, x_2, \ldots, x_n]$
  - *We compute an output*: a predicted class $\hat{y}^i \in \{0,1\}$

Examples of inputs and features: spam

- X is an email
  - X_1 is “count of the word *sale* in the email
  - X_2 is “count of mentions of drugs”
  - X_3 is “number of URLS in the email”
  - X_4 is “length of email in bytes”
  - X_5 is “uses phrase ‘Prestigious Non-Accredited Universities’

Features in logistic regression

- For each feature x_i, we'll have a weight w_i
- Weight w_i tells us how important feature x_i is for classification

  x_i =”1 if review contains ‘awesome": w_i very positive +10

  x_j =”1 if review contains ‘abysmal": w_j very negative -10

  x_k =“1 if review contains ‘mediocre": w_k a little negative -2

Logistic Regression for one observation x

- Input observation: vector $x = [x_1, x_2, \ldots, x_n]$
- Weights: one per feature: $W = [w_1, w_2, \ldots, w_n]$
  - Sometimes we call the weights $\theta = [\theta_1, \theta_2, \ldots, \theta_n]$
- Output: a predicted class $\hat{y} \in \{0, 1\}$

(multinomial logistic regression: $y \in \{0, 1, 2, 3, 4\}$)

Components of a probabilistic ML classifier

- Four components for any ML system/algorithm:
  - A feature representation of the input
  - A classification function that computes predicted output y

    Sigmoid, Softmax
  - An objective function for learning, usually involving minimizing error on training examples.

    Ex. Cross-entropy loss function
  - An algorithm for optimizing the objective function.

    Stochastic gradient descent algorithm.

^

Logistic regression : two phases

- Training:
  - Train the system (specifically the weights w and b) using stochastic gradient descent and the cross-entropy loss.
- Test:
  - Given a test example *x* we compute *p(y|x)* and return the higher probability label y = 1 or y = 0.

How to do classification

- **For each feature x_i,** the weight w_i tells us the importance of x_i
  - a bias b
    - The bias term allows the line to **shift up or down** instead of always passing through the origin (0,0).
- We'll sum up all the weighted features and the bias

$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b$$

$$z = w \cdot x + b$$

- Can be represented as a dot product
- If this sum is high, we say y=1; if low, then y=0
- **w** determines **how much the output changes** when the input changes, while *b* determines **where the prediction starts** (the baseline value)

But we want a probabilistic classifier

$$z = w \cdot x + b$$

- We need to formalize "sum is high".
- We'd like a to have a classifier that gives us a probability
- We want a model that can tell us:

  p(y=1|x) (probability of +ve) or

  p(y=0|x) (probability of -ve)

The problem

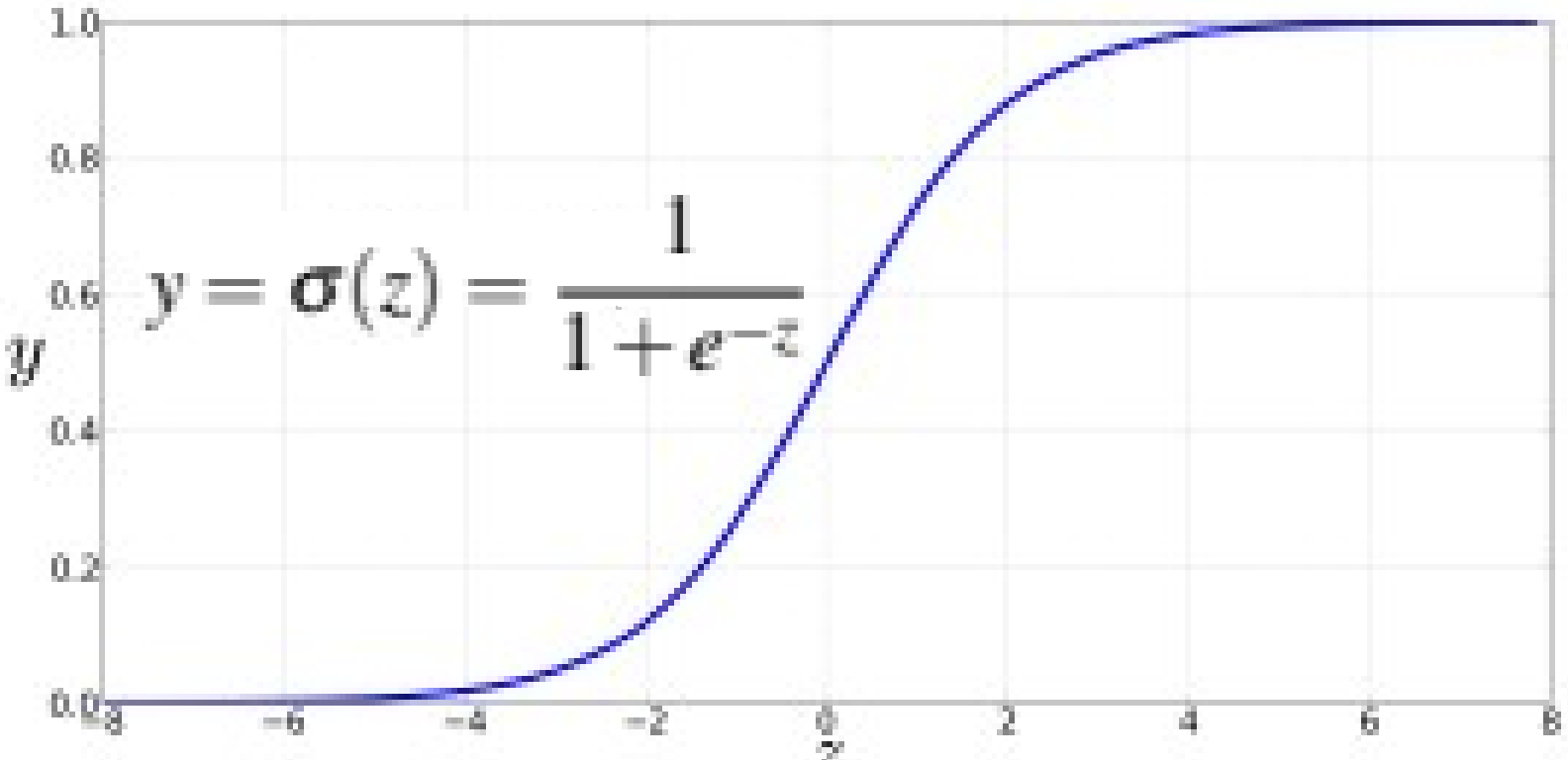
z isn't a probability; it's just a number!
Ranges from $+\infty$ to $-\infty$

$$z = w \cdot x + b$$

- Solution: use a function of z that goes from *0 to 1*
- *Called the sigmoid function σ (or s)*

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

The very useful sigmoid or logistic function



Idea of logistic regression

- We'll compute $w \cdot x+b$
- And then we'll pass it through the sigmoid function: $\sigma(w \cdot x+b)$
- And we'll just treat it as a probability
- If we apply $\sigma(w \cdot x+b)$ *we get a number between 0 and 1.*

Making probabilities with sigmoids

To make it a probability, we have to have the sum of P(y=1) and P (y=0) equal to 1.
We can do it as follows

$$P(y=1) = \sigma(w \cdot x + b) = \frac{1}{1+e^{-(w \cdot x+b)}}$$

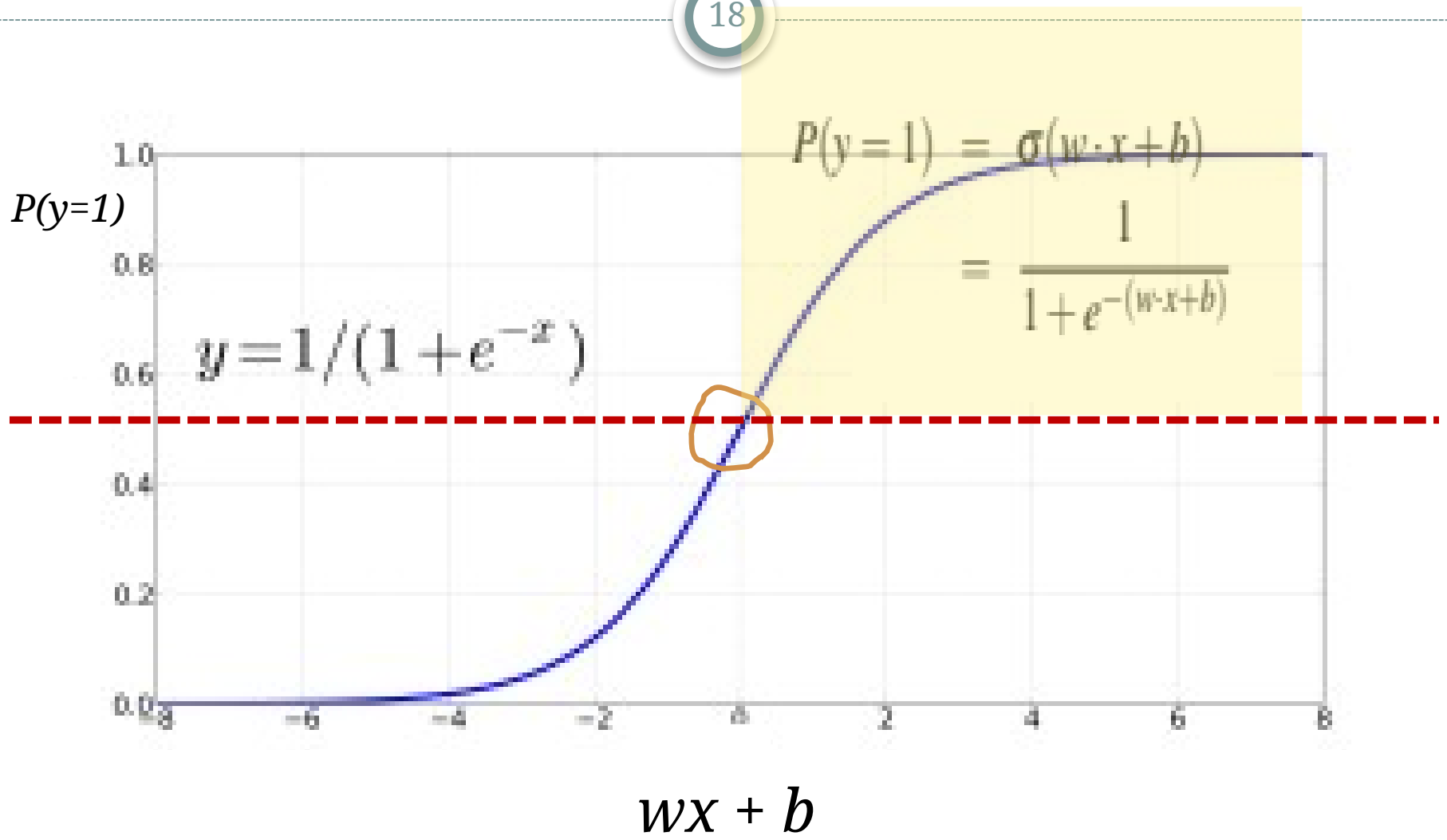
$$P(y=0) = 1-\sigma(w \cdot x + b) = 1-\frac{1}{1+e^{-(w \cdot x+b)}} = \frac{e^{-(w \cdot x+b)}}{1+e^{-(w \cdot x+b)}}$$

The decision boundary

- Given an instance *x* computes the probability *P(y = 1 | x).*
- How do we make a decision?
  - For a test instance x, we say yes if the probability P(y= 1 | x) is more than 0.5, and no otherwise.
  - We call 0.5 the decision boundary

$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

The probabilistic classifier



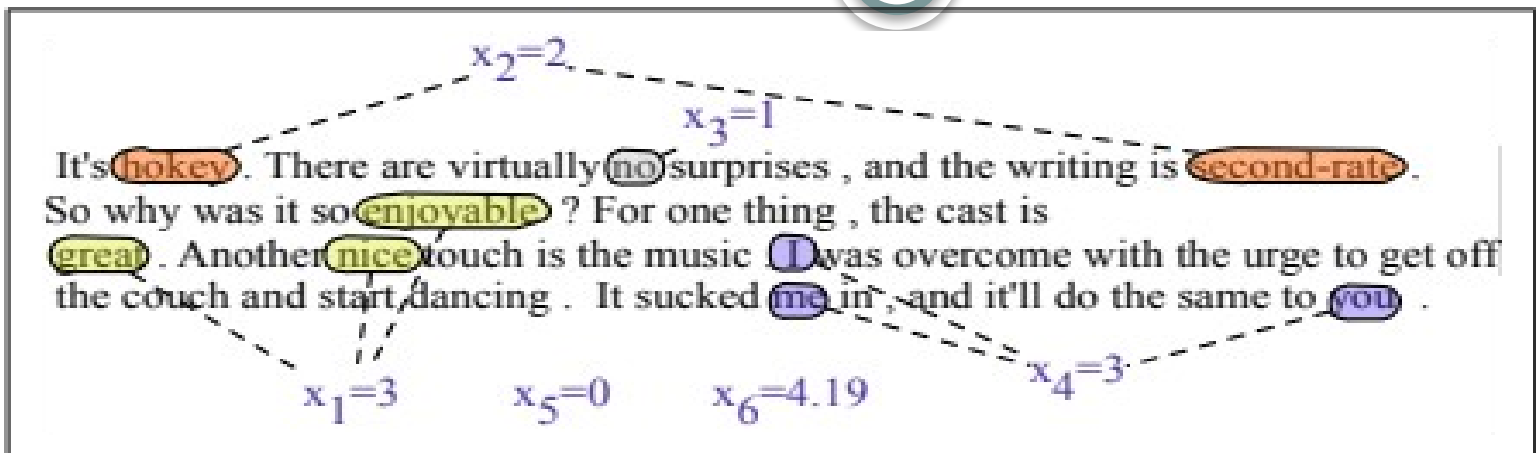
Decision boundary

$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

if $w \cdot x + b > 0$

if $w \cdot x + b \leq 0$

Sentiment example



Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(66) = 4.19$

Classifying sentiment for input x

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(66) = 4.19$

Suppose w = $[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$

b = 0.1

Classifying sentiment for input x

Suppose w = $[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$

b = 0.1

$$\begin{aligned} p(+|x) = P(Y=1|x) &= \sigma(w \cdot x + b) \\ &= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\ &= \sigma(.833) \\ &= 0.70 \end{aligned} \tag{5.6}$$

$$\begin{aligned} p(-|x) = P(Y=0|x) &= 1 - \sigma(w \cdot x + b) \\ &= 0.30 \end{aligned}$$

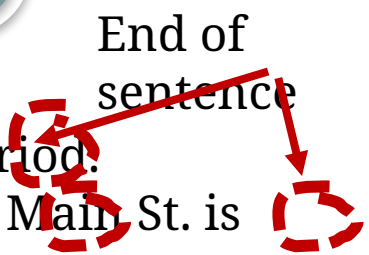
Period disambiguation

- Deciding if a period is the
  - end of a sentence or
  - part of a word,

  by classifying each period into one of two classes
  - EOS (end-of-sentence) and
  - not-EOS.

Period disambiguation

This ends in a period.
The house at 465 Main St. is new.



$$x_1 = \begin{cases} 1 & \text{if } \text{“}Case(w_i) = \text{Lower”} \\ 0 & \text{otherwise} \end{cases}$$

$$x_2 = \begin{cases} 1 & \text{if } \text{“}w_i \in \text{AcronymDict”} \\ 0 & \text{otherwise} \end{cases}$$

$$x_3 = \begin{cases} 1 & \text{if } \text{“}w_i = \text{St. \& } Case(w_{i-1}) = \text{Cap”} \\ 0 & \text{otherwise} \end{cases}$$

Classification in (**binary)** logistic regression: summary

- Given:
  - a set of classes: (+ sentiment,- sentiment)
  - a vector **x** of features `[x1, x2, …, xn]`
    - x1= count("awesome")
    - x2 = log(number of words in review)
  - A vector **w** of weights `[w1, w2, …, wn]`

    w_i for each feature

$$P(y=1) = \sigma(w \cdot x + b) = \frac{1}{1+e^{-(w \cdot x+b)}}$$

Wait, where did the W's come from?

- **Training**: given a training set of M observations (example x, class y)
  - Learn the parameters of the model
- **Test**: Given a test example x and class $y \in \{0,1\}$
  - return the higher probability class

- Logistic regression is an instance of supervised classification in which we know the correct label y (either 0 or 1) for each observation x.
  - But what the system produces is an estimate, ^y.
  - We want to learn parameters (meaning w and b) that make $\hat{y}$ for each training observation as close as possible to the true y.

Learning: Cross-Entropy Loss

Where did the W's come from?

- Supervised classification:
  - We know the correct label y (either 0 or 1) for each x.
  - But what the system produces is an estimate, $\hat{y}$
  - We want to set w and b to minimize the **distance** between our estimate $\hat{y}$ and the true y.
- We need a distance estimator: a **loss function** or a **cost function**
  - We need an optimization algorithm to update w and b to minimize the loss.

Learning components

- Two component
- A loss function:
  - This is how close the current label ($\hat{y}$) is to the true gold label y.
  - Rather than measure similarity we usually talk about the opposite of this:

    the *distance* between the system output and the gold output, and we call this distance the **loss** function or the **cost** function.

    The cross-entropy loss is used in LR

Learning components

- An optimization algorithm
  - **Stochastic gradient descent**

    The second thing we need is an optimization algorithm for iteratively updating the weights to minimize this loss function.

    The standard algorithm for this is stochastic gradient descent;

The distance between $\hat{y}$ and y

- We want to know how far is the classifier output

  $\hat{y} = \sigma(w \cdot x + b)$

  from the true output: y [= either 0 or 1]

- We'll call this difference:

  $L(\hat{y}, y)$ = how much $\hat{y}$ differs from the true y

Negative log likelihood loss = cross-entropy loss

- A case of conditional maximum likelihood estimation
  - We choose the parameters w, b that maximize the log probability of the true y labels in the training data given the observations x

- **Goal**: maximize probability of the correct label $p(y|x)$
  - Since there are only 2 discrete outcomes (0 or 1) we can express the probability $p(y|x)$ from our classifier (the thing we want to maximize) as

$$p(y|x) = \hat{y}^y(1-\hat{y})^{1-y}$$

if y=1, this simplifies to $\hat{y}$

if y=0, this simplifies to 1- $\hat{y}$

cross-entropy loss for a single observation x

Goal: maximize probability of the correct label $p(y|x)$

Maximize:

$$p(y|x) = \hat{y}^y(1-\hat{y})^{1-y}$$

- Now take the log of both sides (mathematically handy)

Maximize:

$$\log p(y|x) = \log\left[\hat{y}^y(1-\hat{y})^{1-y}\right]$$
$$= y\log\hat{y} + (1-y)\log(1-\hat{y})$$

- Whatever values maximize log p(y|x) will also maximize p(y|x)

cross-entropy loss for a single observation x

Goal: maximize probability of the correct label $p(y|x)$

Maximize :

$$\log p(y|x) = \log \left[\hat{y}^y (1-\hat{y})^{1-y}\right]$$
$$= y \log \hat{y} + (1-y) \log (1-\hat{y})$$

- Now flip sign to turn this into a loss: something to minimize
- **Cross-entropy loss** (because is formula for cross-entropy(y, $\hat{y}$))Or, plugging in definition of $\hat{y}$:

Minimize :

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y) \log (1-\hat{y})]$$

$$L_{CE}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1-y) \log (1-\sigma(w \cdot x + b))]$$

Let's see if this works for our sentiment example

- We want loss to be
  - smaller if the model estimate is close to correct
  - bigger if model is confused
- Let's first suppose the true label of this is y=1 (positive)

It's hokey . There are virtually no surprises , and the writing is second-rate . So why was it so enjoyable ? For one thing , the cast is great . Another nice touch is the music . I was overcome with the urge to get off the couch and start dancing . It sucked me in , and it'll do the same to you .

Let's see if this works for our sentiment example

- True value is y=1. How well is our model doing?

$$\begin{aligned} p(+|x) = P(Y=1|x) &= \sigma(w \cdot x + b) \\ &= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\ &= \sigma(.833) \\ &= 0.70 \end{aligned} \tag{5.6}$$

- Pretty well! What's the loss?

$$\begin{aligned} L_{CE}(\hat{y}, y) &= -[y \log \sigma(w \cdot x + b) + (1-y) \log(1 - \sigma(w \cdot x + b))] \\ &= -[\log \sigma(w \cdot x + b)] \\ &= -\log(.70) \\ &= .36 \end{aligned}$$

Let's see if this works for our sentiment example

- The loss when model was right (if true y=1)

$$\begin{aligned} L_{CE}(\hat{y},y) &= -[y\log\sigma(w\cdot x+b)+(1-y)\log(1-\sigma(w\cdot x+b))] \\ &= -[\log\sigma(w\cdot x+b)] \\ &= -\log(.70) \\ &= .36 \end{aligned}$$

- Is lower than the loss when model was wrong (if true y=0):

$$\begin{aligned} L_{CE}(\hat{y},y) &= -[y\log\sigma(w\cdot x+b)+(1-y)\log(1-\sigma(w\cdot x+b))] \\ &= -[\log(1-\sigma(w\cdot x+b))] \\ &= -\log(.30) \\ &= 1.2 \end{aligned}$$

- Sure enough, loss was bigger when model was wrong!

Stochastic Gradient Descent

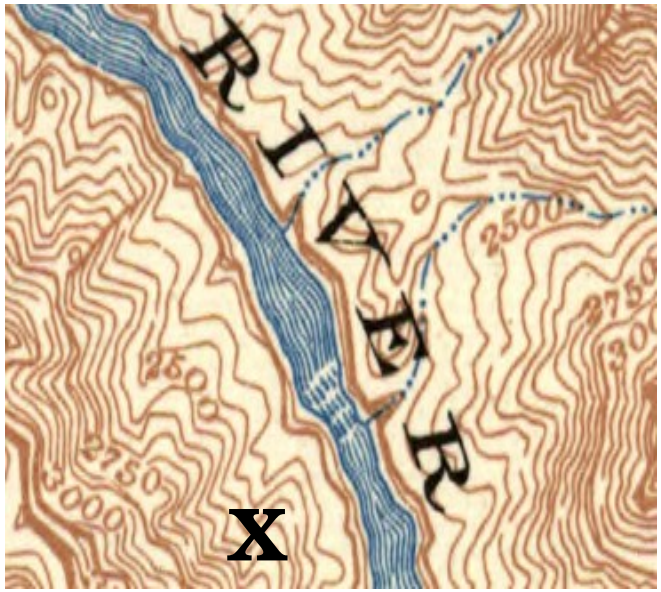
Our goal: minimize the loss

- Let's make explicit that the loss function is parameterized by weights θ=(w, b)
  - And we'll represent $\hat{y}$ as $f(x; \theta)$ to make the dependence on θ more obvious
  - in the case of logistic regression $\theta = w, b$
  - The goal is to find the set of weights which minimizes the loss function
- We want the weights that minimize the loss, averaged over all examples:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^{m} L_{\mathrm{CE}}(f(x^{(i)}; \theta), y^{(i)})$$

Intuition of gradient descent

- How do I get to the bottom of this river canyon?



Look around me 360° Find the direction of steepest slope down Go that way

How do we find the minimum of a loss function?

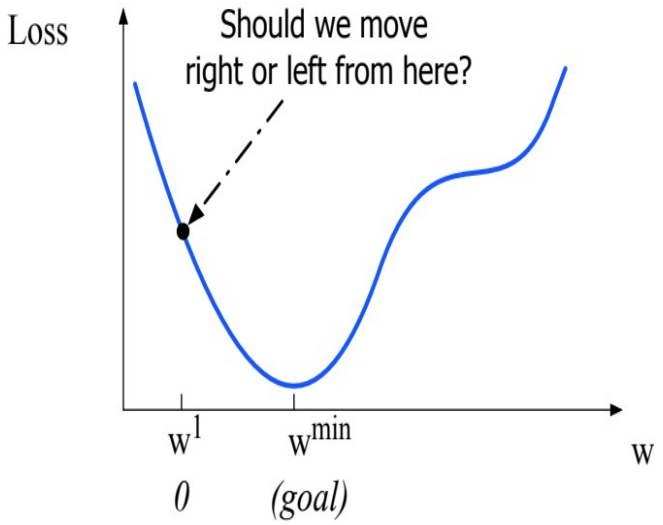
Gradient descent is a method that finds a minimum of a function by figuring out in which direction the function's slope is rising the most steeply, and moving in the opposite direction.

Our goal: minimize the loss

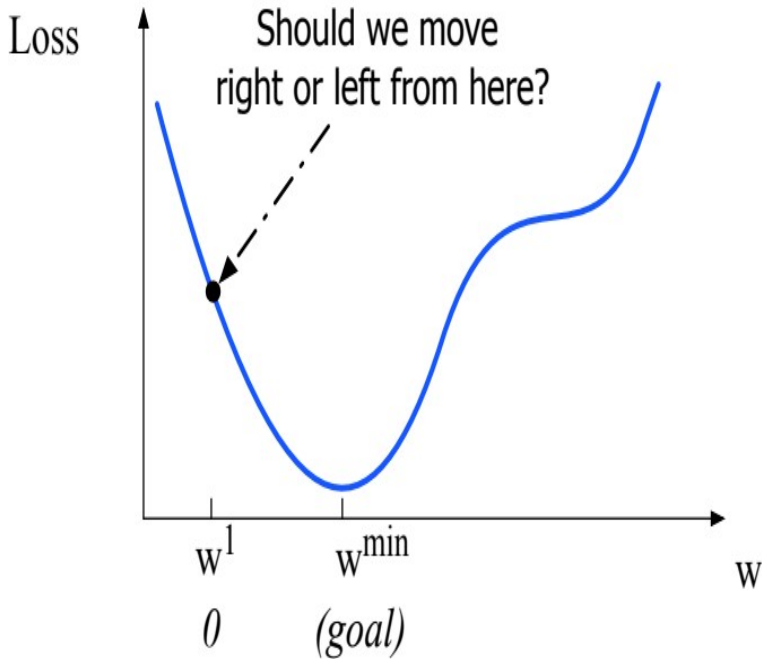
- For logistic regression, loss function is **convex**
  - A convex function has just one minimum and no local minima
  - Gradient descent starting from any point is guaranteed to find the minimum

Q: Given current w, should we make it bigger or smaller?

A: Move *w* in the reverse direction from the slope of the function



Let's first visualize for a single scalar w

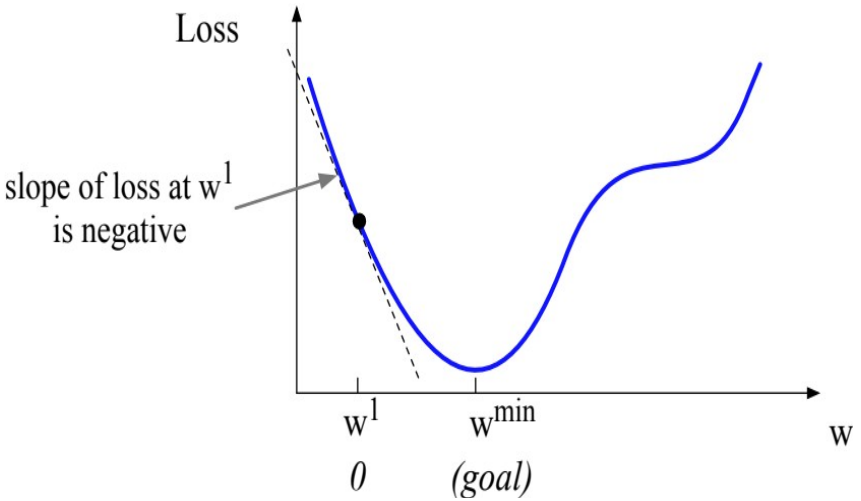


- Given a random initialization of *w* at some value *w*1, and assuming the loss function *L* happened to have this shape,
- we need the algorithm to tell us whether at the next iteration we should move left (making *w*2 smaller than *w*1) or right (making *w*2 bigger than *w*1) to reach the minimum.

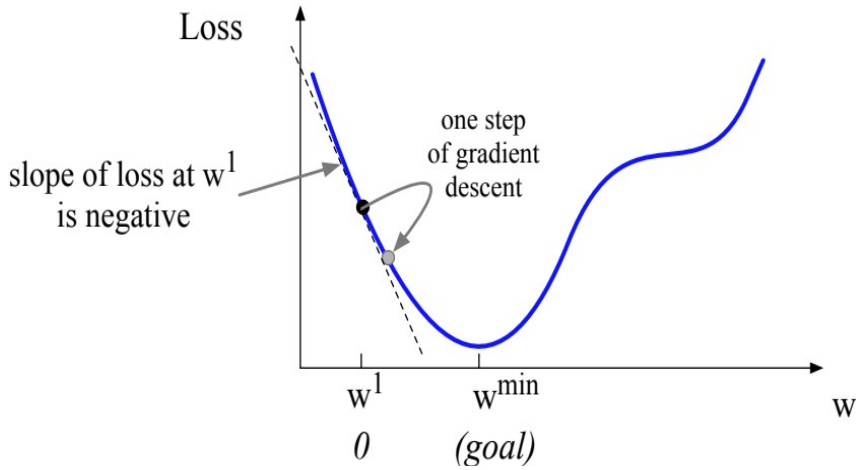
- The gradient descent algorithm answers this question by finding the gradient of the loss function at the current point and moving in the opposite direction.

Let's first visualize for a single scalar w

Q: Given current w, should we make it bigger or smaller?
A: Move *w* in the reverse direction from the slope of the function



So we'll move positive



How much do we move in that direction ?

- The value of the gradient (slope in our example) $\frac{d}{dw} L(f(x;w),y)$ weighted by a **learning rate** η
- Higher learning rate means move *w* faster

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x;w),y)$$

Real gradients

- Are much longer; lots and lots of weights
- For each dimension w_i the gradient component i tells us the slope with respect to that variable.
  - "How much would a small change in w_i influence the total loss function L?"
  - We express the slope as a partial derivative ∂ of the loss ∂w_i
- The gradient is then defined as a vector of these partials.

What are these partial derivatives?

The loss function

$$L_{CE}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

The elegant derivative of this function

$$\frac{\partial L_{CE}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Stochastic Gradient Descent: An Example

Multinomial Logistic Regression

- The probability of everything must still sum to 1

$$P(\text{positive} \mid \text{doc}) + P(\text{negative} \mid \text{doc}) + P(\text{neutral} \mid \text{doc}) = 1$$

- Need a generalization of the sigmoid called the **softmax**
  - Takes a vector $z = [z1, z2, ..., zk]$ of k arbitrary values
  - Outputs a probability distribution

    each value in the range [0, 1]

    all the values summing to 1

The **softmax** function

Turns a vector $z = [z_1, z_2, \ldots, z_k]$ of k arbitrary values into probabilities

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} \quad 1 \leq i \leq k$$

The denominator $\sum_{i=1}^{k} e^{z_i}$ is used to normalize all the values into probabilities.

$$\text{softmax}(z) = \left[\frac{e^{z_1}}{\sum_{i=1}^{k} e^{z_i}}, \frac{e^{z_2}}{\sum_{i=1}^{k} e^{z_i}}, \ldots, \frac{e^{z_k}}{\sum_{i=1}^{k} e^{z_i}} \right]$$

The **softmax** classifier

- Turns a vector $z = [z_1, z_2, \ldots, z_k]$ of k arbitrary values into probabilities

$$\text{softmax}(z) = \left[\frac{e^{z_1}}{\sum_{i=1}^{k} e^{z_i}}, \frac{e^{z_2}}{\sum_{i=1}^{k} e^{z_i}}, \ldots, \frac{e^{z_k}}{\sum_{i=1}^{k} e^{z_i}} \right]$$

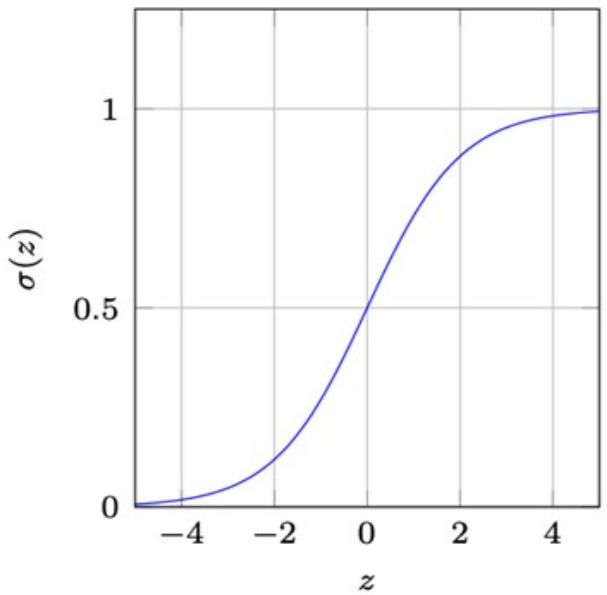
$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$[0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

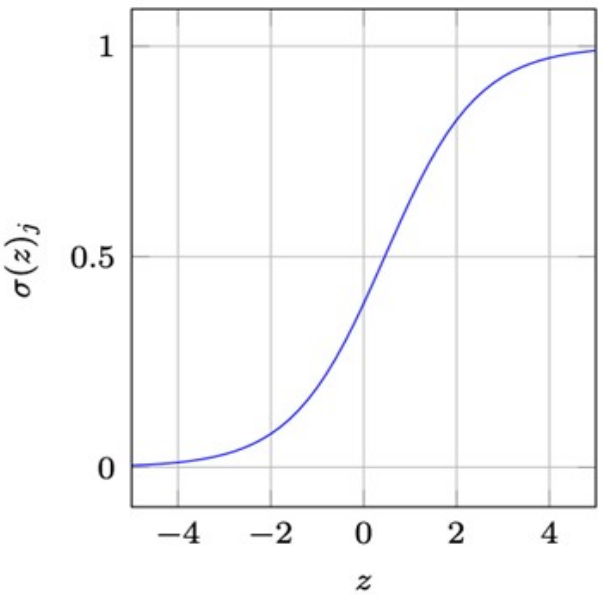
Sigmoid vs Softmax

- Sigmoid
  - Sigmoid function is used for two class logistic regression. Sum of probabilities need not to be 1.
  - The high value will have the high probability but it need not to be the highest probability
- Softmax
  - Softmax Function is used for Multi class Logistic Regression.
  - Sum of the Probabilities equals to 1.
  - It is used in different layers of Neural Network.
  - The high value will have the higher probability than others.

Sigmoid vs Softmax



(a) Sigmoid activation function.



(b) Softmax activation function.

Binary versus multinomial logistic regression

- Multinomial is obviously applicable to many more classification tasks
  - Softmax is important for neural net classifiers
- Binary logistic regression is simpler to model

vanishing gradient problem

- The derivative of a function is very small
- The flat derivative makes it difficult to improve the weights through gradient descent.
- This problem gets worse as the model has more layers.
- This was called the **vanishing gradient problem**.

THANK YOU

Vector Semantics and Word Embedding



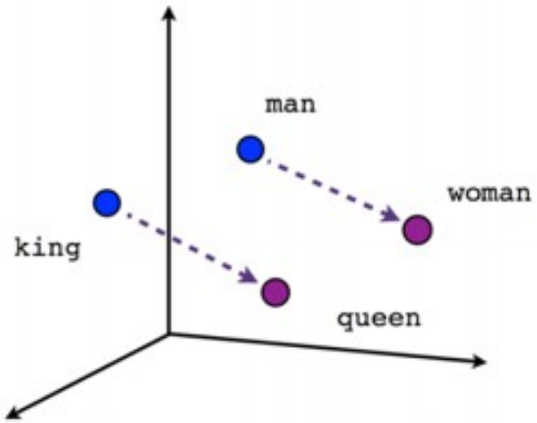
DR. K.M. AZHARUL HASAN

Vector Embedding of Words

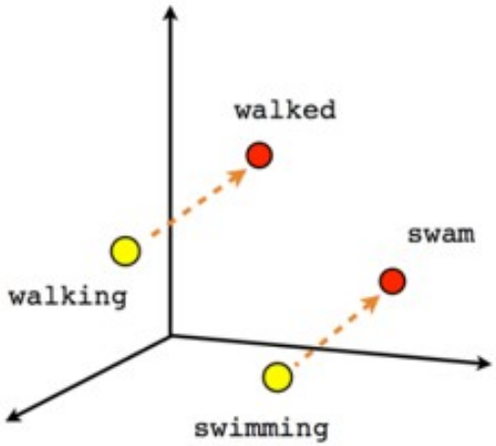
- A word is represented as a vector.
  - Word embeddings depend on a notion of ***word similarity***.
  - Similarity is computed such as cosine.
- A very useful definition is paradigmatic similarity:
  - ***Similar words*** occur in ***similar contexts.*** They are ***exchangeable.***
  - Yesterday { POTUS / The President / Trump } called a press conference.

"POTUS: President of the United States."

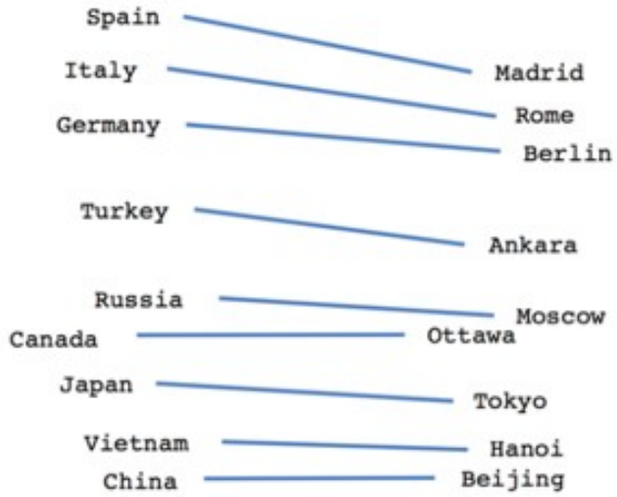
Example



Male-Female



Verb tense

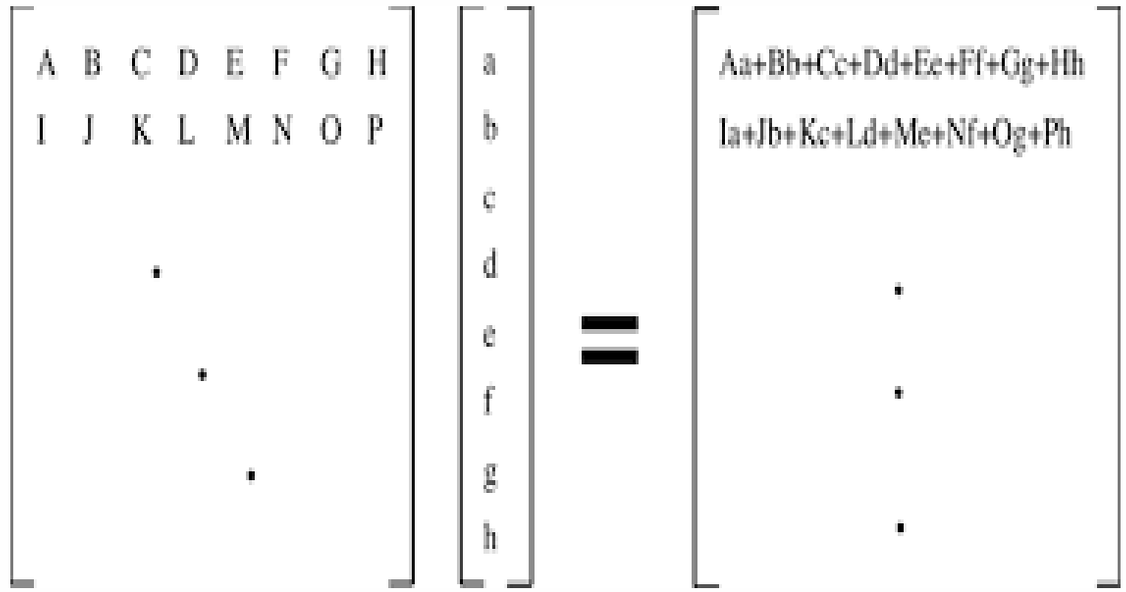


Country-Capital

This can be interpreted as “France is to Paris as Italy is to Rome”.

Working with vectors

- Finding the most similar words to $\overrightarrow{dog}$.
  - Compute the similarity from word $\overrightarrow{dog}$ to all other words.
  - This is a single matrix-vector product: $W \cdot \overrightarrow{dog}$
    - W is the word embedding matrix of $|V|$ rows and d columns.
    - Result is a |V| sized vector of similarities.
    - Take the indices of the k-highest values.



Working with vectors

- Similarity to a group of words
  - "Find the words most similar to cat, dog and cow".
  - Calculate the pairwise similarities and sum them:

$$W \cdot \overrightarrow{cat} + W \cdot \overrightarrow{dog} + W \cdot \overrightarrow{cow}$$

  - Now find the indices of the highest values as before.
  - Matrix-vector products are wasteful. Better option:

$$W \cdot (\overrightarrow{cat} + \overrightarrow{dog} + \overrightarrow{cow})$$

Applications of Word Vectors

- Word Similarity
- Machine Translation
- Part-of-Speech and Named Entity Recognition
- Relation Extraction
- Sentiment Analysis
- Co-reference Resolution
  - Chaining entity mentions across multiple documents - can we find and unify the multiple contexts in which mentions occurs?
- Clustering
  - Words in the same class naturally occur in similar contexts, and this feature vector can directly be used with any conventional clustering algorithms (K-Means, agglomerative, etc).
  - Human doesn't have to waste time hand-picking useful word features to cluster on.
- Semantic Analysis of Documents
  - Build word distributions for various topics, etc.

Vector Semantics

- Distributional hypothesis
  - Words that occur in similar contexts tend to have similar meanings.
  - This link between similarity in how words are distributed and similarity in what they mean is called the distributional hypothesis.

- Vector semantics
  - Represents this linguistic hypothesis by learning representations of the meaning of words, called embeddings
  - representation learning, automatically learns useful representations of the input text

    self-supervised ways to learn representations of the input,

    instead of creating representations by feature engineering

Lexical Semantics

- How should we represent the meaning of a word?
  - representation of a word is as a string of letters, or
  - as an index in a vocabulary list
- Linguistic study of word meaning is called lexical semantics
- Lemma
  - Finding the root

    sing is the lemma for sing, sang, sung
  - lemmas can be polysemous (have multiple senses) can make interpretation difficult
-

- Synonymy
  - an important component of word meaning, is the relationship between word senses.
  - two words are synonymous if they are substitutable one for the other in any sentence without changing the truth conditions of the sentence
  - Same propositional meaning

Lemmas and senses



lemma

mouse (N)

sense

1. any of numerous small rodents...
2. a hand-operated device that controls a cursor...

A sense or "concept" is the meaning component of a word

Lemmas can be polysemous (have multiple senses)

Lexical Semantics

- Principle of contrast
  - is the assumption that a difference in linguistic form is always associated with at least some difference in meaning.

    The word H2O is used in scientific contexts and would be inappropriate in a hiking guide—water
- Word relatedness
  - A semantic field is a set of words which cover a particular semantic domain and bear structured relations with each other.

    words might be related by being in the semantic field of
    - hospitals (surgeon, scalpel, nurse, anesthetic, hospital),
    - restaurants (waiter, menu, plate, food, chef),
    - houses (door, roof, kitchen, family, bed).
  - Semantic fields are also related to topic modeling

Lexical Semantics

- Three dimensions of affective meaning
  - valence: the pleasantness of the stimulus

    happy or satisfied are high on valence, while unhappy or annoyed are low on valence.
  - arousal: the intensity of emotion provoked by the stimulus

    Excited or frenzied are high on arousal, while relaxed or calm are low on arousal
  - dominance: the degree of control exerted by the stimulus

    Important or controlling are high on dominance, while awed or influenced are low on dominance

Lexical Semantics

- Each word is thus represented by three numbers, corresponding to its value on each of the three dimensions
  - these 3 numbers to represent the meaning of a word
  - representing each word as a point in a three dimensional space
  - a vector whose three dimensions corresponded to the word's rating on the three scales (heartbreak can be represented as the point [2:45; 5:65; 3:58])
  - expression of the vector semantics models

	Valence	Arousal	Dominance
courageous	8.05	5.5	7.38
music	7.67	5.57	6.5
heartbreak	2.45	5.65	3.58
cub	6.71	3.95	4.24
life	6.68	5.59	5.89

Meaning as a point in space

- 3 affective dimensions for a word
  - **valence**: pleasantness
  - **arousal**: intensity of emotion
  - **dominance**: the degree of control exerted

	Word	Score		Word	Score
Valence	love	1.000		toxic	0.008
	happy	1.000		nightmare	0.005
Arousal	elated	0.960		mellow	0.069
	frenzy	0.965		napping	0.046
Dominance	powerful	0.991		weak	0.045
	leadership	0.983		empty	0.081

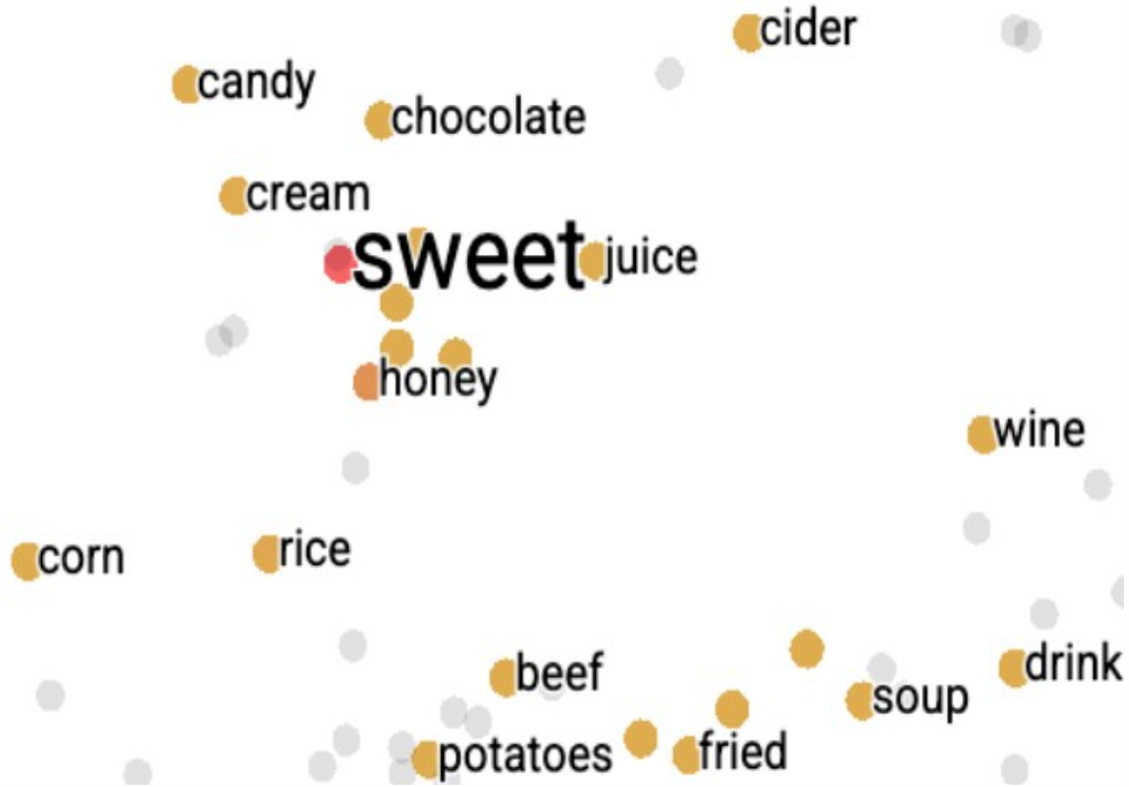
- Hence the connotation of a word is a vector in 3-space

Vector Semantics

- Vector semantic define a word by its environment or distribution of language usage
- Distribution of a word is the set of contexts in which it occurs, the neighboring words or grammatical environments.
  - The idea is that two words that occur in very similar distributions (that occur together with very similar words) are likely to have the same meaning.

Defining meaning as a point in space based on distribution

- Similar words are "**nearby in semantic space**"
- We build this space automatically by seeing which words are **nearby in text**



We define a word as a vector

- *The idea of vector semantics is thus to represent a word as a point in some multidimensional semantic space.*
  - called an "embedding" because it's embedded into a space
- The standard way to represent meaning in NLP
  - positive and negative words seem to be located in distinct portions of the space: sentiment clustering
  - vector semantic models are also extremely practical because they can be learned automatically from text without any complex labeling or supervision.

Vector Semantics



Displays a visualization of embeddings that were learned for a sentiment analysis task, showing the location of some selected words projected down from the original

60-dimensional space into a two dimensional space.

Words can be vectors too

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

battle is the kind of word that occurs in "Julius Caesar and Henry V"

fool is the kind of word that occurs in comedies, especially " Twelfth Night"

Term-document matrix

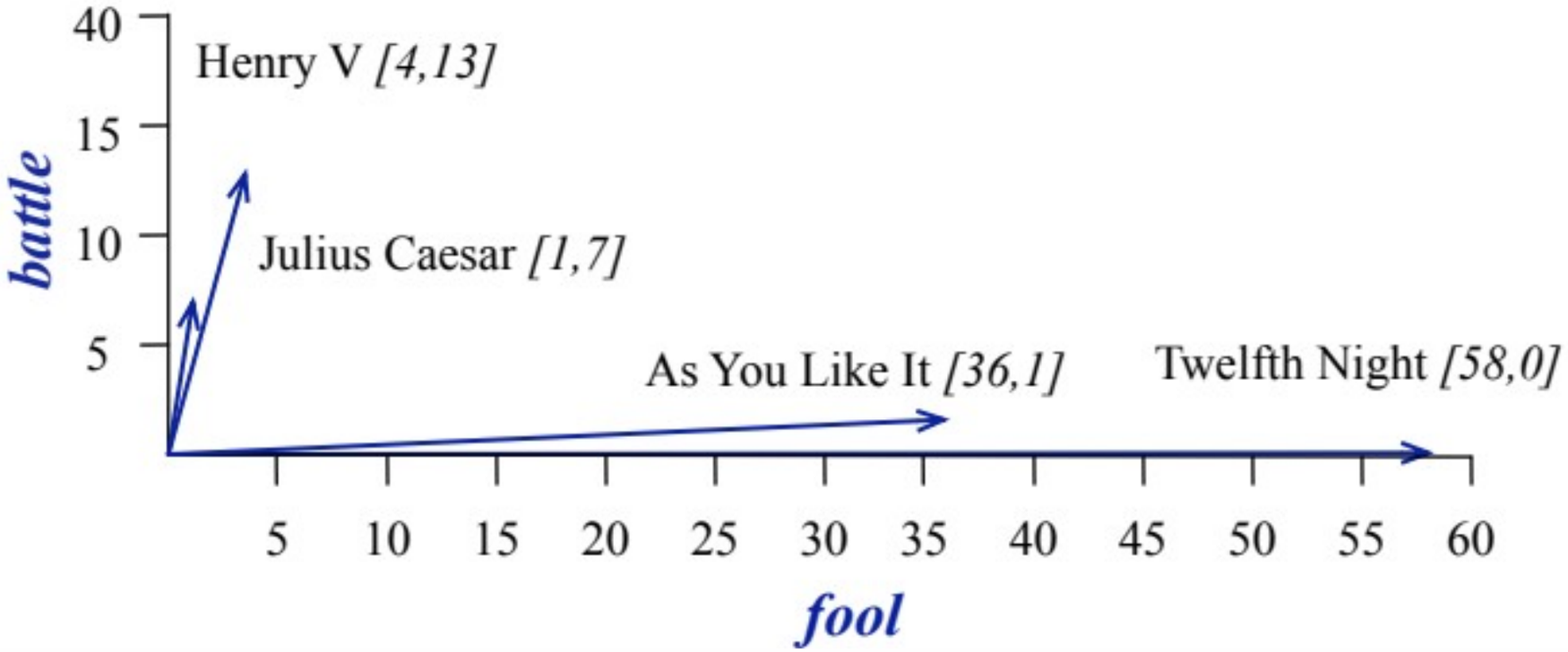
Each document is represented by count vector of words

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

As You Like It is represented as the list [1, 114, 36, 20]
Julius Caesar is represented as the list [7, 62, 1, 2].
A *vector space* is a collection of vectors, characterized by their dimension.

vector for a document is identifying a point in V-dimensional space;

Visualizing document vectors



term-context matrix

- The columns are labeled by words rather than documents
  - v × v matrix
  - Generally, a window around the word, for example, 4 words to the left and 4 words to the right, in which case the cell represents the number of times

Two words are similar in meaning if their context vectors are similar

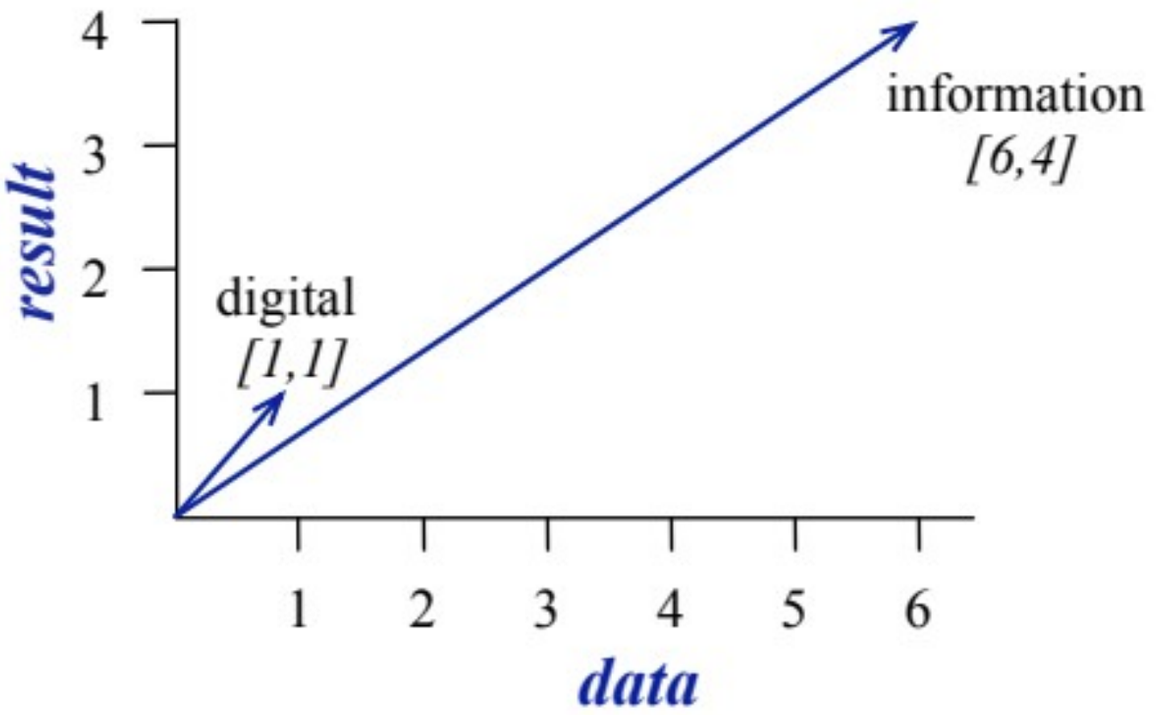
sugar, a sliced lemon, a tablespoonful of	**apricot**	jam, a pinch each of,
their enjoyment. Cautiously she sampled her first	**pineapple**	and another fruit whose taste she likened
well suited to programming on the digital	**computer**.	In finding the optimal R-stage policy from
for the purpose of gathering data and	**information**	necessary for the study authorized in the

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	
strawberry	0	...	0	0	1	60	19	
digital	0	...	1670	1683	85	5	4	
information	0	...	3325	3982	378	5	13	

Co-occurrence matrix within the window

The words cherry and strawberry are more similar to each other

term-context matrix visualization



Cosine for measuring similarity

- To define similarity between two target words v and w, we need a measure for taking two such vectors and giving a measure of vector similarity.
- By far the most common similarity metric is the cosine of the angle between the vectors.

$$\text{dot-product}(\vec{v},\vec{w}) = \vec{v}\cdot\vec{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

- it will tend to be high just when the two vectors have large values in the same dimensions.
- dot product of 0, representing their strong dissimilarity

$$\text{vector length } |\vec{v}| = \sqrt{\sum_{i=1}^{N} v_i^2}$$

The dot product is higher if a vector is longer, with higher values in each dimension

Cosine for computing similarity

- The simplest way to normalize for the vector length is to
  - divide the dot product by the lengths of each of the two vectors.
- This **normalized dot product** turns out to be the same as the cosine of the angle between the two vectors

$$\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| \cos\theta$$

$$\frac{\vec{a} \cdot \vec{b}}{|\vec{a}||\vec{b}|} = \cos\theta$$

Cosine for computing similarity

The cosine similarity metric between two vectors v and w computed as:

$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}||\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2} \sqrt{\sum_{i=1}^{N} w_i^2}}$$

$\vec{v_i}$ is the count for word v in context i

$\vec{w_i}$ is the count for word w in context i.

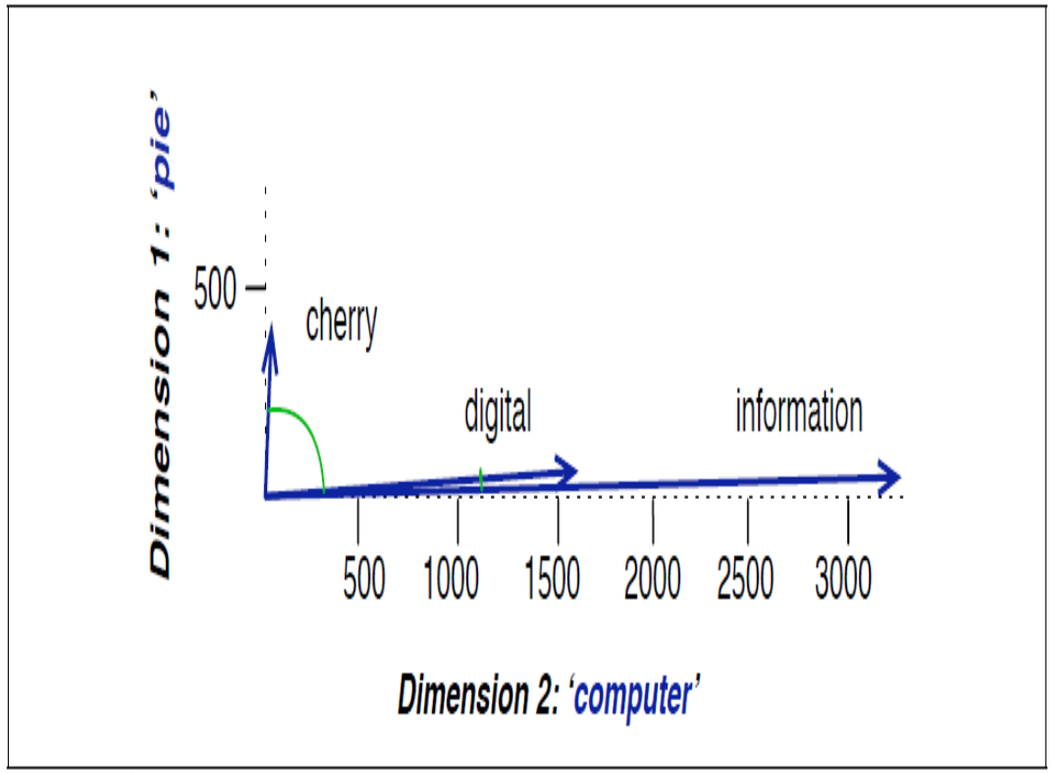
Cos(v, w) is the cosine similarity of $\vec{v}$ and $\vec{w}$

Cosine for computing similarity

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{information}) = \frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .017$$

$$\cos(\text{digital}, \text{information}) = \frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996$$



Cosine for computing similarity

$$\cos(\vec{v},\vec{w}) = \frac{\vec{v}\cdot\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\vec{v}}{|\vec{v}|}\cdot\frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

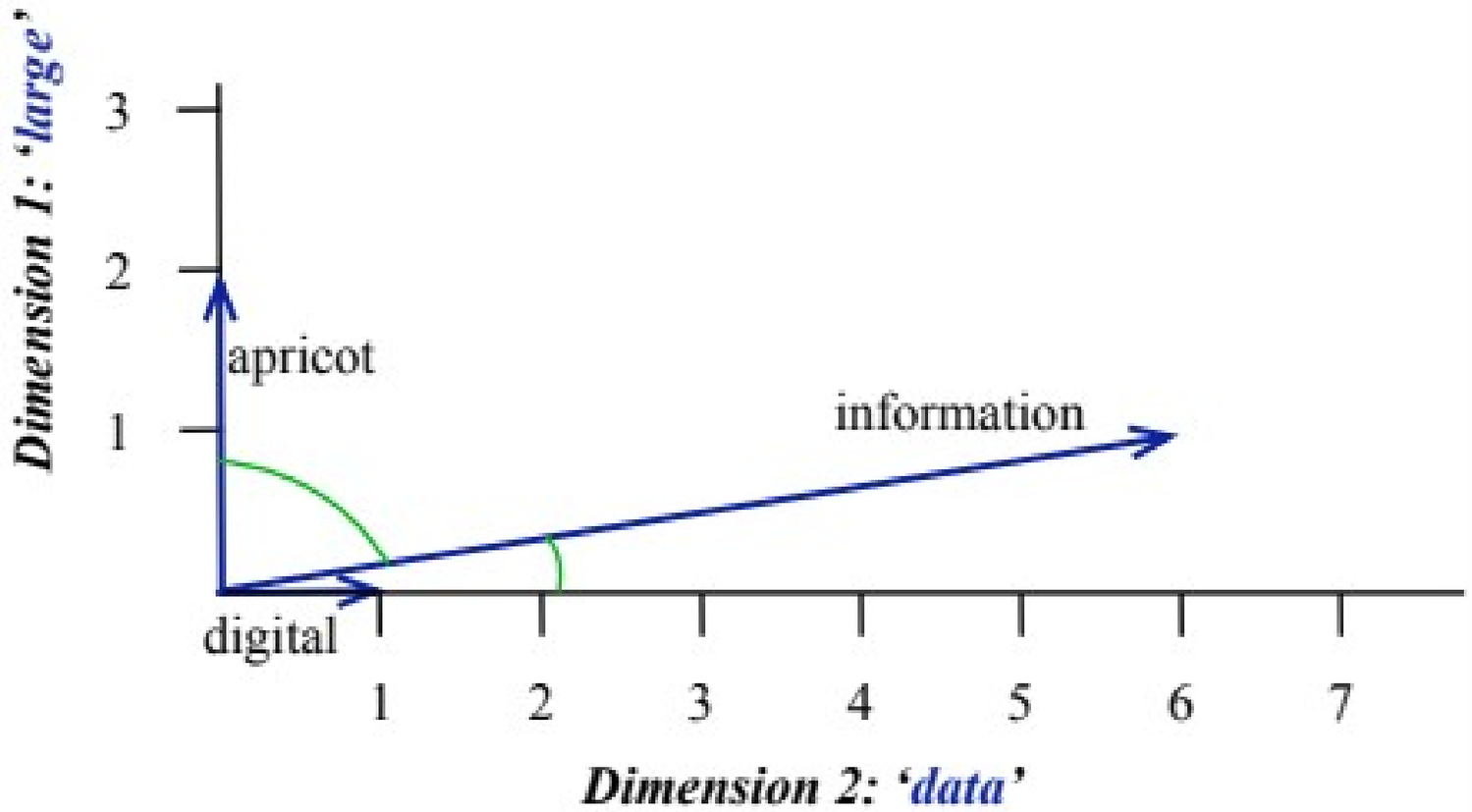
	large	data	computer
apricot	1	0	0
digital	0	1	2
information	1	6	1

cosine(apricot, information) = $\frac{1+0+0}{\sqrt{1+0+0}\sqrt{1+36+1}} = \frac{1}{\sqrt{38}} = .16$

cosine(digital, information) = $\frac{0+6+2}{\sqrt{0+1+4}\sqrt{1+36+1}} = \frac{8}{\sqrt{38}\sqrt{5}} = .58$

cosine(apricot, digital) = $\frac{0+0+0}{\sqrt{1+0+0}\sqrt{0+1+4}} = 0$

Visualizing cosines (well, angles)



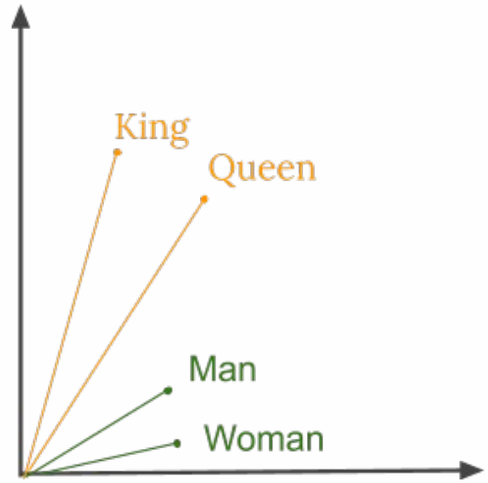
Word Embedding

Word Embedding

- Word embedding is a form of word representation that
  - bridges the human understanding of language to that of a machine.
  - learned representations of text in an n-dimensional space where

    words that have the same meaning have a similar representation.
  - Two similar words are represented by almost similar vectors that are very closely placed in a vector space.

- For example, we humans understand the words like king and queen, man and woman, tiger and tigress have a certain type of relation between them but how can a computer figure this out?



Word Embedding

- **Word embedding** is a way of **representing words as numerical vectors** so that a machine can understand relationships and similarities between words.

How to represent?

Word embedding converts words into numerical vectors in such a way that semantic and syntactic relationships between words can be captured mathematically.

Representational learning

- Text representation
  - Sparse representations/ Count based embedding
    - One-Hot Encoding
    - Bag-of-Words (BoW)
    - TF-IDF
  - Dense representations / Learning based Embedding
    - Static
      - Word2Vec
      - Glove
      - Fast Text
    - Contextual
      - ELMo
      - BERT/BanglaBERT
      - Roberta

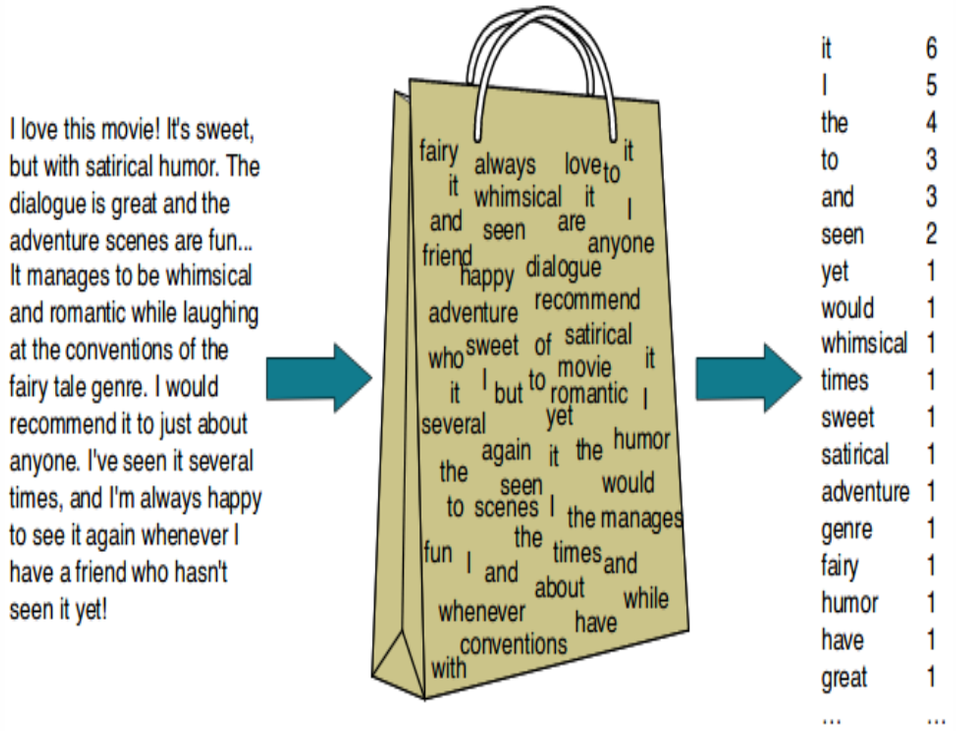
Representational learning :

is the process of allowing a machine-learning model to **automatically learn useful representations and features from raw data.**

Bag of Words (BoW)

Bag of Words

- The bag-of-words : a statistical language model used to analyze text and documents based on word count.
- a text is represented as the bag of its words,
  - disregarding grammar and even word order but keeping multiplicity.
- any information about the order or structure of words in the document is discarded.



Bag of Words

Document	the	dog	sat	in	hat	with
The dog sat	1	1	1	0	0	0
The dog sat in the hat	2	1	1	1	1	0
The dog with the hat	2	1	0	0	1	1

- This generates a 6-length vector for each document.
  - The dog sat: [1,1,1,0,0,0]
  - The dog sat in the hat: [2,1,1,1,1,0]
  - The dog with the hat: [2,1,0,0,1,1]
- Limitations:
  - BOW disregards context and the order of words which carry many useful information.
  - There is no semantic information.

One Hot Encoding

One Hot Encoding

- Transforms categorical data into numbers
  - Categorical variables are types of data that may be divided into groups.

    Example : sex, age group, and educational level.
- A vector is created with the size of the total number of unique words.
  - The value of vectors is assigned such that the value of each word corresponding to its index is 1 and the others are 0.
  - A one-hot vector is a $1 \times N$ matrix (vector) used to distinguish each word in a vocabulary

One Hot Encoding

red, green, blue

1, 0, 0
0, 1, 0
0, 0, 1

- *Each bit represents a possible category.*
- If the variable cannot belong to multiple categories at once, then only one bit in the group can be *"ON."* *This is called one-hot encoding*
  - There are three categories, therefore, three binary variables are needed.
  - A "1" value is placed in the binary variable for the color and "0" values for the other colors.

One Hot Encoding

Student	Department	Gender	Grade
Alice	CSE	Female	A
Bob	EEE	Male	B
Carol	BME	Female	A
David	CSE	Male	C
Emma	EEE	Female	B

One-hot encoding converts categorical information into a binary numerical representation without implying any order among the categories.

For the above data set
The one hot encoding becomes

Student	CSE	EEE	BME	Male	Female	A	B	C
Alice	1	0	0	0	1	1	0	0
Bob	0	1	0	1	0	0	1	0
Carol	0	0	1	0	1	1	0	0
David	1	0	0	1	0	0	0	1
Emma	0	1	0	0	1	0	1	0

For example, **Alice is to be encoded as**

[1, 0, 0, 0, 1, 1, 0, 0]

Term Frequency(TF)—Inverse Document Frequency(IDF)

TF-IDF

TF-IDF

- A common baseline model
- Sparse vectors
- Words are represented by a simple function of the counts of nearby words

TF-IDF

- TF-IDF is a statistical measure used to determine the mathematical significance of words in documents.
  - The vectorization process is similar to One Hot Encoding.
  - The value corresponding to the word is assigned a TF-IDF value instead of 1.
  - The TF-IDF value is obtained by multiplying the TF and IDF values.
  - TF-IDF is a technique is used to find meaning of sentences consisting of words

TF-IDF

- *TF = (Number of occurrences of a word in a document) / (total number of words in a document)*
- *IDF =Log[(total number of documents) / (Number of documents containing the word)]*

- How many times a word is used in a document TF gives a probability and
- IDF answers questions like how important is the word in the entire list of documents

TF-IDF

- By evaluating TF-IDF we understand
  - how useful a word is to a sentence

    which helps us to understand the importance of a word in a sentence.
  - how useful a word is to a document

    which helps us understand the important words with more frequencies in a document.

TF-IDF

- TF: term frequency is the ratio of the number of target terms in the document to the total number of terms in the document.

EX.

[He is Walter],
[He is William],
[He isn't Peter or September]

TF:

- [0.33, 0.33, 0.33],
- [0.33, 0.33, 0.33],
- [0.20, 0.20, 0.20, 0.20, 0.20]

TF-IDF

- The IDF value is the logarithm of the ratio of the total number of documents to the number of documents in which the target term occurs.

- He”: Log(3/3)= 0,
- “is”: Log(3/2):0.1761,
- “or, Peter, ..”: log(3/1) : 0.4771

tf-idf: combine two factors

- **tf: term frequency**. frequency count (usually log-transformed):

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t,d) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

- **Idf: inverse document frequency: tf-**

$$\mathrm{idf}_i = \log\left(\frac{N}{\mathrm{df}_i}\right)$$

Total # of docs in collection

of docs that have word i

Words like "the" or "good" have very low idf

tf-idf value for word t in document d:

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

Word2Vec

Word2Vec: Local contexts

- Instead of entire documents, ***Word2Vec*** uses words ***k*** positions away from each center word.
  - These words are called **context words**.
- Example for ***k=3***
  - "It was *a bright cold* **day** *in April, and* the clocks were striking".
  - Center word day: red (also called focus word).
  - Context words: blue (also called target words).
- Word2Vec considers
  - all words as center words, and
  - all their context words.

Word2Vec: Data generation (window size = 2)

Example: d1 = “king brave man” , d2 = “queen beautiful women”

Center word	Word one hot encoding	Context word	Context one hot encoding
king	[1,0,0,0,0,0]	brave	[0,1,0,0,0,0]
king	[1,0,0,0,0,0]	man	[0,0,1,0,0,0]
brave	[0,1,0,0,0,0]	king	[1,0,0,0,0,0]
brave	[0,1,0,0,0,0]	man	[0,0,1,0,0,0]
man	[0,0,1,0,0,0]	king	[1,0,0,0,0,0]
man	[0,0,1,0,0,0]	brave	[0,1,0,0,0,0]
queen	[0,0,0,1,0,0]	beautiful	[0,0,0,0,1,0]
queen	[0,0,0,1,0,0]	women	[0,0,0,0,0,1]
beautiful	[0,0,0,0,1,0]	queen	[0,0,0,1,0,0]
beautiful	[0,0,0,0,1,0]	women	[0,0,0,0,0,1]
woman	[0,0,0,0,0,1]	queen	[0,0,0,1,0,0]
woman	[0,0,0,0,0,1]	beautiful	[0,0,0,0,1,0]

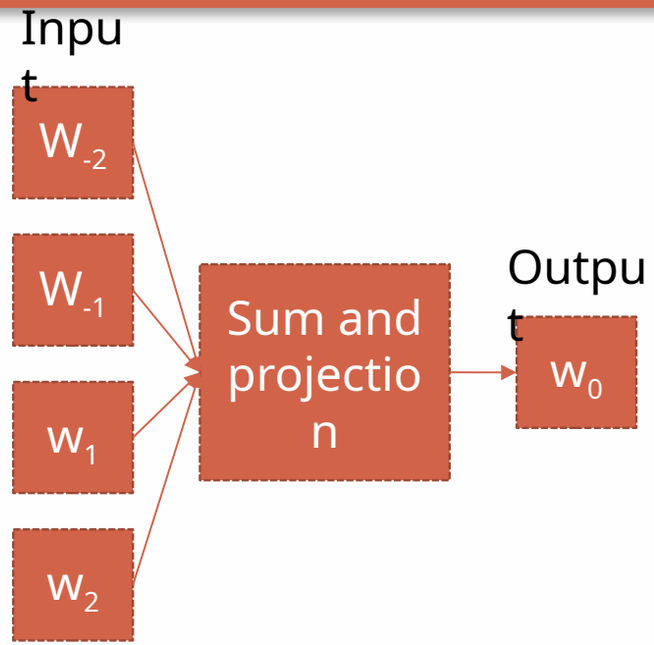
Word2Vec: Data generation (window size = 2)

Example: d1 = “king brave man” , d2 = “queen beautiful women”

Focus Word/ Target word	Word one hot encoding	Neighbor/Context word	Neighbor one hot encoding
king	[1,0,0,0,0,0]	brave	[0,1,1,0,0,0]
		man	
brave	[0,1,0,0,0,0]	king	[1,0,1,0,0,0]
		man	
man	[0,0,1,0,0,0]	king	[1,1,0,0,0,0]
		brave	
queen	[0,0,0,1,0,0]	beautiful	[0,0,0,0,1,1]
		women	
beautiful	[0,0,0,0,1,0]	queen	[0,0,0,1,0,1]
		women	
woman	[0,0,0,0,0,1]	queen	[0,0,0,1,1,0]
		beautiful	

Word2Vec: main context representation models

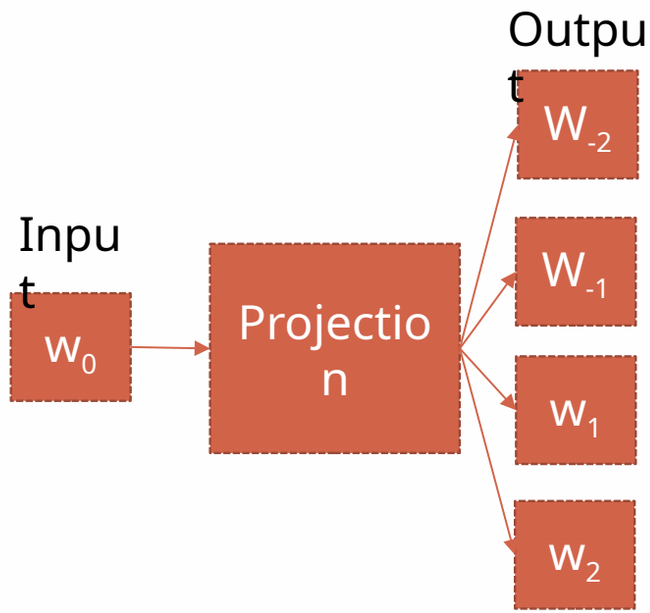
Continuous Bag of Words (CBOW)



Given Context word finds the target word

- Word2Vec is a predictive model.
- Will focus on Skip-gram model

Skip-Gram



Given target word finds the context word

Word2vec

- Word2vec is a method to create word embeddings
  - using a two-layer neural network.

    input layer, one hidden layer, and an output layer.
- The **input** of word2vec a text corpus
- Output is a set of vectors known as feature vectors that represent words in that corpus.
- More specifically

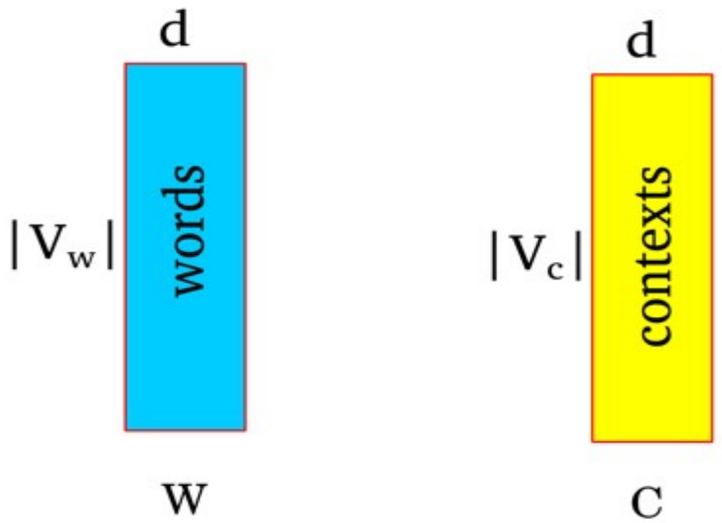
  “Word2Vec employs a shallow neural network consisting of an input layer, a single hidden layer, and an output layer.

Word2vec

- The Word2vec objective function causes the words that have a similar context to have similar embeddings
  - Makes their vectors similar
  - Thus, in this vector space, these words are close.
  - Mathematically, the cosine of the angle between such vectors should be close to 1, i.e., the angle is close to 0.

How does word2Vec work?

- Represent each target word as a ***d-dimensional*** vector.
- Represent each context as a ***d-dimensional*** vector.
- Arrange vectors in two matrices, W and C.

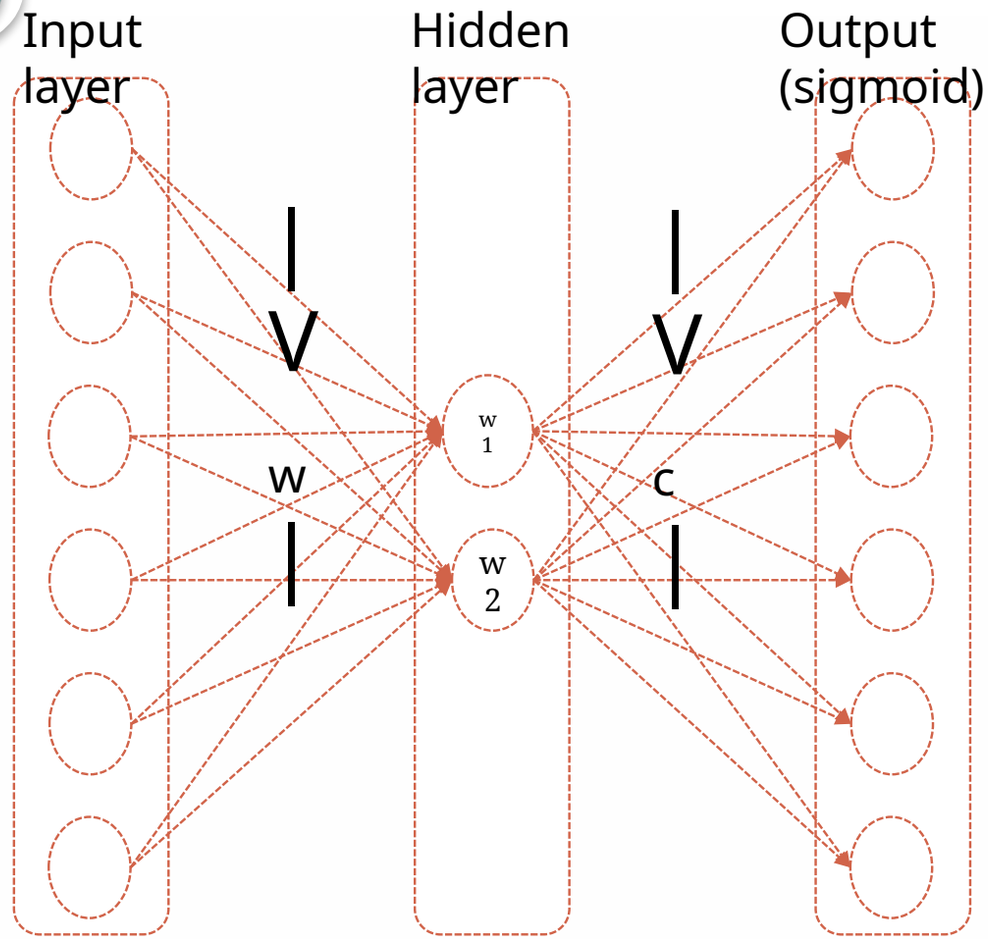


Word2Vec : Neural Network representation

- How do shallow neural networks learn dense word embeddings?
  - Skip-Gram / CBOW -> Objective Function -> Embedding Matrix
- The training task is prediction
  - Skip-Gram predicts context words from a target word;

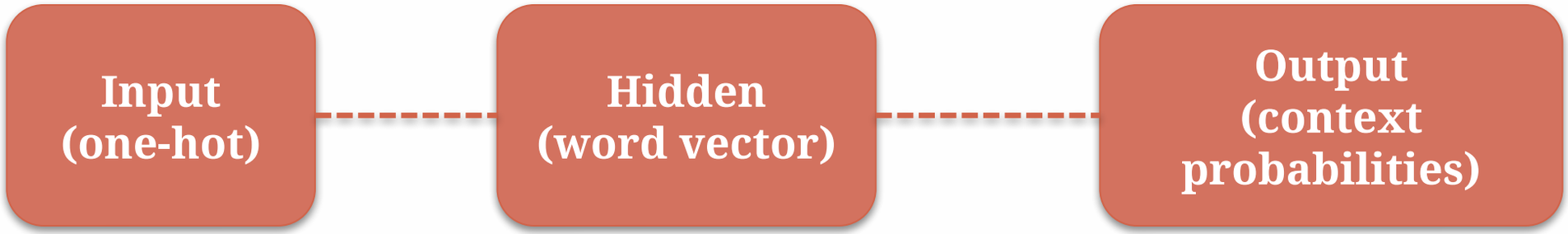
    maximize P(context | target)
  - CBOW predicts the target from context.

    maximize P(target | context)
- After training, the learned input-to-hidden weight matrix is used as the word-embedding matrix.



Word2Vec : Neural Network Representation

Sentence: "The cat sits on the mat"



cat = [0, 1, 0, 0, 0]

V_{cat} = [0.7, 0.8, 0.3]

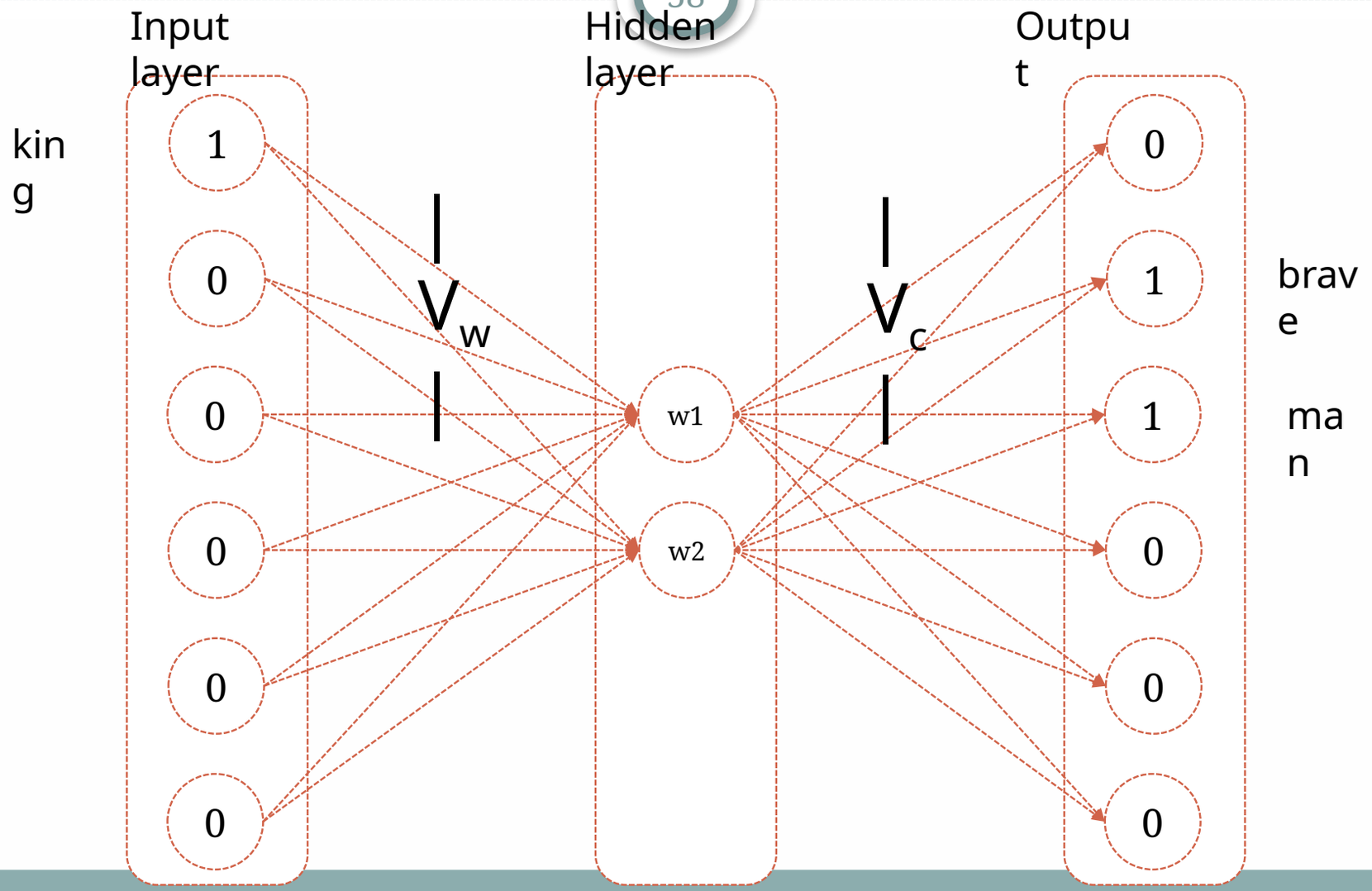
P(the), P(sits), P(on), ...

The hidden representation is the learned word embedding.

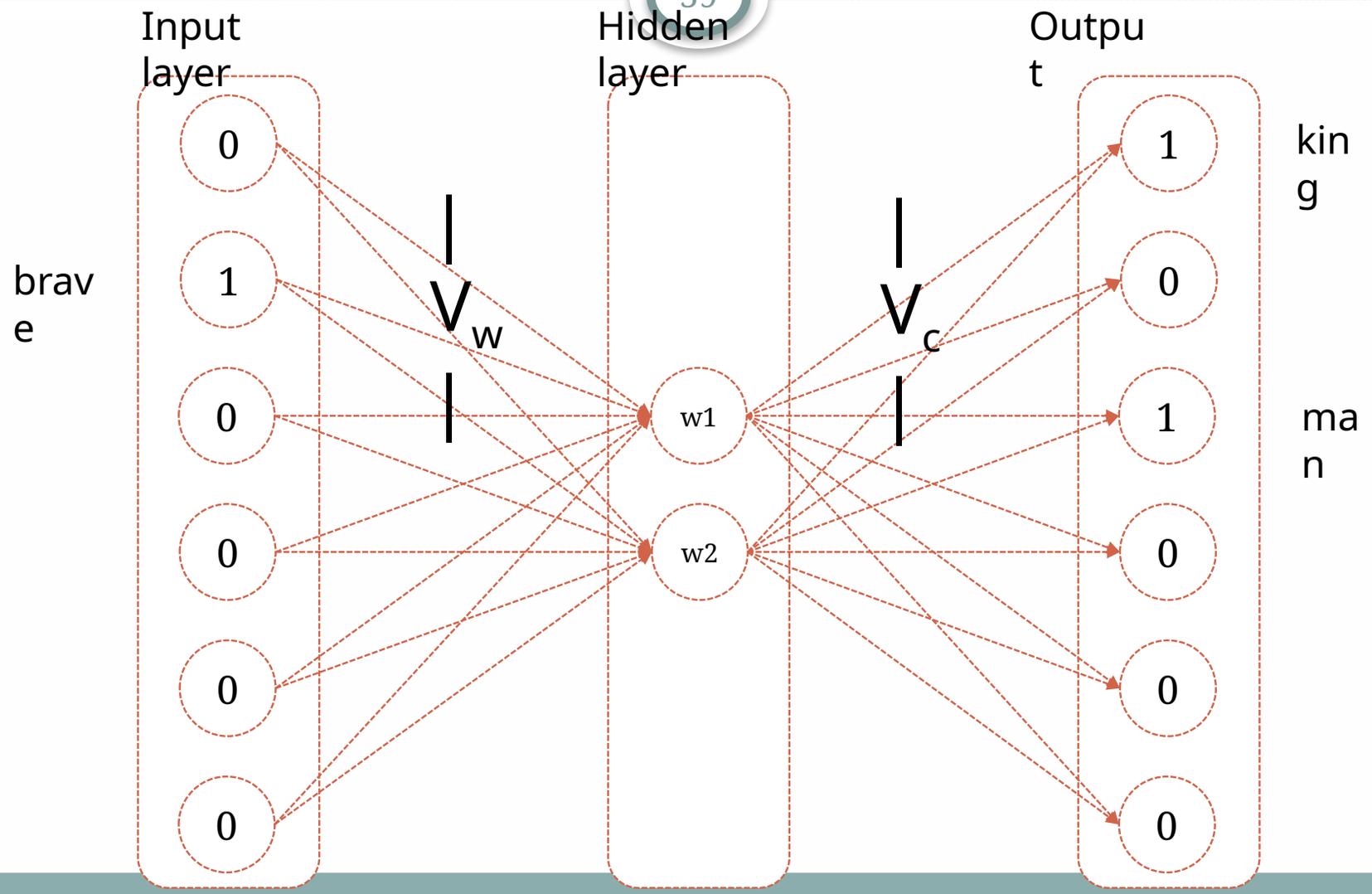
Target word
cat → Predict nearby context words
the sits on

Goal: maximize P(context | target)

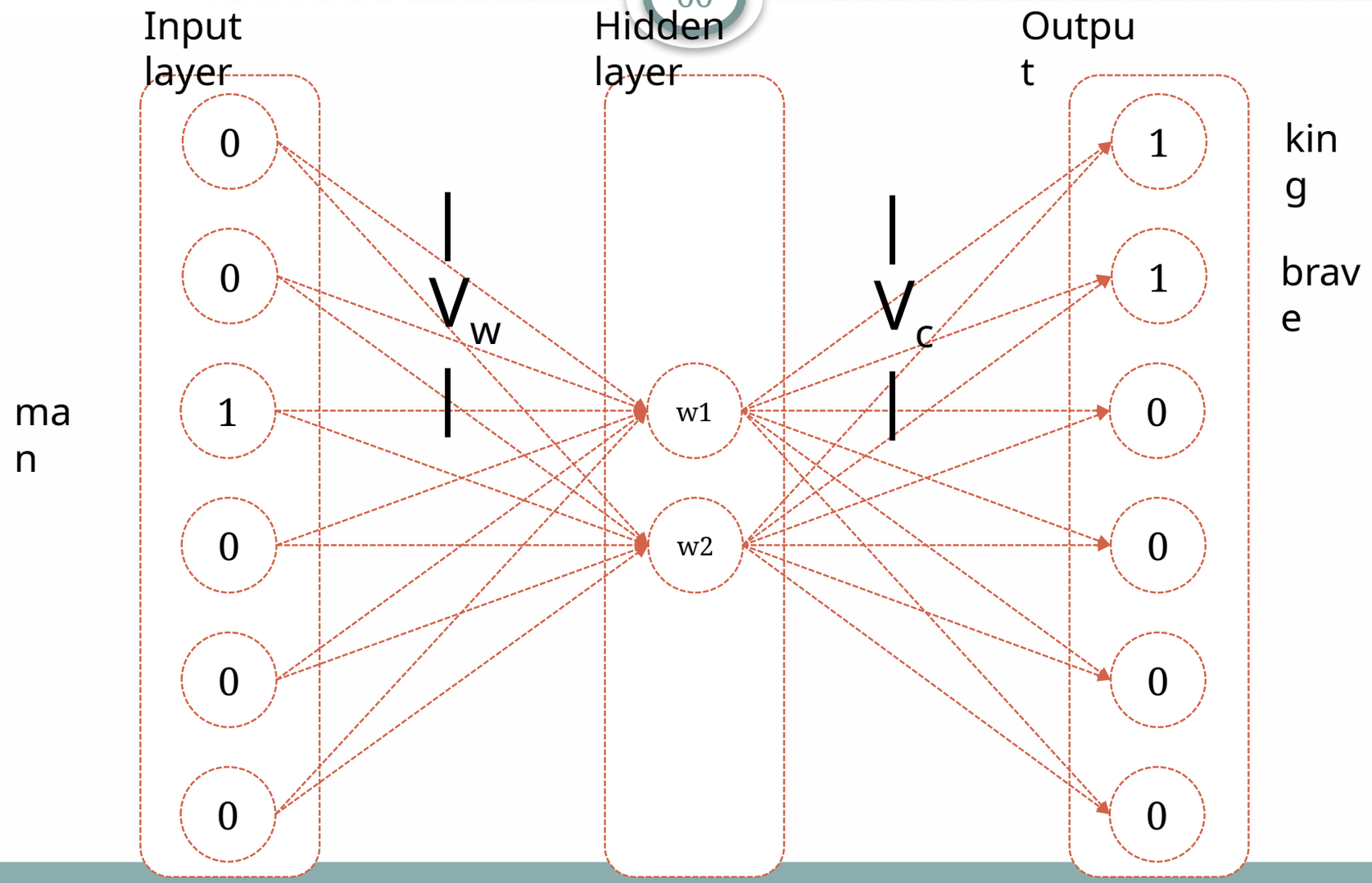
Word2Vec : Neural Network representation



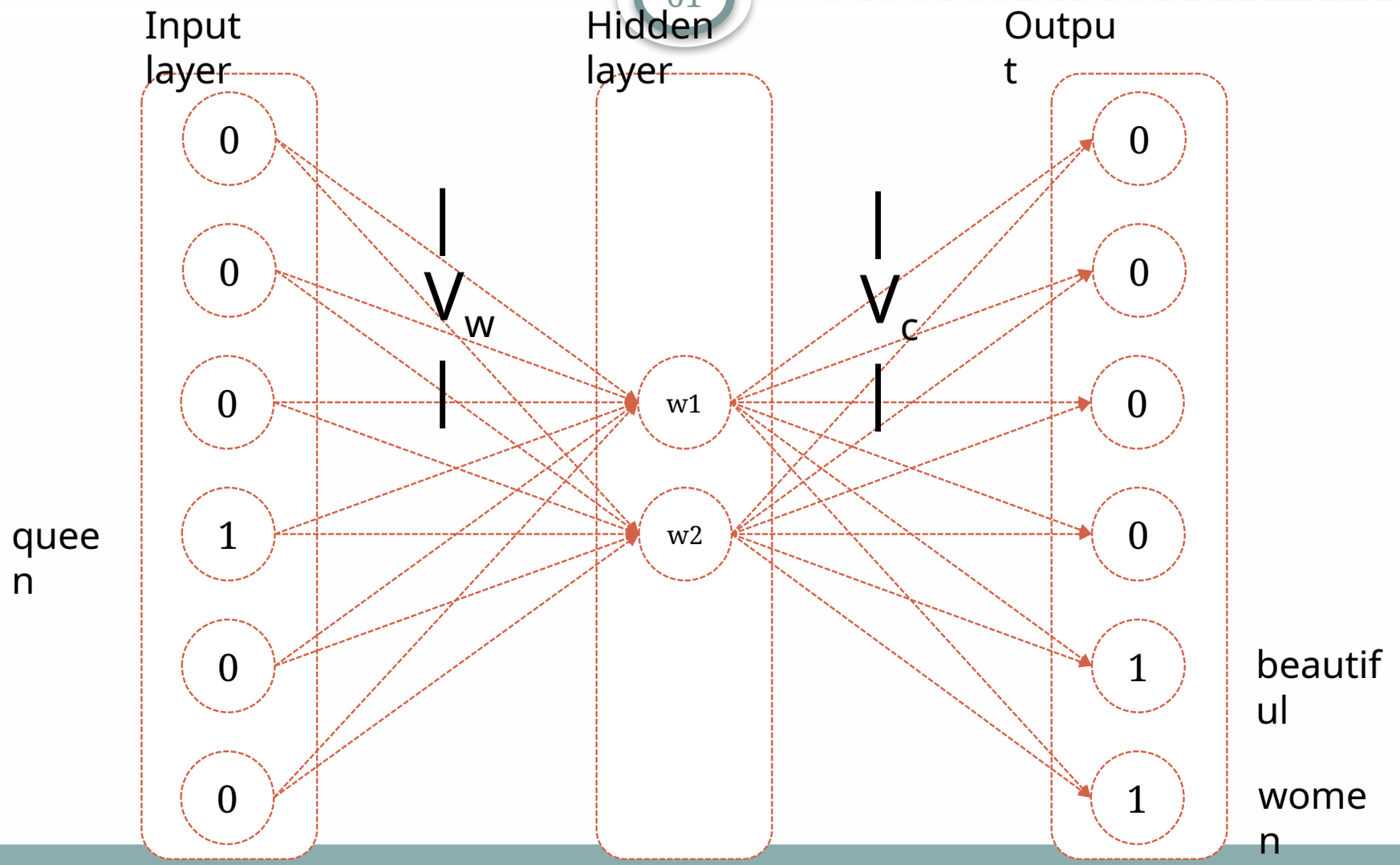
Word2Vec : Neural Network representation



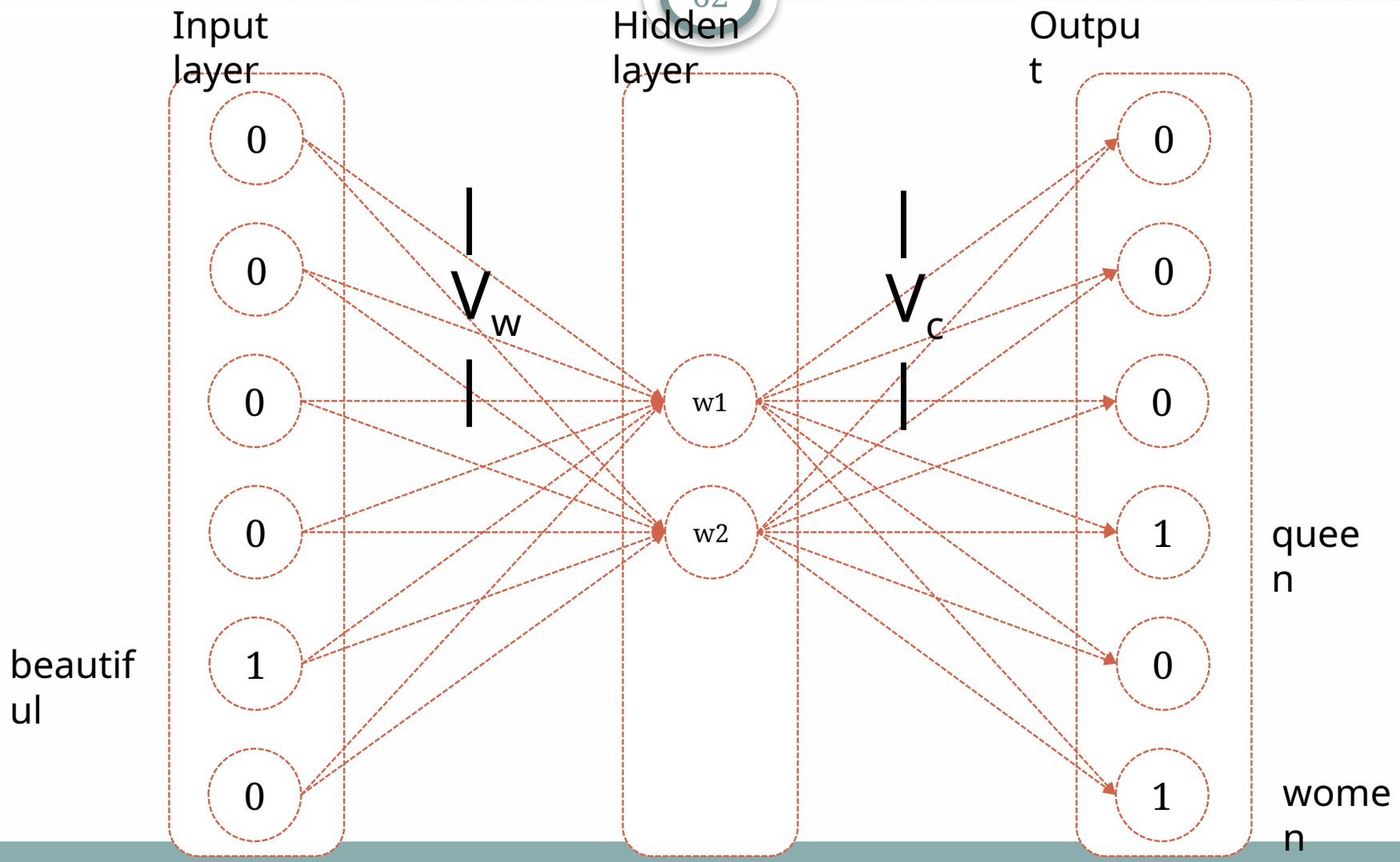
Word2Vec : Neural Network representation



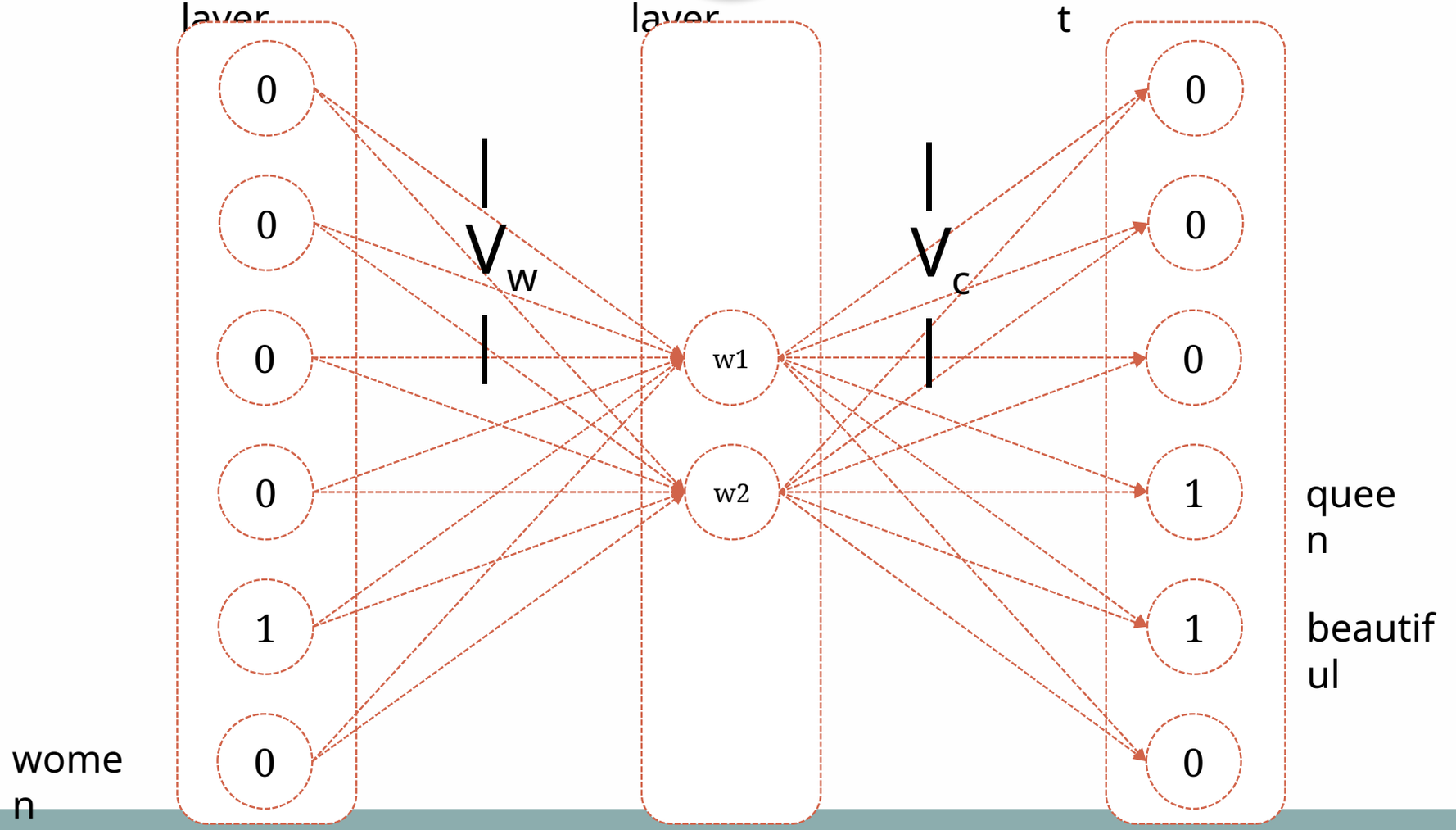
Word2Vec : Neural Network representation



Word2Vec : Neural Network representation

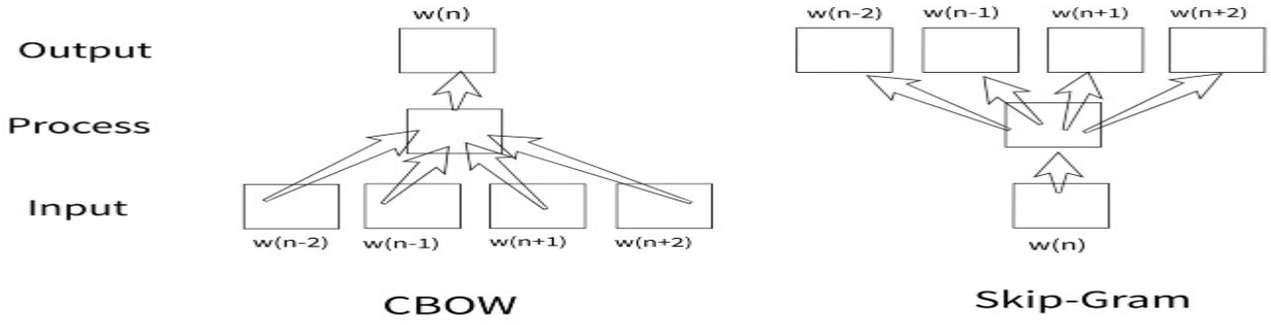


Word2Vec : Neural Network representation



Word2Vec

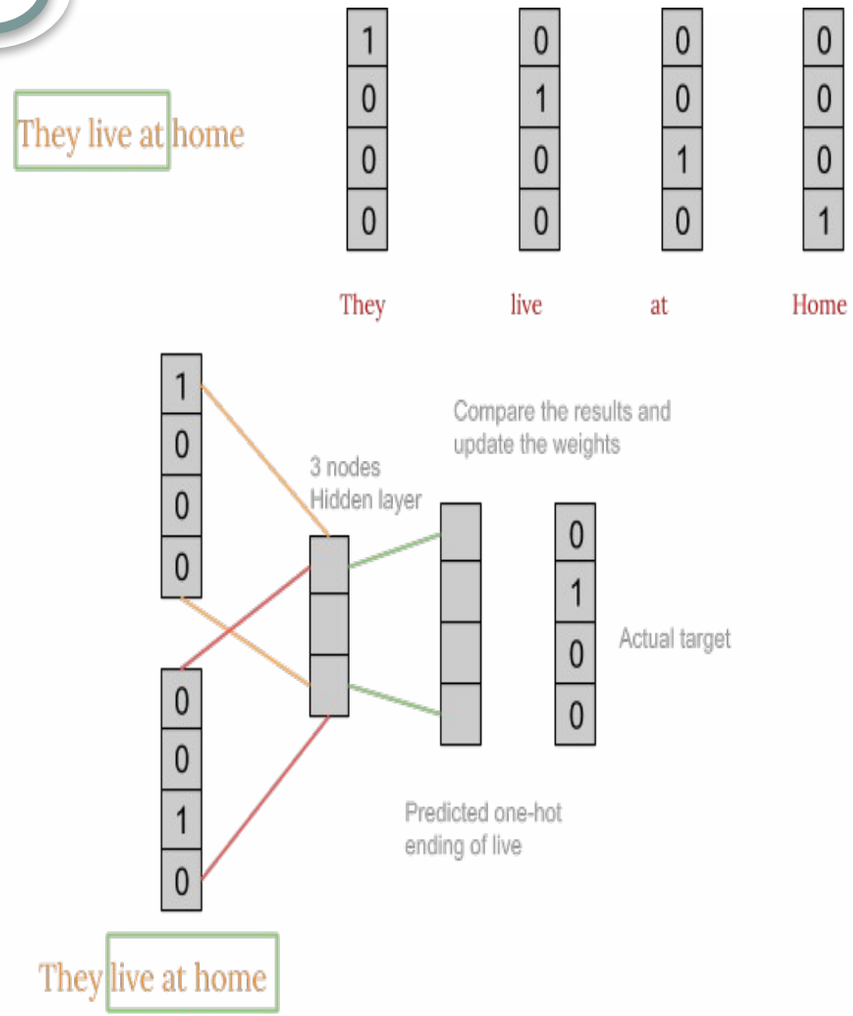
- Word2vec is not a single algorithm but a combination of two techniques –
  - CBOW(Continuous bag of words)
  - Skip-gram
- Both of these are shallow neural networks which
  - map word(s) to the target variable which is also word(s).
- Both of these techniques
  - learn weights that act as word vector representations.



Word2Vec : CBOW

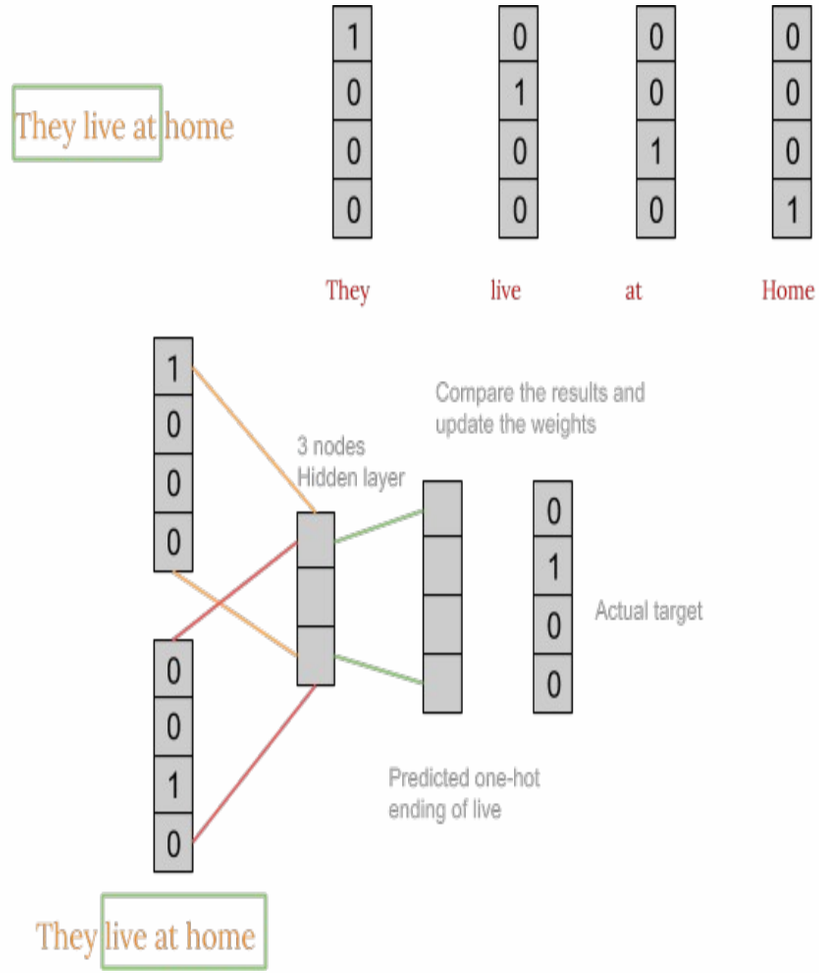
Continuous Bag-of-Words (CBOW)

- CBOW predicts the probability of a word to occur given the words surrounding it.
  - we will take a single context word and try to predict a single target word.
  - we will consider that the corpus contains only one sentence, 'They live at home'.
- The basic idea is
  - given the input and output hence learn the weights to minimize the errors



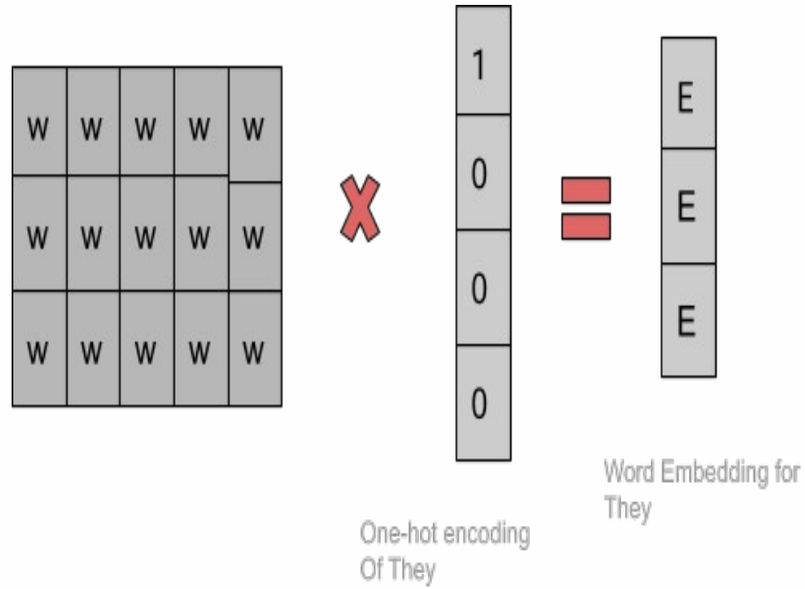
CBOW

- First, we convert each word into a one-hot encoding form.
- we'll not consider all the words in the sentence but will only take certain words that are in a window.
  - For example for a window size equal to three, we only consider three words in a sentence.
  - The middle word is to be predicted and the surrounding two words are fed into the neural network as context.
  - The window is then slide and the process is repeated again.

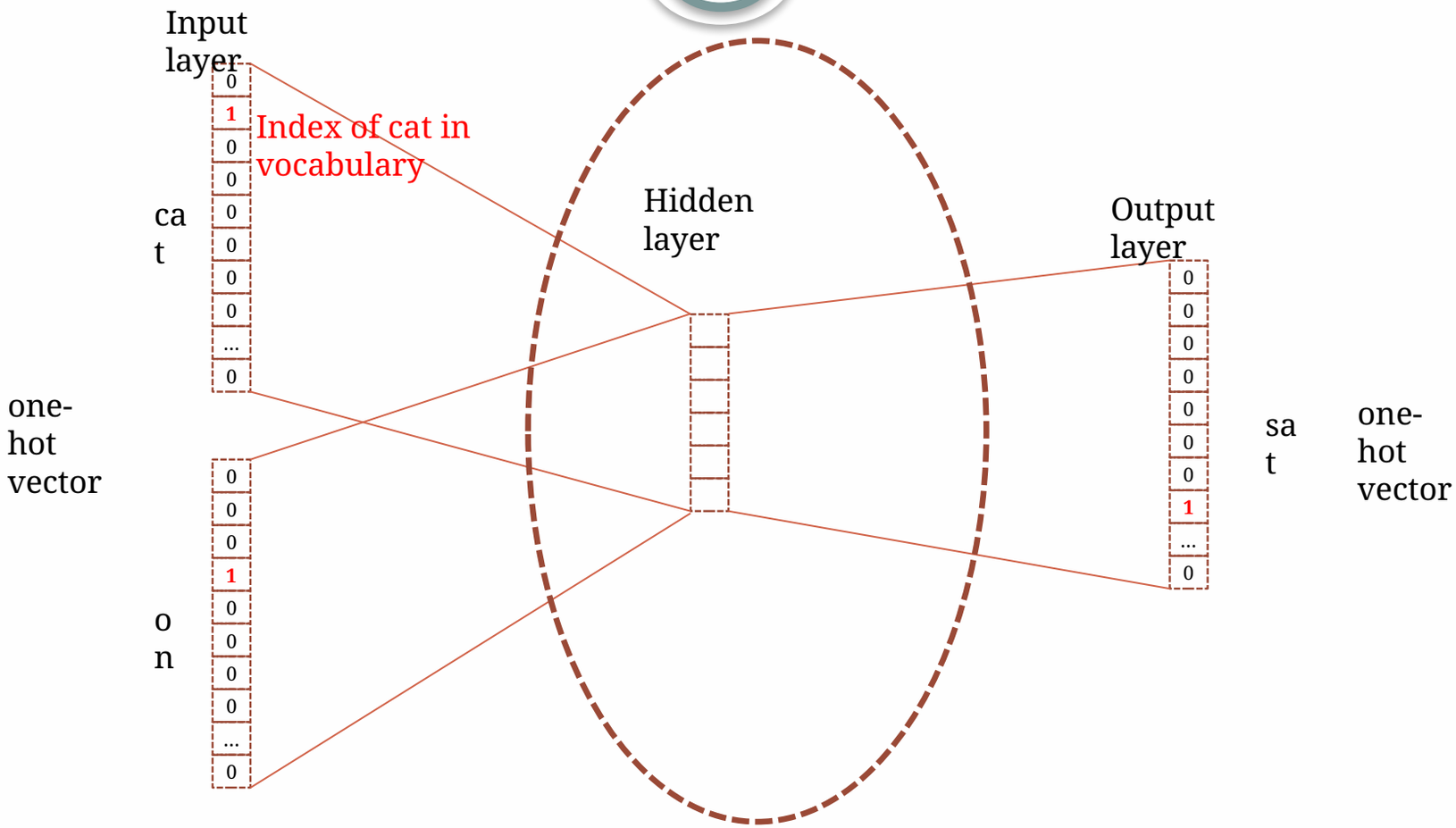


CBOW

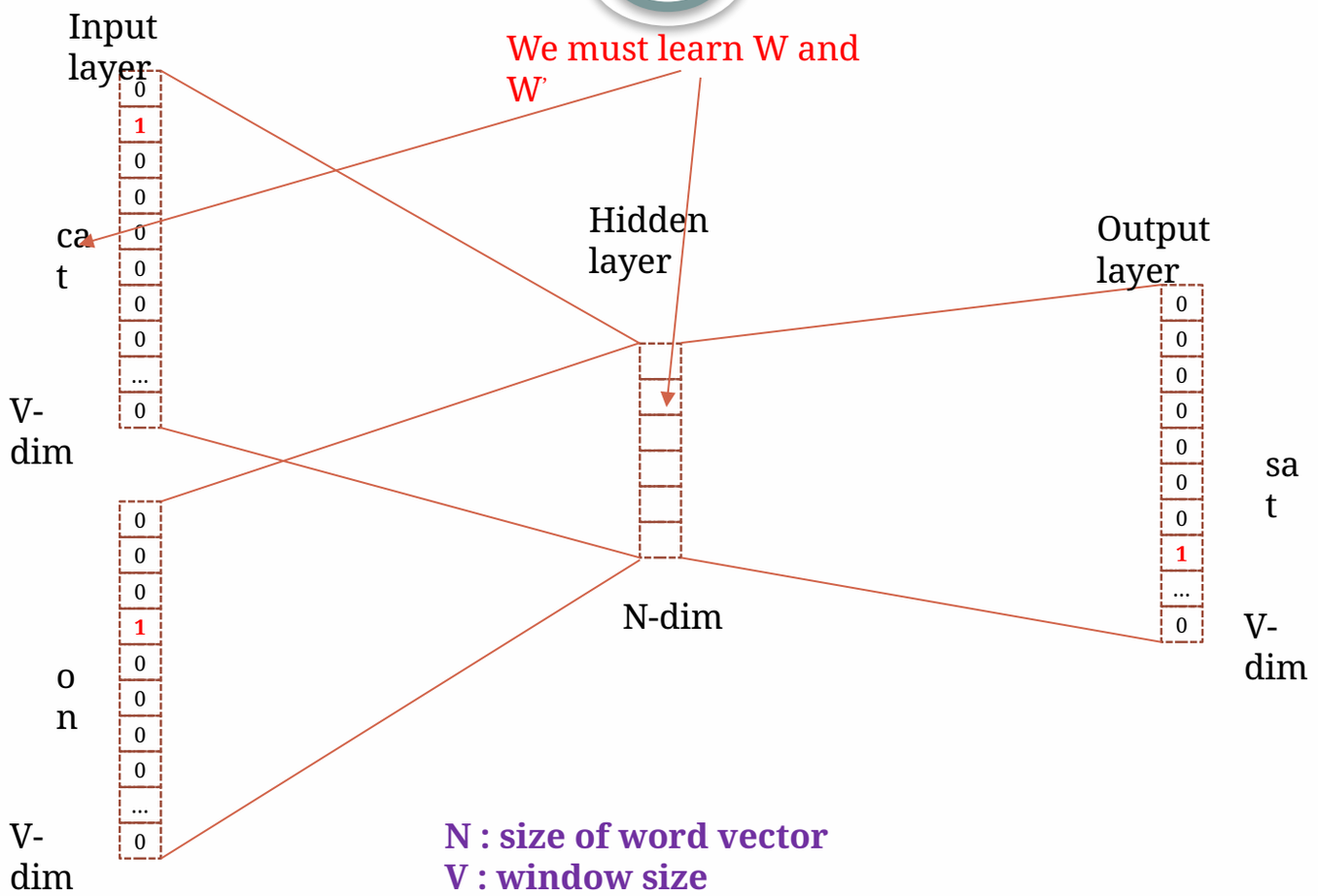
- After training the network repeatedly by sliding the window, we get weights which we use to get the embeddings.
- Usually, we take a window size of around 8-10 words and have a vector size of 300.



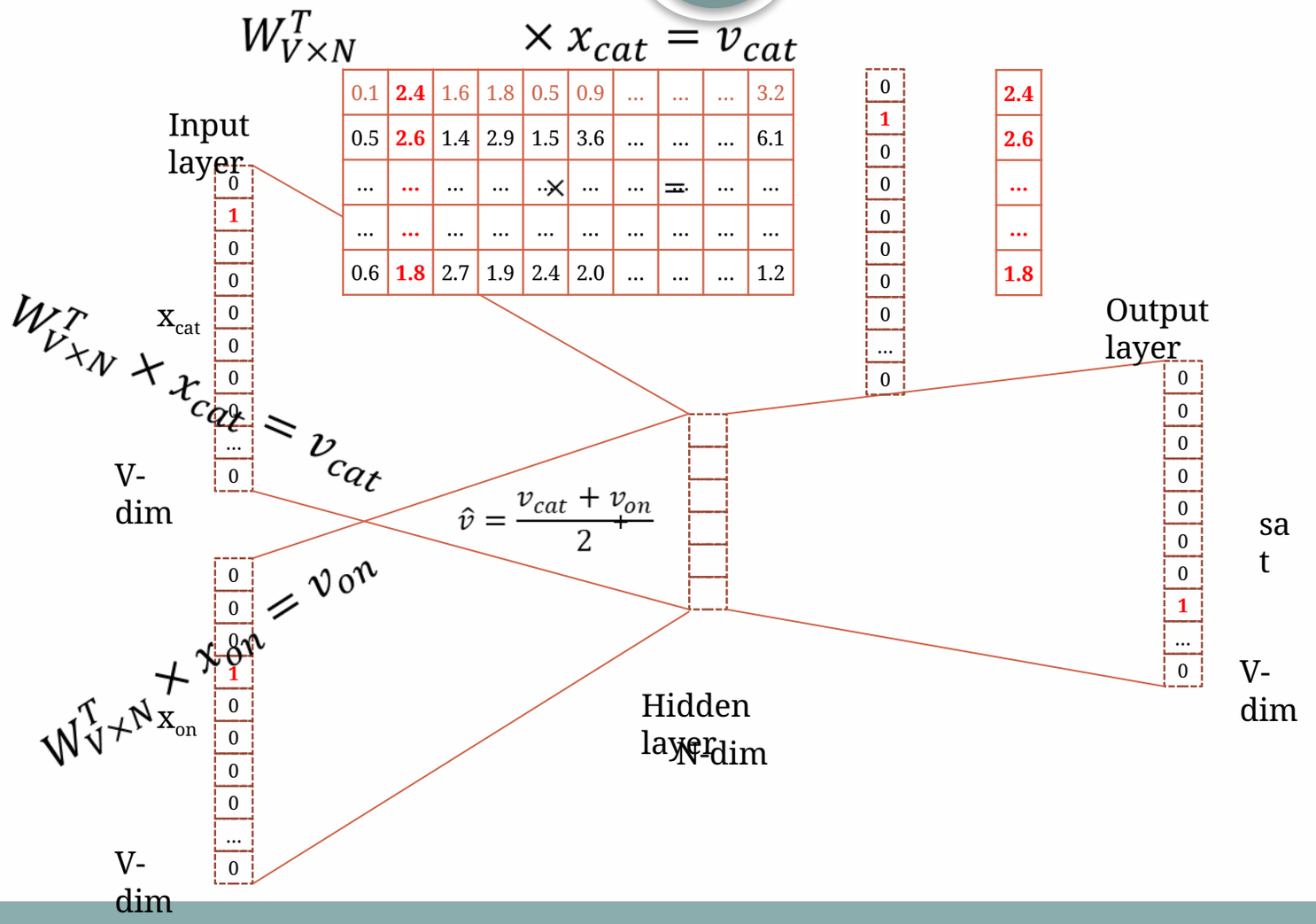
CBOW


Input layer
Index of cat in vocabulary
Hidden layer
Output layer
ca t
one-hot vector
o n
sa t
one-hot vector

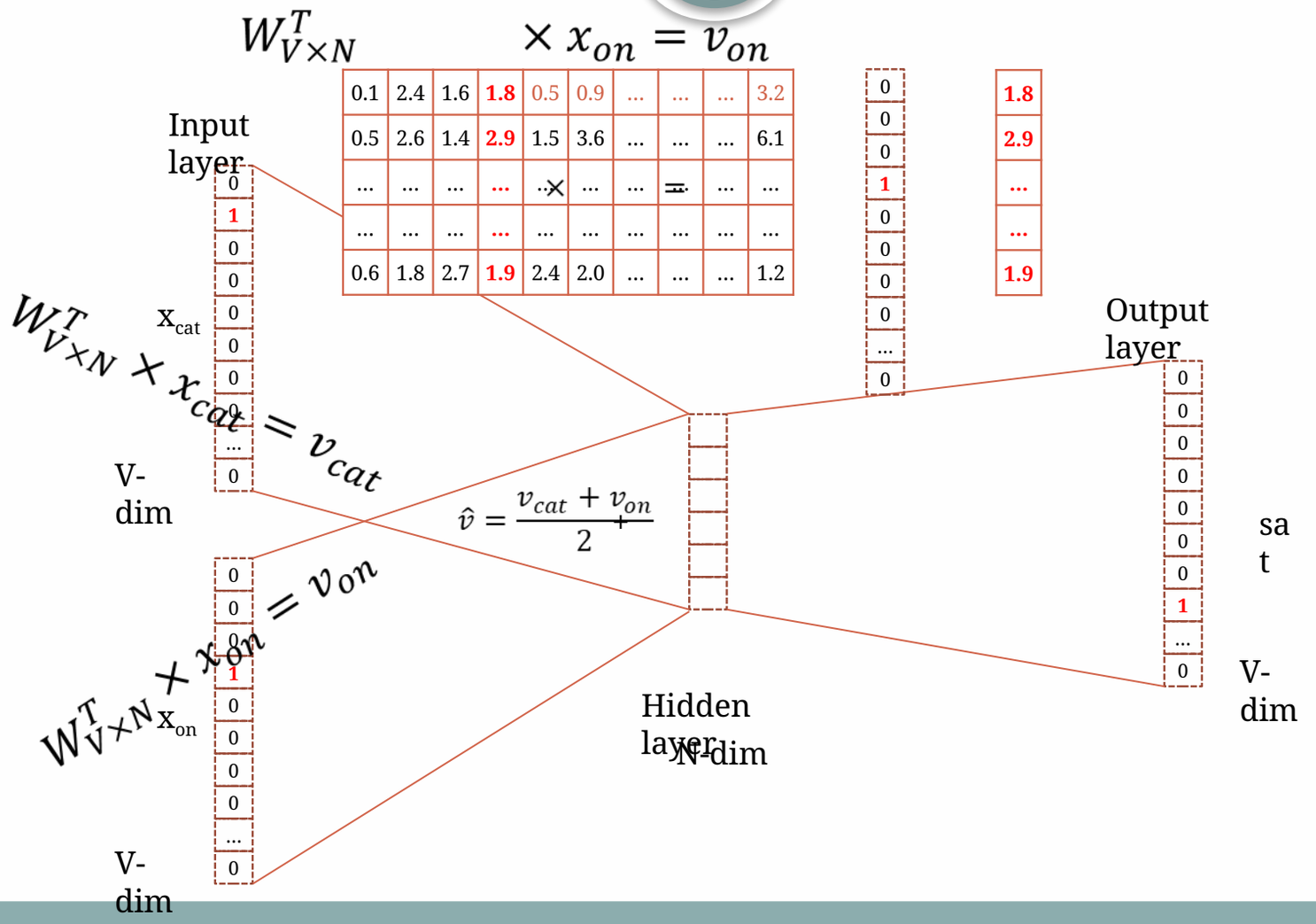
CBOW


Input layer
We must learn W and W'
Hidden layer
Output layer
ca
t
V-
dim
o
n
V-
dim
N-dim
sa
t
V-
dim
N : size of word vector
V : window size

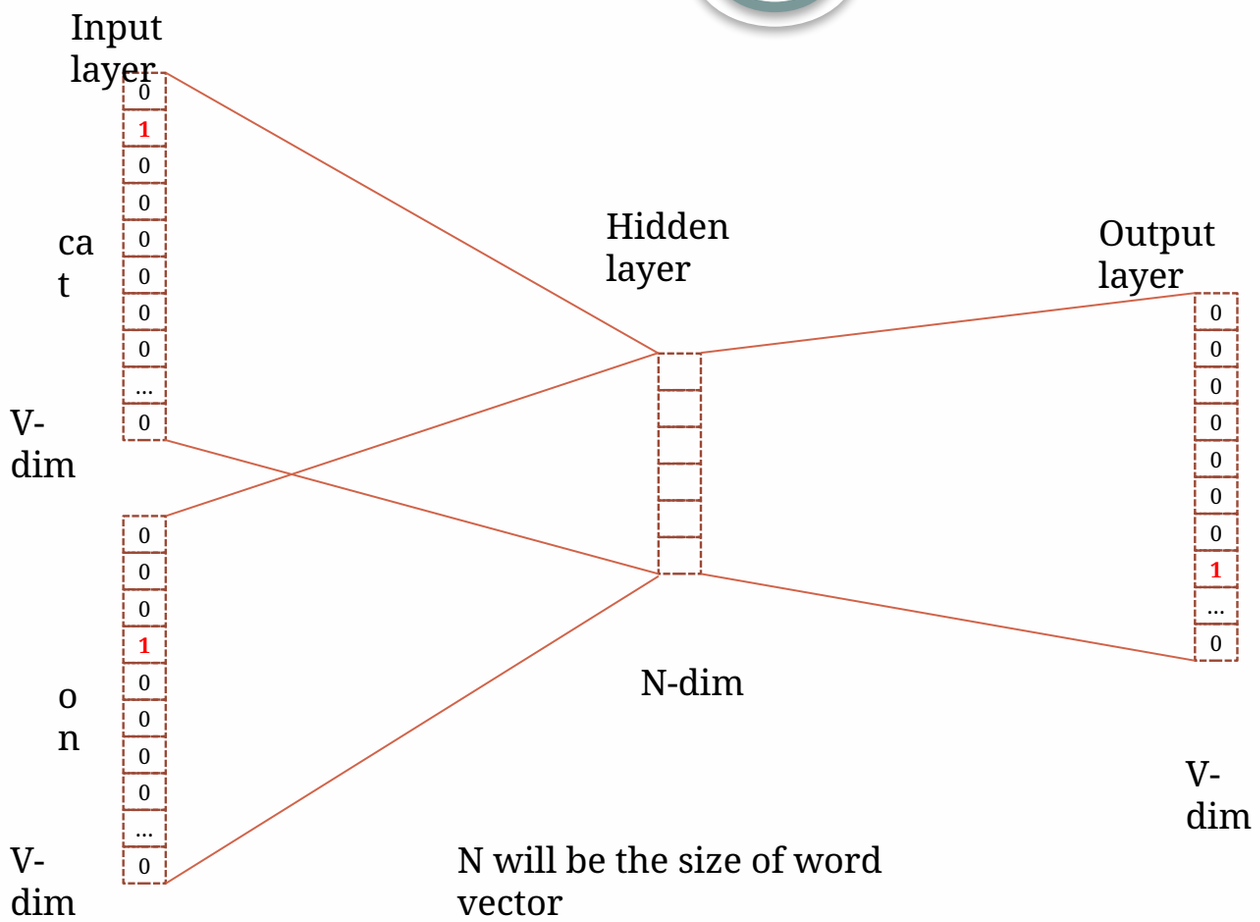
CBOW


$W^T_{V\times N}$
$\times x_{cat} = v_{cat}$
Input layer
x_{cat}
$W^T_{V\times N} \times x_{cat} = v_{cat}$
V-dim
$\hat{v} = \frac{v_{cat} + v_{on}}{2}$
$W^T_{V\times N} \times x_{on} = v_{on}$
x_{on}
V-dim
Hidden layer
N-dim
Output layer
sa
t
V-dim

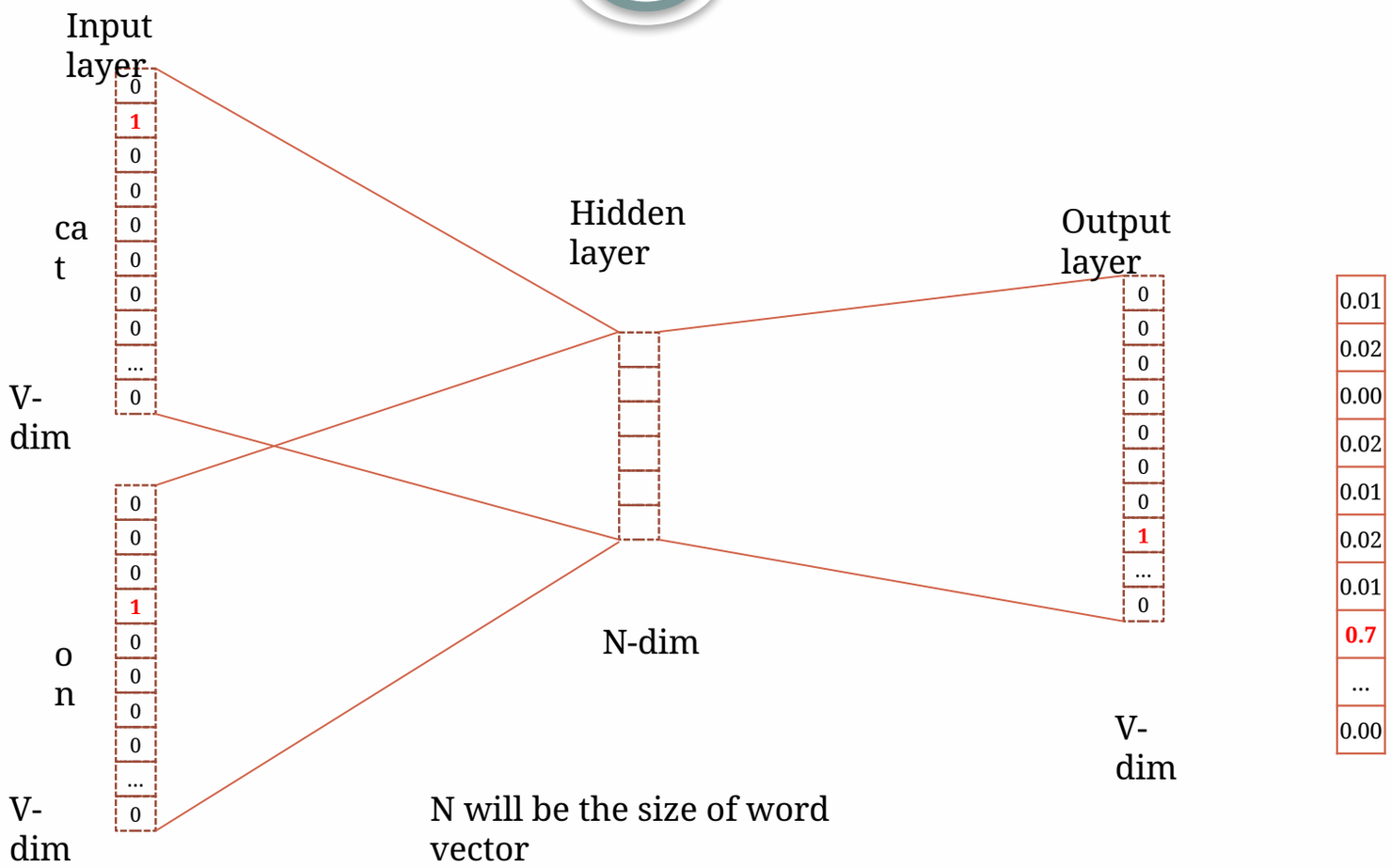
CBOW


$W^T_{V\times N}$
$\times x_{on} = v_{on}$
Input layer
x_{cat}
$W^T_{V\times N} \times x_{cat} = v_{cat}$
V-dim
$\hat{v} = \frac{v_{cat} + v_{on}}{2}$
$W^T_{V\times N} \times x_{on} = v_{on}$
x_{on}
V-dim
Hidden layer
N-dim
Output layer
sa
t
V-dim

CBOW

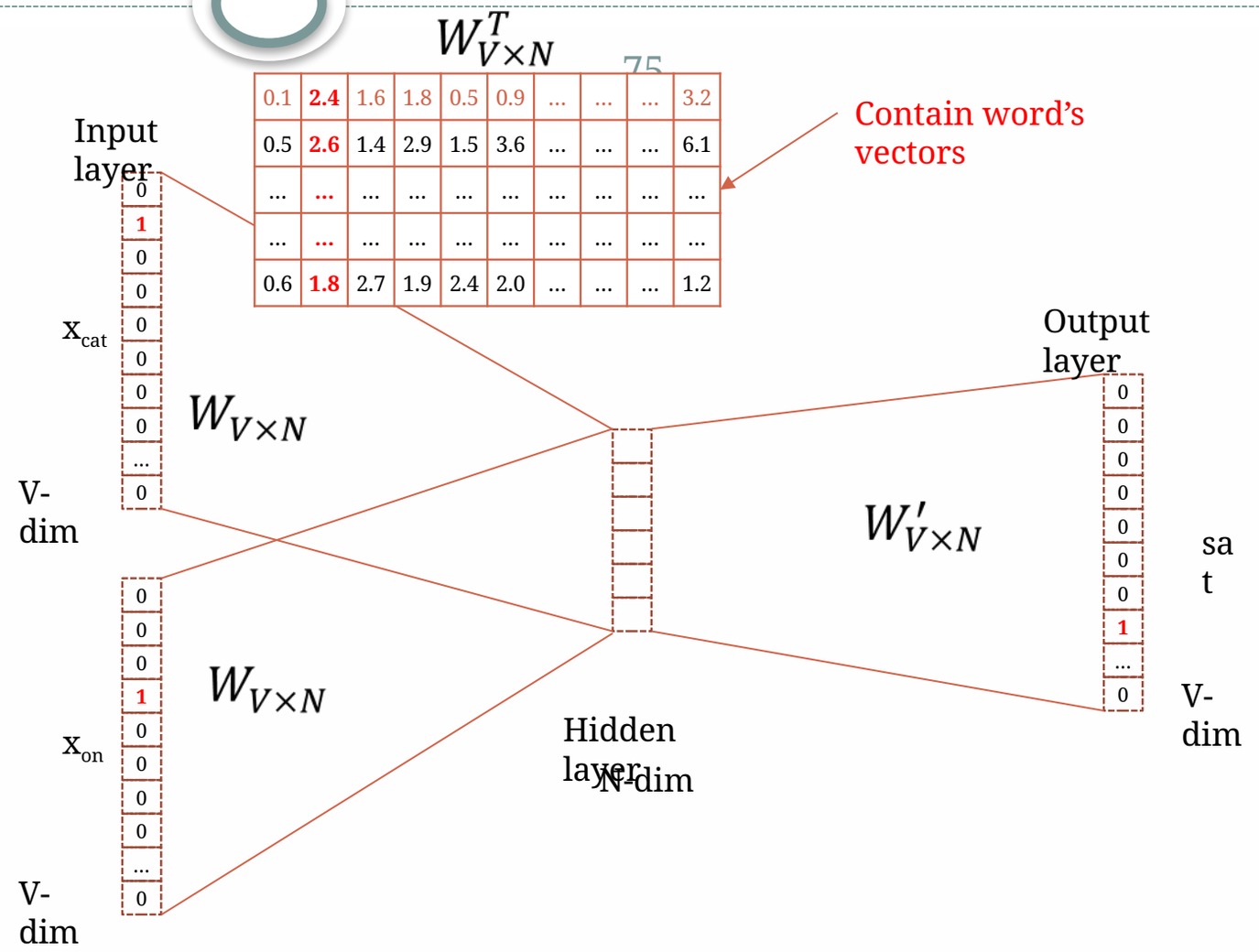

Input layer
ca t
V-dim
o n
V-dim
Hidden layer
N-dim
Output layer
V-dim
N will be the size of word vector

CBOW

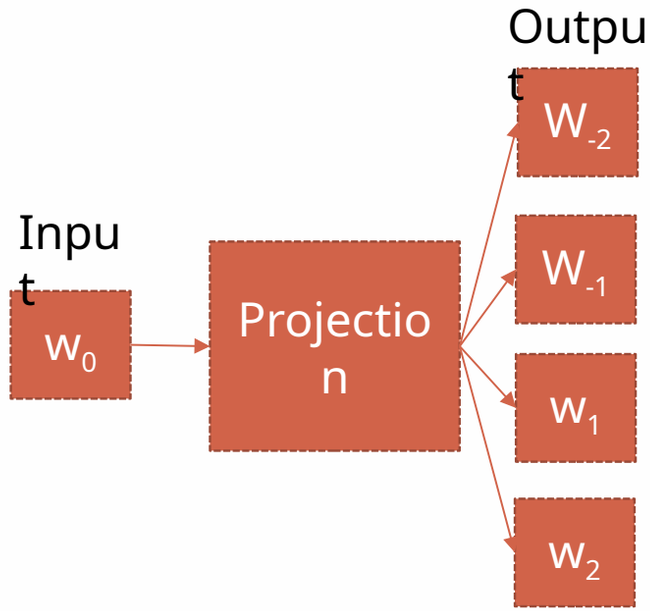

Input layer
cat
V-dim
on
V-dim
Hidden layer
N-dim
Output layer
V-dim
0.01
0.02
0.00
0.02
0.01
0.02
0.01
0.7
0.00
N will be the size of word vector

CBOW

- W contains input word vectors.
- W′ contains output word vectors.
- We can consider either W or W′ as the word's representation. or
- We can take the average.



Word2Vec : Skip-gram



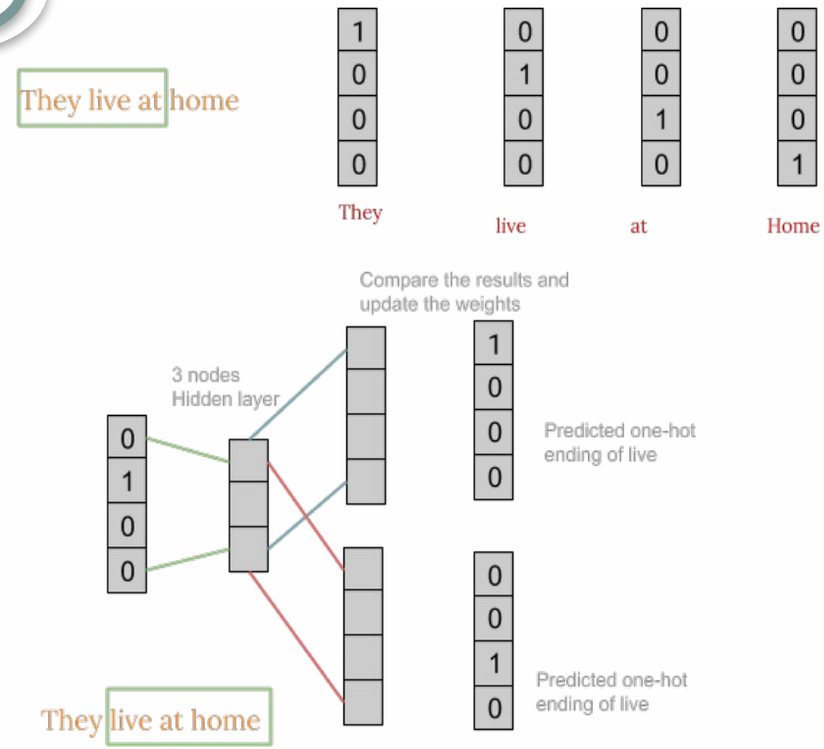
Skip-gram

- Given a specific word in the middle of a sentence
  - The skip gram tells us the probability for every word in our vocabulary to the “nearby word” of the specific word.
  - "nearby", is actually a "window size“

    For example, if you gave the trained network the input word “Soviet”, the output probabilities are going to be much higher for words “Union” and “Russia” than for unrelated words like “watermelon” and “kangaroo”.

Skip-gram model

- The Skip-gram model tries to achieve the reverse of what the CBOW model does.
  - tries to predict the context words (surrounding words) given a target word (the center word)
- The working of the skip-gram model is quite similar to the CBOW
  - but there is just a difference in the architecture of the network and the way the weight matrix is generated



Approach: predict if candidate word *c* is a "neighbor"



1. Treat the target word *t* and a neighboring context word *c* as positive examples.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish between these two cases
4. Use the learned weights as the embeddings

Skip-Gram Training Data

- Assume a +/- 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

[target]

c1 c2 c3 c4

Skip-Gram Classifier

- (assuming a +/- 2 word window)

…lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 [target] c3 c4

- Goal: train a classifier that is given a candidate (**w**ord, **c**ontext) pair

  (apricot, jam)

  (apricot, aardvark)

  …

  And assigns each pair a probability:

  $P(+|w, c)$

  $P(-|w, c) = 1 - P(+|w, c)$

Similarity is computed from dot product

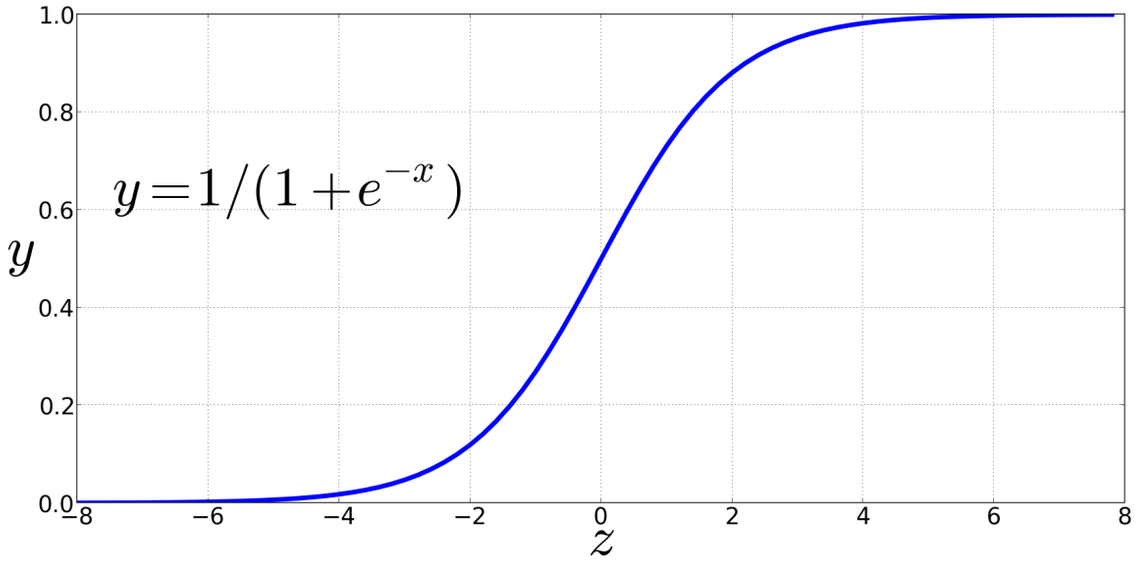


- Remember: two vectors are similar if they have a high dot product
  - Cosine is just a normalized dot product
- So
  - Similarity(w, c) $\propto$ $w \cdot c$
- We'll need to normalize to get a probability
  - (cosine isn't a probability either)

Turning dot product into a probability

- The sigmoid lies between 0 and 1:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Turning dot products into probabilities

- Sim(w,c) $\approx$ w $\cdot$ c
- To turn this into a probability
- We'll use the sigmoid from logistic regression

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

$$P(-|w,c) = 1 - P(+|w,c)$$

$$= \sigma(-c \cdot w) = \frac{1}{1+\exp(c \cdot w)}$$

How Skip-Gram Classifier computes $P(+|w, c)$

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$$

- This equation gives us the probability for one word, but there are many context words in the window.
- We'll assume independence and just multiply them

$$P(+|w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i \cdot w)$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i \cdot w)$$

Skip-gram classifier: summary

- A probabilistic classifier, given
  - a test target word w
  - its context window of L words $c_{1:L}$
- Estimates probability that w occurs in this window based on the similarity of w (embeddings) to $\boldsymbol{c}_{1:L}$ (embeddings).
- To compute this, we just need embeddings for all the words.

Skip-Gram Training data



...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

This example has a target word *w* (apricot), and 4 context words in the *L* = ±2 window, resulting in 4 positive training instances

Skip-Gram Training data



...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

- **positive examples +**

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

The noise words are chosen according to their weighted unigram frequency.

Skipgram: how to learn vectors



- Given the set of positive and negative training instances, and an initial set of embedding vectors
- The goal of learning is to adjust those word vectors such that
  - **Maximize** the similarity of the target word, context word pairs (w, c_{pos}) drawn from the positive data
  - **Minimize** the similarity of the (w, c_{neg}) pairs drawn from the negative data.

    (where similarity(w, c) $\propto w \cdot c$)

Loss function

- Maximize the similarity of the target with the actual context words, and minimize the similarity of the target with the *k* negative-sampled non-neighbor words.
- To do so we minimize the loss function (hence the −);

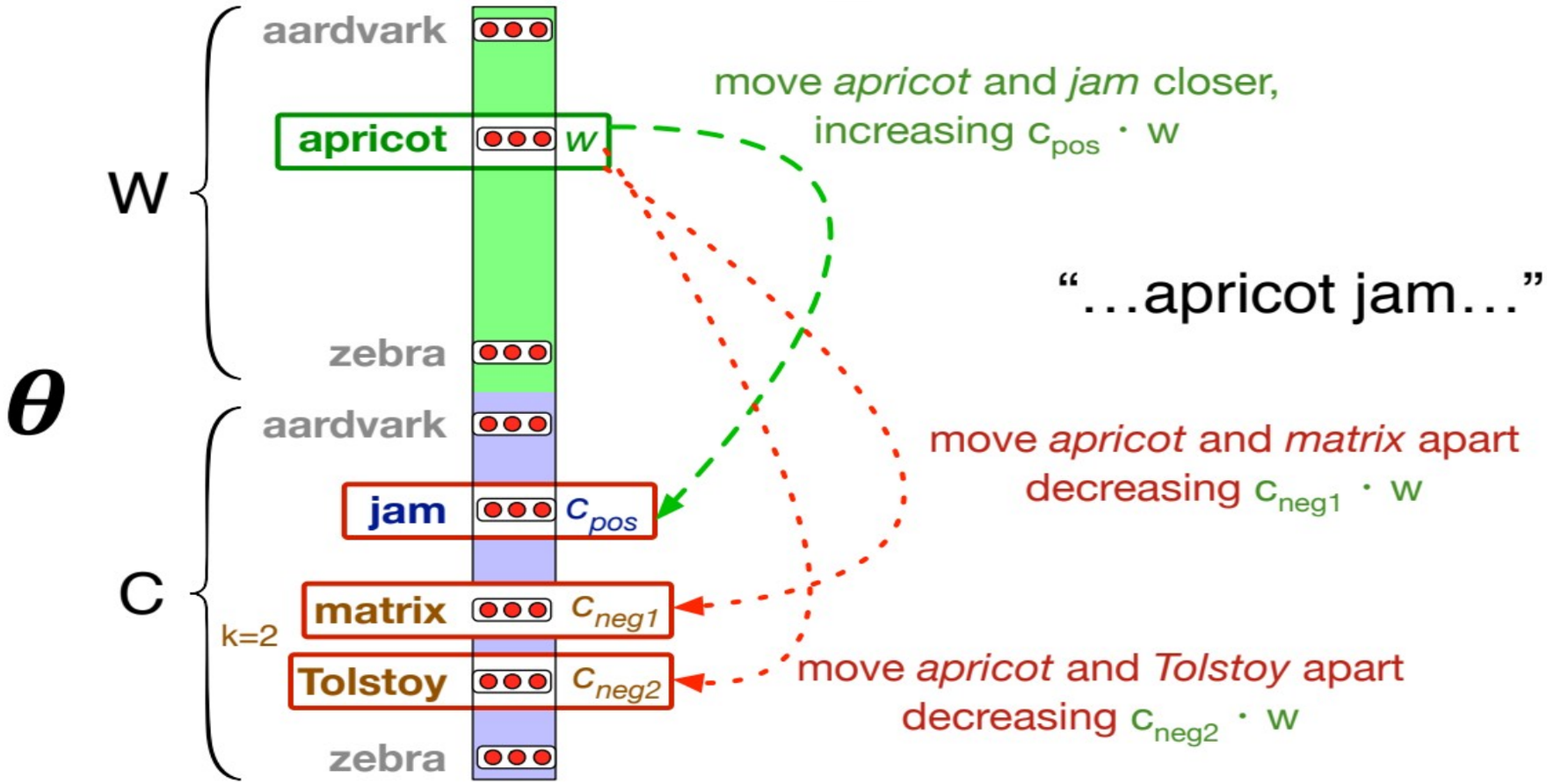
$$
\begin{aligned}
L_{CE} &= -\log\left[P(+|w,c_{pos})\prod_{i=1}^{k}P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log\left(1-P(+|w,c_{neg_i})\right)\right] \\
&= -\left[\log \sigma(c_{pos}\cdot w) + \sum_{i=1}^{k}\log\sigma(-c_{neg_i}\cdot w)\right]
\end{aligned}
$$

Learning the classifier



- How to learn?
  - We minimize this loss function using stochastic gradient descent.
- We'll adjust the word weights to
  - make the positive pairs more likely
  - the negative pairs less likely,
  - over the entire training set.

Intuition of one step of gradient descent



Reminder: gradient descent

- At each step
  - We move in the reverse direction from the gradient of the loss function
  - We move the value of this gradient $\frac{d}{dw} L(f(x;w),y)$ weighted by a **learning rate** η
  - Higher learning rate means move w faster

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x;w),y)$$

The derivatives of the loss function

$$L_{CE} = -\left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^{k} \log \sigma(-c_{neg_i} \cdot w)\right]$$

$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(c_{pos} \cdot w) - 1]w$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(c_{neg} \cdot w)]w$$

$$\frac{\partial L_{CE}}{\partial w} = [\sigma(c_{pos} \cdot w) - 1]c_{pos} + \sum_{i=1}^{k} [\sigma(c_{neg_i} \cdot w)]c_{neg_i}$$

Update equation in SGD

Start with randomly initialized C and W matrices, then incrementally do updates

$$c_{pos}^{j+1} = c_{pos}^{j} - \eta[\sigma(c_{pos}^{j} \cdot w^{j}) - 1]w^{j}$$

$$c_{neg}^{j+1} = c_{neg}^{j} - \eta[\sigma(c_{neg}^{j} \cdot w^{j})]w^{j}$$

$$w^{j+1} = w^{j} - \eta\left[[\sigma(c_{pos} \cdot w^{j}) - 1]c_{pos} + \sum_{i=1}^{k}[\sigma(c_{neg_i} \cdot w^{j})]c_{neg_i}\right]$$

The learning algorithm starts with randomly initialized *W* and *C* matrices, and then walks through the training corpus using gradient descent to move *W* and *C* so as to maximize the objective by making the updates here.

Two sets of embeddings

- SG learns two sets of embeddings
  - Target embeddings matrix W and Context embedding matrix C
- It's common to just add them together, representing word *i* as the vector $w_i + c_i$

Summary

- Start with V random d-dimensional vectors as initial embeddings
- Train a classifier based on embedding similarity
  - Take a corpus and take pairs of words that co-occur as positive examples
  - Take pairs of words that don't co-occur as negative examples
  - Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
  - Throw away the classifier and keep the embeddings.

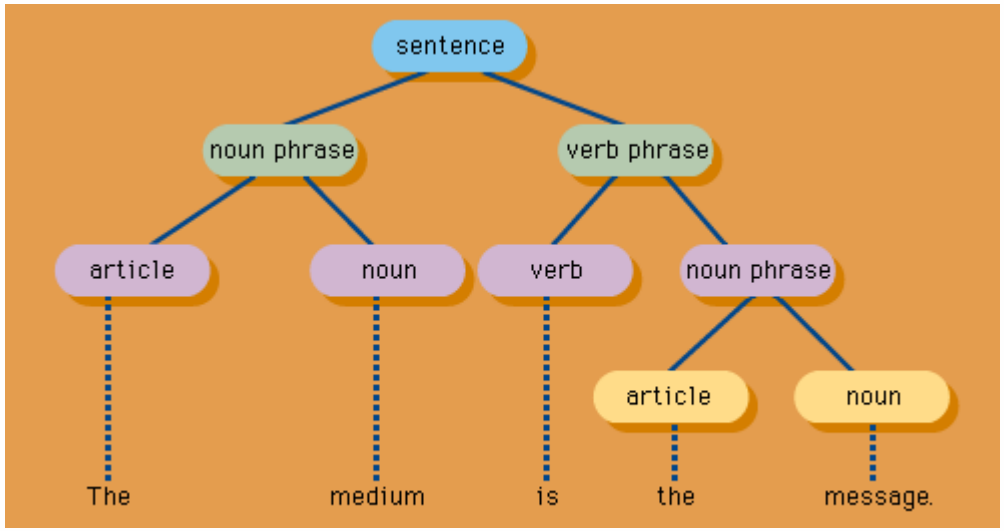
THANK YOU

Parsing Natural Language

DR. K. M. AZHARUL HASAN
DEPT. OF CSE, KUET

Parsing

- Parsing is the process of taking a string and a grammar and returning a (or many?) parse tree(s) for that string
  - If the parse tree can be generated, then the string follows the grammar



Background

- Syntactic parsing
  - The task of recognizing a sentence and assigning a syntactic structure to it
- Parse trees are useful in applications such as
  - Grammar checking
  - Semantic analysis
  - Machine translation
  - Question answering
  - Information extraction

Parsing Types

- Syntactic Parsing
  - Builds a parse tree based on grammar rules defined in a context-free grammar (CFG).
  - Constituency Parsing

    Divides sentences into sub-phrases (constituents) using a hierarchical tree structure, identifying noun phrases, verb phrases, and clauses.
  - Dependency Parsing

    Focuses on the relationships between words (dependencies), such as subject-verb-object relations, to create a structure showing how words in a sentence depend on each other.
- Semantic Parsing
  - Transforms text into a structured, machine-interpretable representation of its meaning, often as logical forms or frames.
  - Used for extracting actionable information from text and for question-answering systems.

Ambiguity

- Common structural ambiguity
  - Attachment ambiguity

    A sentence has an attachment ambiguity if a particular constituent can be attached to the parse tree at more than one place.

    Various kinds of adverbial phrases are subject to this kind of ambiguity

    Attachment ambiguity occurs when a word or phrase can grammatically attach to more than one part of a sentence, resulting in different interpretations

    It is especially common with **prepositional phrases (PPs) or** adverbial phrases
    - Ex. I saw the man with a telescope
      - I used a telescope to see the man (PP attaches to the verb "saw“) or
      - I saw a man who had a telescope (PP attaches to the noun "man“)
    - *We saw the Eiffel Tower flying to Paris.*

Ambiguity

- Coordination ambiguity

  Different sets of phrases are joined by a conjunction such as **and**.

  For example, the phrase old men and women can be

  - bracketed as [old [men and women]], referring to old men and old women,
  - or as [old men] and [women], in which case it is only the men who are old.

7

Syntactic Parsing

The CYK Algorithm

- *The membership problem*
  - Problem:

    Given a context-free grammar **G** and a string **w**
    - **G** = (V, ∑ ,P , S) where
      - V finite set of variables
      - ∑ (the alphabet) finite set of terminal symbols
      - P finite set of rules
      - S start symbol (distinguished element of V)
      - V and ∑ are assumed to be disjoint
    - **G** is used to generate the string of a language
  - Question:

    Is **w** in **L(G)**?

The CYK Algorithm Basics

- The Structure of the rules in a Chomsky Normal Form (CNF) grammar
  - Uses a “dynamic programming” or “table-filling algorithm”
  - Based on bottom-up parsing and requires first normalizing the grammar.

- Context-free grammar is in CNF if each rule has one of the following forms:
  - A → BC at most 2 symbols on right side
  - A → a, or terminal symbol
  - S → λ null string

Construct a Triangular Table

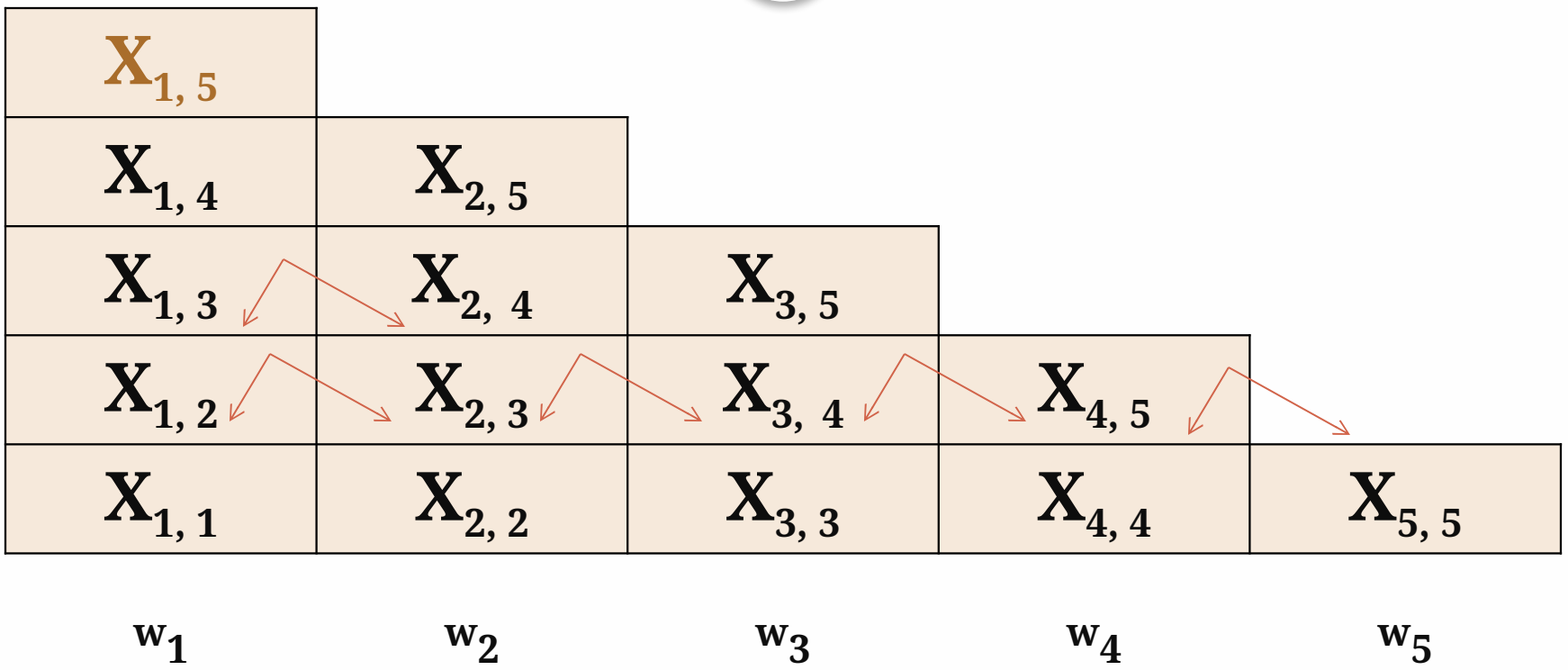
- Each row corresponds to one length of substrings
  - Bottom Row – Strings of length 1
  - Second from Bottom Row – Strings of length 2 ….
  - Top Row – string ‘w’
- $\mathbf{X_{i,\,i}}$ is the set of variables A such that
  $A \rightarrow w_i$ is a production of G
- Compare at most n pairs of previously computed sets
  $(\mathbf{X_{i,\,i}}, \mathbf{X_{i+1,\,j}}), (\mathbf{X_{i,\,i+1}}, \mathbf{X_{i+2,\,j}}) \ldots (\mathbf{X_{i,\,j-1}}, \mathbf{X_{j,\,j}})$

Construct a Triangular Table

$X_{1,5}$				
$X_{1,4}$	$X_{2,5}$			
$X_{1,3}$	$X_{2,4}$	$X_{3,5}$		
$X_{1,2}$	$X_{2,3}$	$X_{3,4}$	$X_{4,5}$	
$X_{1,1}$	$X_{2,2}$	$X_{3,3}$	$X_{4,4}$	$X_{5,5}$
w_1	w_2	w_3	w_4	w_5

Table for string '**w**' that has length 5

Construct a Triangular Table



Looking for pairs to compare

Example CYK Algorithm

- Show the CYK Algorithm with the following example
  - CNF grammar **G**

    S → AB | BC

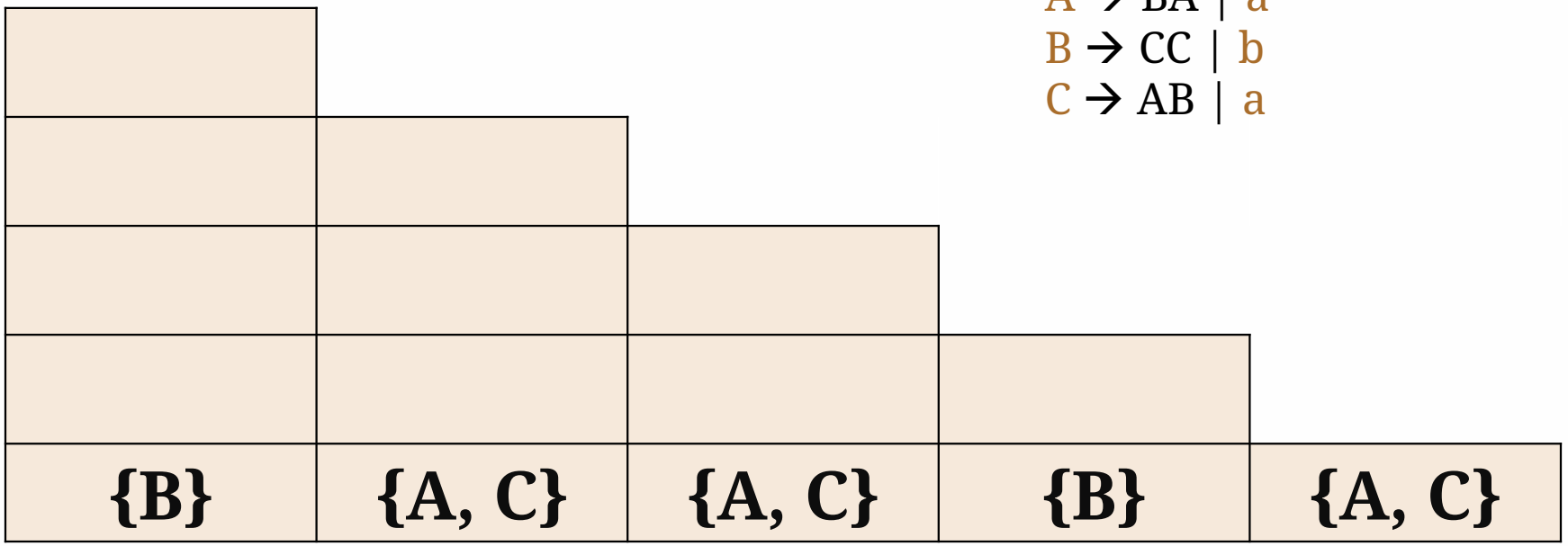
    A → BA | a

    B → CC | b

    C → AB | a
  - **w** is baaba
  - Question Is **baaba** in L(G)?

Constructing The Triangular Table

S → AB | BC
A → BA | a
B → CC | b
C → AB | a



b **a** **a** **b** **a**

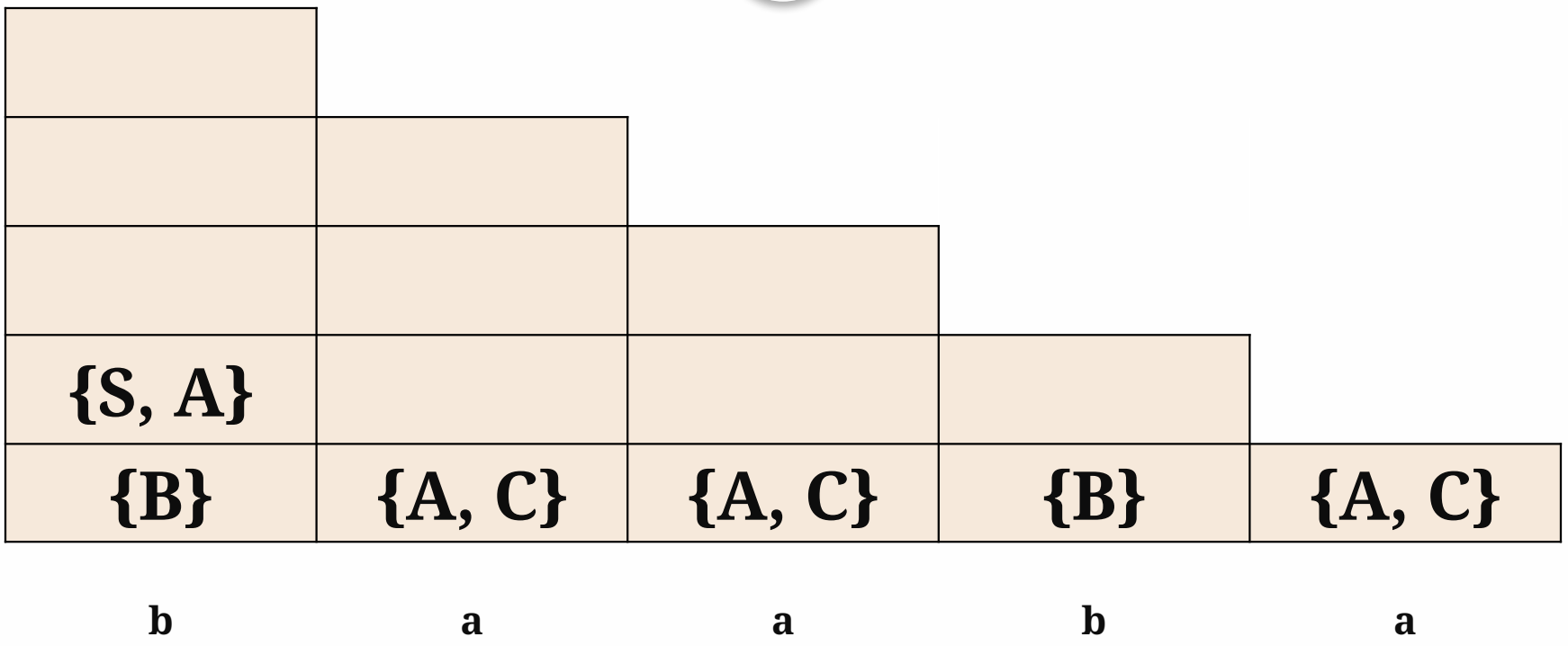
Calculating the Bottom ROW

Constructing The Triangular Table

- $X_{1,2} = (X_{i,i}, X_{i+1,j}) = (X_{1,1}, X_{2,2})$
- ➔ {B}{A,C} = {BA, BC}
- Steps:
  - Look for production rules to generate BA or BC
  - There are two: S and A
  - $X_{1,2}$ = **{S, A}**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

Constructing The Triangular Table

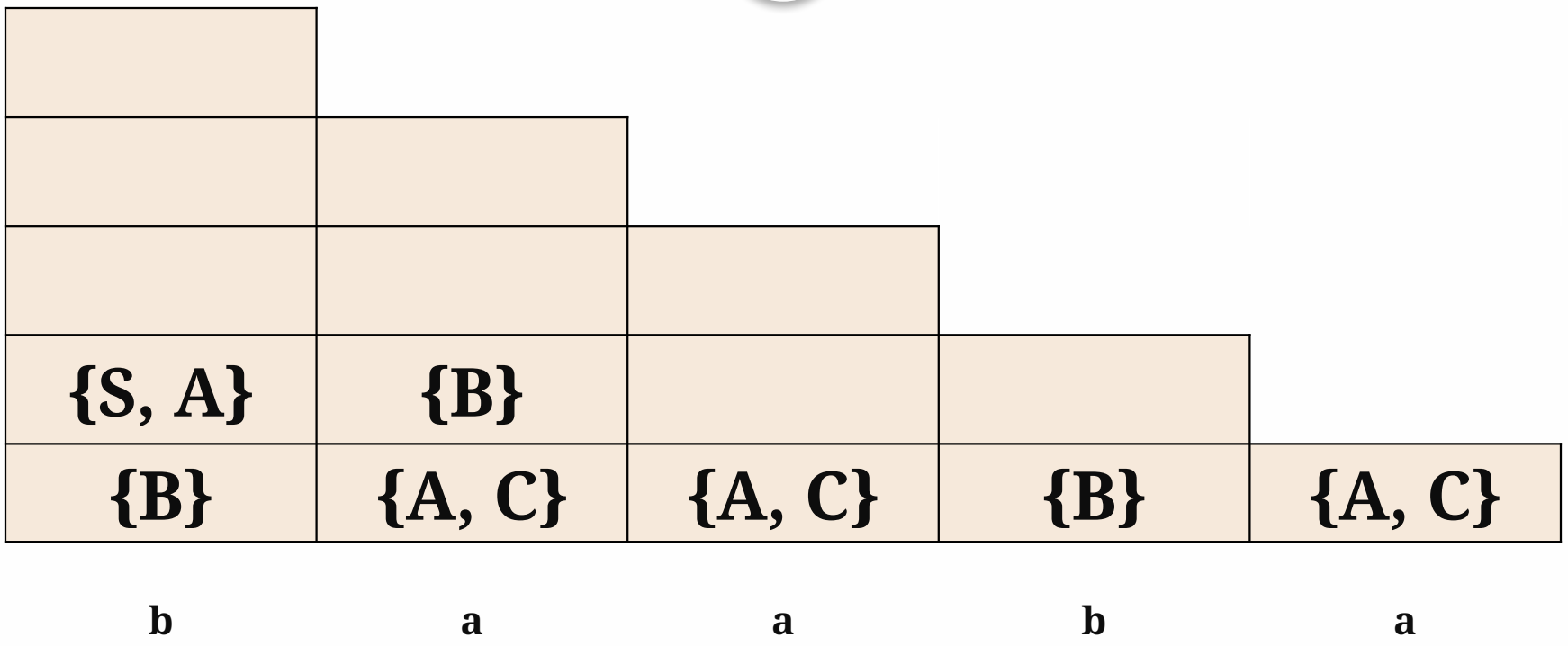


Constructing The Triangular Table

- $X_{2,3} = (X_{i,i}, X_{i+1,j}) = (X_{2,2}, X_{3,3})$
- ➔ {A, C}{A,C} = {AA, AC, CA, CC} = Y
- Steps:
  - Look for production rules to generate Y
  - There is one: B
  - $X_{2,3} = \{B\}$

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

Constructing The Triangular Table

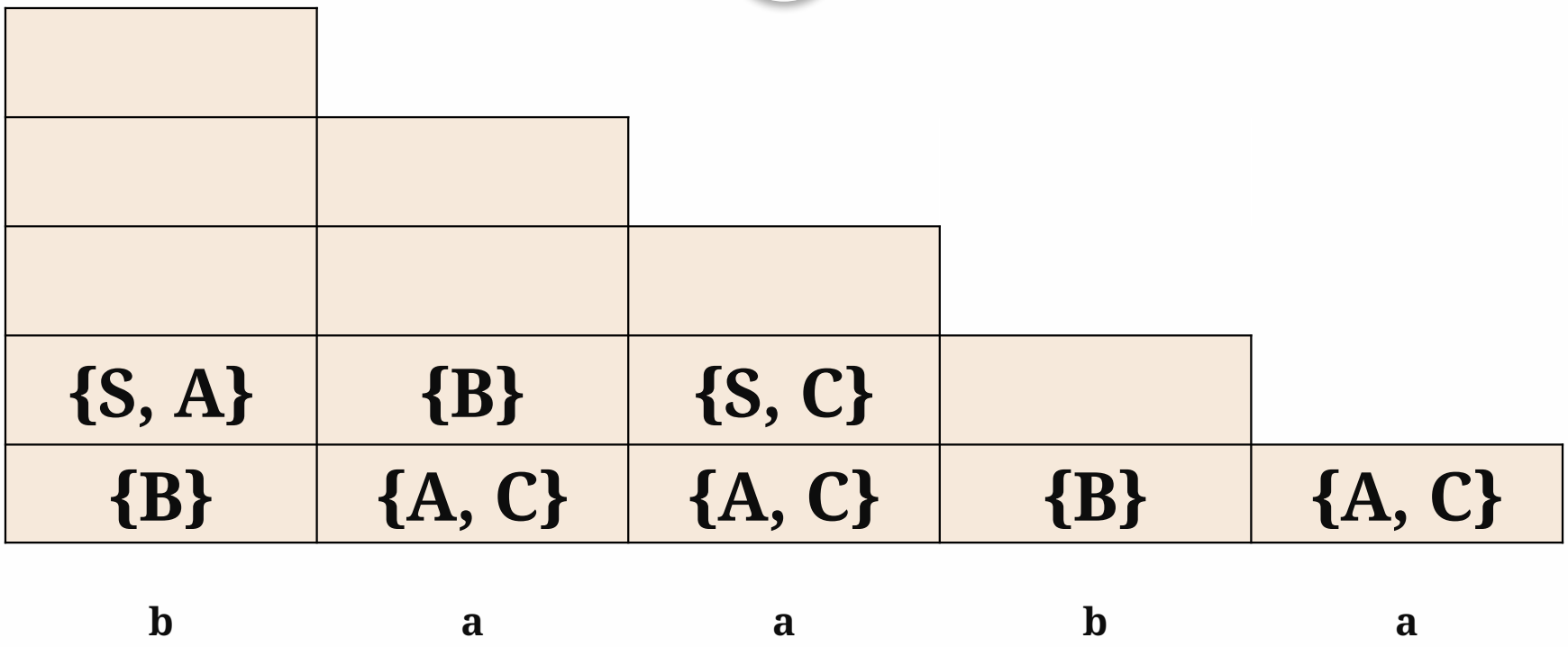


Constructing The Triangular Table

- $X_{3,4} = (X_{i,i}, X_{i+1,j}) = (X_{3,3}, X_{4,4})$
- ➔ {A, C}{B} = {AB, CB} = Y
- Steps:
  - Look for production rules to generate Y
  - There are two: S and C
  - $X_{3,4} = \{S, C\}$

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

Constructing The Triangular Table

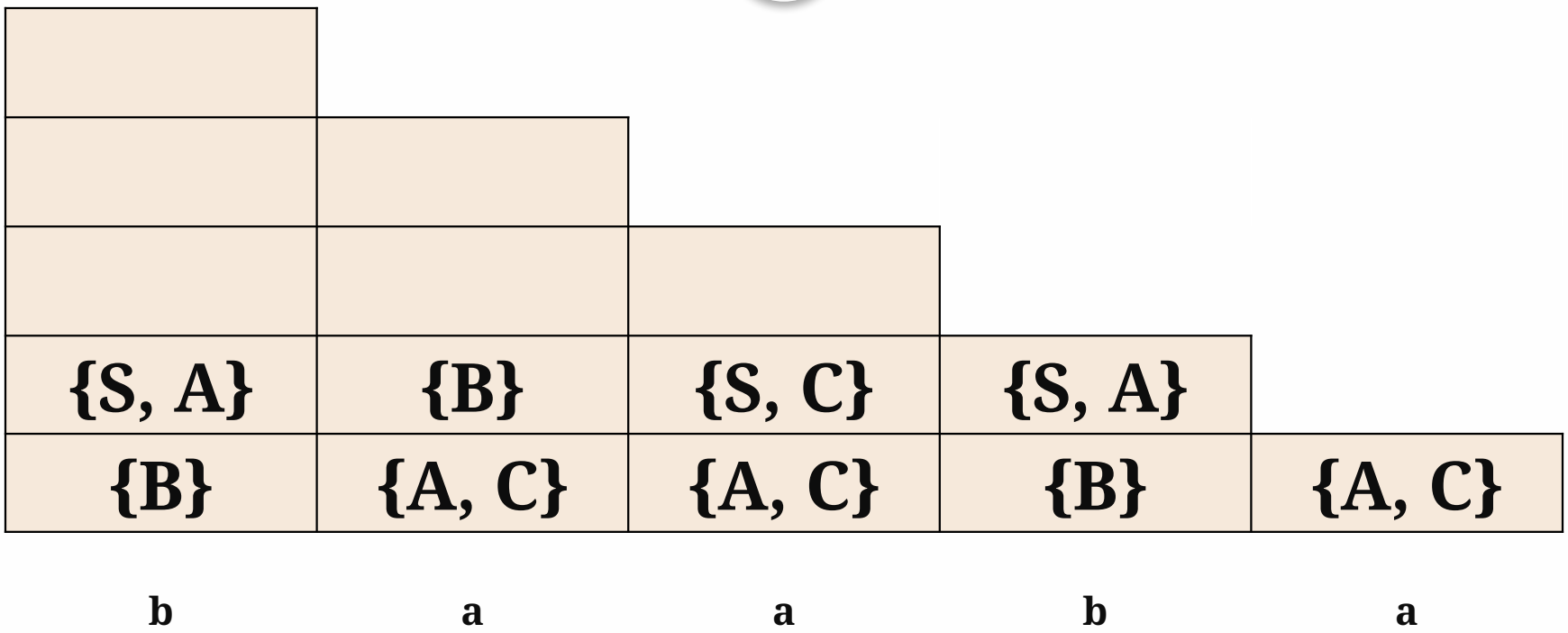


Constructing The Triangular Table

- $X_{4,5} = (X_{i,i}, X_{i+1,j}) = (X_{4,4}, X_{5,5})$
- ➔ {B}{A, C} = {BA, BC} = Y
- Steps:
  - Look for production rules to generate Y
  - There are two: S and A
  - $X_{4,5} = \{S, A\}$

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

Constructing The Triangular Table



Constructing The Triangular Table

- $X_{1,3} = (X_{i,i}, X_{i+1,j})\ (X_{i,i+1}, X_{i+2,j})$

  $= (X_{1,1}, X_{2,3})\ , (X_{1,2}, X_{3,3})$

- ➔ {B}{B} **U** {S, A}{A, C}= {BB, SA, SC, AA, AC} = Y
- Steps:
  - Look for production rules to generate Y
  - There are NONE: S and A
  - $X_{1,3} = \emptyset$
  - **no elements in this set (empty set)**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

Constructing The Triangular Table

Ø				
{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

Constructing The Triangular Table

- $X_{2,4} = (X_{i,i}, X_{i+1,j})\ (X_{i,i+1}, X_{i+2,j})$

  $= (X_{2,2}, X_{3,4}), (X_{2,3}, X_{4,4})$

- ➔ {A, C}{S, C} **U** {B}{B}= {AS, AC, CS, CC, BB} = Y
- Steps:
  - Look for production rules to generate Y
  - There is one: B
  - $X_{2,4}$ = **{B}**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

Constructing The Triangular Table

Ø	{B}			
{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

Constructing The Triangular Table

- $X_{3,5} = (X_{i,i}, X_{i+1,j})\ (X_{i,i+1}, X_{i+2,j})$

  $= (X_{3,3}, X_{4,5})\ , (X_{3,4}, X_{5,5})$

- ➔ {A,C}{S,A} **U** {S,C}{A,C}

  = {AS, AA, CS, CA, SA, SC, CA, CC} = Y

- Steps:
  - Look for production rules to generate Y
  - There is one: B
  - $X_{3,5}$ = **{B}**

S → AB | BC
A → BA | a
B → CC | b
C → AB | a

Constructing The Triangular Table

Ø	{B}	{B}		
{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}
b	a	a	b	a

Final Triangular Table

{S, A, C}	← $X_{1, 5}$			
Ø	{S, A, C}			
Ø	{B}	{B}		
{S, A}	{B}	{S, C}	{S, A}	
{B}	{A, C}	{A, C}	{B}	{A, C}
b	**a**	**a**	**b**	**a**

- Table for string '**w**' that has length 5
- The algorithm populates the triangular table

Example (Result)

- Is baaba in L(G)?

Yes

We can see the S in the set X_{1n} where 'n' = 5

We can see the table

the cell X_{15} = (S, A, C) then

if S ϵ X_{15} then baaba ϵ L(G)

Statistical Parsing

Probabilistic Parsing

- One crucial use of probabilistic parsing is to solve the problem of disambiguation.
- A probabilistic parser offers a solution to the problem:
  - compute the probability of each interpretation and choose the most probable interpretation.

Probabilistic Parsing

- Probabilistic Context Free Grammar (PCFG)
  - A probabilistic grammar which favors more common rules
  - Augment each rule with its associated probability
  - Modify the parser so that it returns most likely parse (CKY Algorithm)
  - Problems and augmentations to the basic model

Probabilistic Context-Free Grammars

- A probabilistic context-free grammar is defined by four parameters, with a slight augmentation to each of the rules in R
  - N a set of non-terminal symbols (or variables)
  - S a set of terminal symbols (disjoint from N)
  - R a set of rules or productions,

    each of the form A -> b [p],

    where A is a non-terminal, b is a string of symbols from the infinite set of strings, and p is a number between 0 and 1 expressing P(b/A)
  - S a designated start symbol

Probabilistic Context-Free Grammars

- PCFG differs from a standard CFG by augmenting each rule with a conditional probability: A->b [p]
  - Here p expresses the probability that the given non-terminal A will be expanded to the sequence b.
  - p is the conditional probability of a given expansion b given the left-hand-side (LHS) non-terminal A.
  - We can represent this probability as

    P(A->b) or as P(A->b/A) or as P(RHS/LHS)
- Thus, if we consider all the possible expansions of a non-terminal, the sum of their probabilities must be 1

$$\sum_{b} (A \rightarrow b) = 1$$

Probability Model

- The expansions for a given non-terminal sum to 1
  - VP -> verb .55
  - VP -> verb NP .40
  - VP -> verb NP NP .05

- Probability captures P(RHS | LHS)

Grammar	Prob	
S → NP VP	0.8	
S → Aux NP VP	0.1	+ 1.0
S → VP	0.1	
NP → Pronoun	0.2	
NP → Proper-Noun	0.2	+ 1.0
NP → Det Nominal	0.6	
Nominal → Noun	0.3	
Nominal → Nominal Noun	0.2 0.5	+ 1.0
Nominal → Nominal PP	0.2	
VP → Verb	0.5	+ 1.0
VP → Verb NP	0.3	
VP → VP PP	1.0	
PP → Prep NP		

Lexicon

Det → the | a | that | this
0.6 0.2 0.1 0.1

Noun → book | flight | meal | money
0.1 0.5 0.2 0.2

Verb → book | include | prefer
0.5 0.2 0.3

Pronoun → I | he | she | me
0.5 0.1 0.1 0.3

Proper-Noun → Houston | NWA
0.8 0.2

Aux → does
1.0

Prep → from | to | on | near | through
0.25 0.25 0.1 0.2 0.2

Using a PCFS

- A PCFS can be used to estimate a number of useful probabilities concerning a sentence and its parse trees
  - Probability of a particular parse tree (useful for disambiguation)
  - Probability of a sentence or piece of a sentence (useful for language modeling)

How?

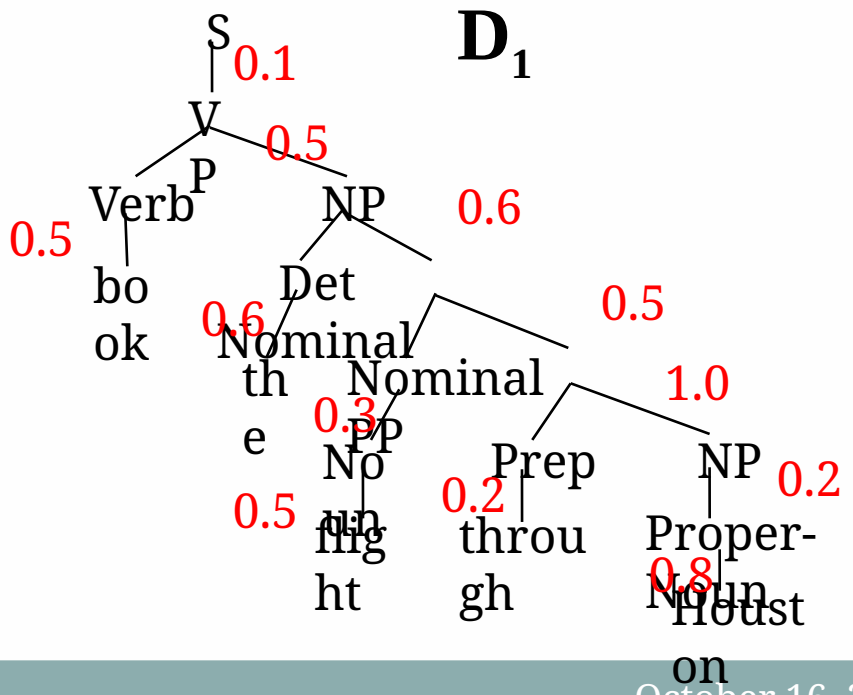
- A derivation (tree) consists of the set of grammar rules that are in the tree
- The probability of a tree is just the product of the probabilities of the rules in the derivation.

Sentence Probability

- Assume productions for each node are chosen independently.
- The probability of a derivation is the product of the probabilities of its productions.

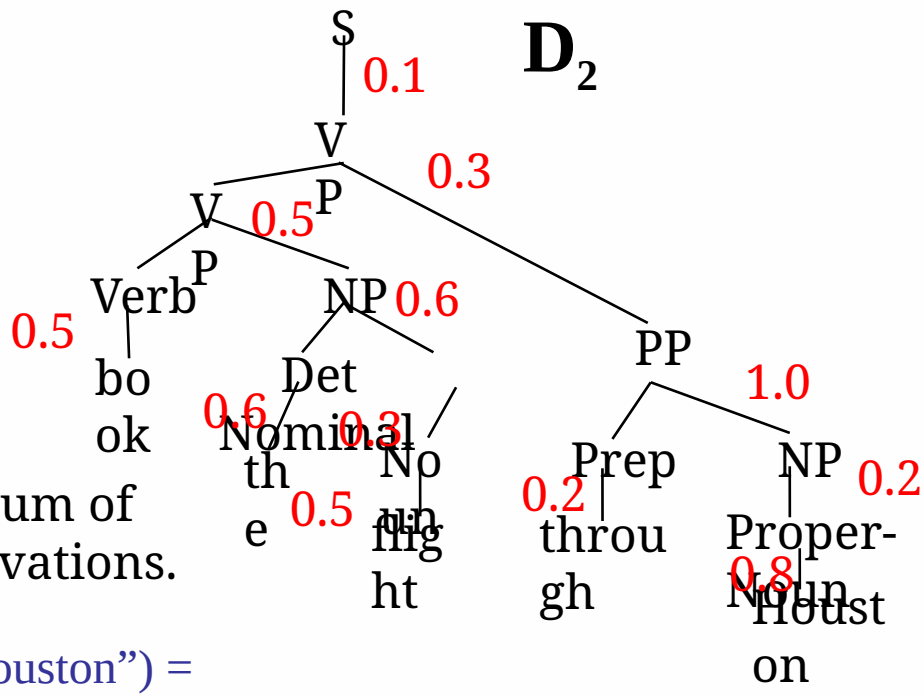
$P(D_1) = 0.1 \times 0.5 \times 0.5 \times 0.6 \times 0.6 \times 0.5 \times 0.3 \times 1.0 \times 0.2 \times 0.2 \times 0.5 \times 0.8$

$= 0.0000216$



Sentence Probability Cont…

$P(D_2)$ = 0.1 x 0.3 x 0.5 x 0.6 x 0.5 x
0.6 x 0.3 x 1.0 x 0.5 x 0.2 x
0.2 x 0.8
= 0.00001296



Probability of a sentence is the sum of the probabilities of all of its derivations.

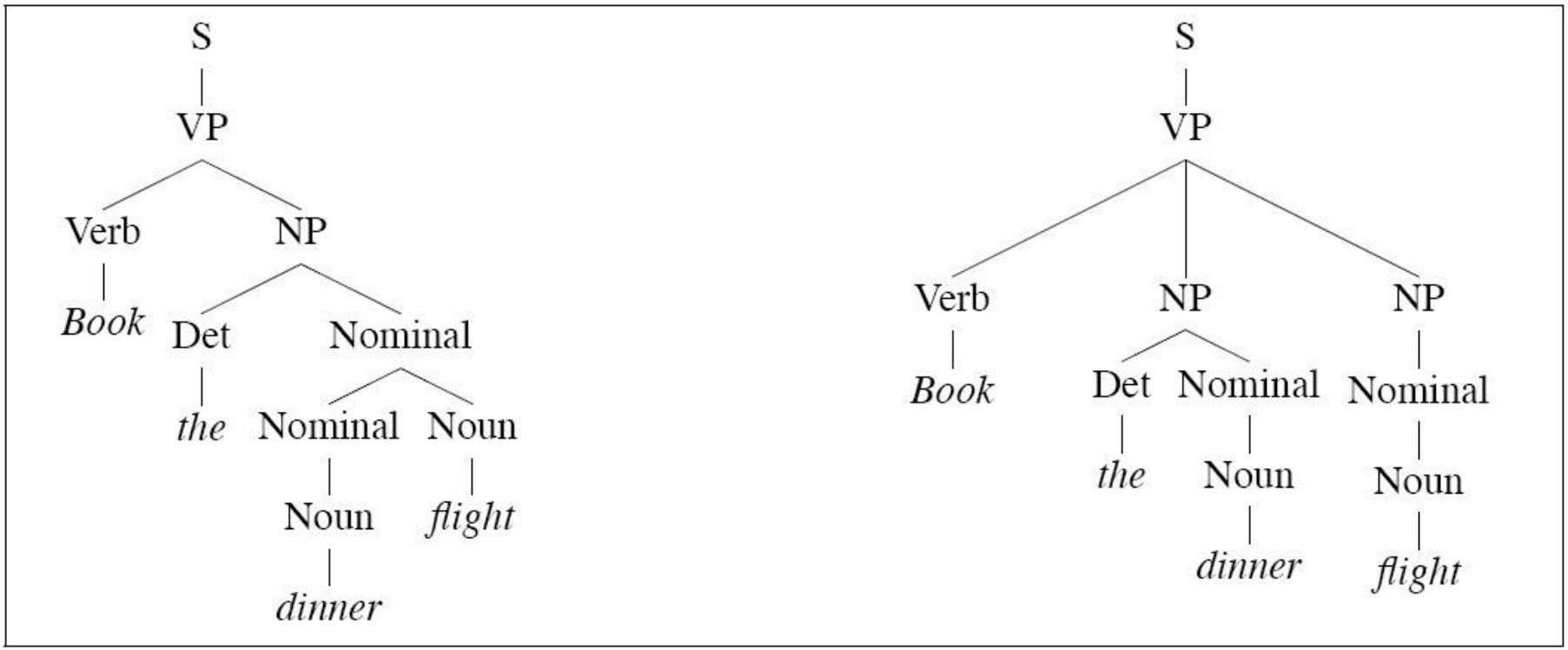
P("book the flight through Houston") =
$P(D_1) + P(D_2)$ = 0.0000216 + 0.00001296
= 0.00003456

PCFGs for Disambiguation

- A PCFG assigns a probability to each parse tree T of a sentence S.
- This attribute is useful in disambiguation.
- The probability of a particular parse tree T is defined as the product of the probabilities of all the *n* rules used to expand each of the n non-terminal nodes in the parse tree.
-

$$P(T,S) = \prod_{i=1}^{n} P(RSI_i / LSI_i)$$

PCFGs for Disambiguation: Example



The grammar is ambiguous

Probability Model

$$P(T,S) = P(T)P(S|T) = P(T);$$

since a parse tree includes all the words of the sentence P(S/T)=1

$P(T_{left})$ = .05 * .20 * .20 * .20 * .75 * .30 * .60 * .10 * .40 = **2.2×10^{-6}**

$P(T_{right})$ = .05 * .10 * .20 * .15 * .75 * .75 * .30 * .60 * .10 * .40 = **6.1×10^{-7}**

The left tree has higher probability than the right.

The disambiguation algorithm selects the parse with the highest PCFG probability.

PCFG Application

- PCFGs can be applied both to
  - disambiguate in syntactic parsing
  - to predict word in language modelling.
  - most likely parse tree derivation.
- These applications require that we are able to compute the probability of parse tree T for a given sentence S.

THANK YOU